A felsőfokú oktatás minőségének és hozzáférhetőségének együttes javítása a Pannon Egyetemen

EFOP-3.4.3-16-2016-00009

Cloud Programming

Zoltan Juhasz

University of Pannonia

2020

H-8200 Veszprém, Egyetem u.10.
H-8201 Veszprém, Pf. 158.
Telefon: (+36 88) 624-000
Internet: www.uni-pannon.hu

Contents

Preface v

1 Introduction 1

2 Technological developments of the past: the road to Cloud Computing 3

2.1 Access to and sharing computational resources 3

2.1.1 Limited resource systems – mainframes 3

2.1.2 Personal computers, Networking and Client-Server systems 4

2.1.3 Supercomputers 6

2.2 Programming client-server systems 8

2.2.1 Remote Procedure Call 11

2.2.2 Remote Method Invocation 12

2.2.3 The case for loosely coupled systems 14

2.3 Service oriented systems 15

2.3.1 XML based services – web services 16

2.3.2 RESTful services 18

2.4 Grid computing 18

2.5 Voluntary resource sharing 20

2.6 Hosting and the emerging utility computing model 21

2.7 Virtualisation 22

3 Cloud technology fundamentals 25

3.1 Core concepts 26

3.2 The economy of cloud 27

3.3 Cloud user and usage support 30

3.4 A bit of cloud history 30

3.4.1 Origins of the Amazon cloud service 31

3.4.2 Microsoft and the cloud 32

3.4.3 Google Cloud 33

3.4.4 IBM and Oracle 33

3.5 Pricing 33

3.6 Programming 33

4 Amazon AWS 35

4.1 IaaS Layer Services 37

4.1.1 Compute services 37

4.1.2 Storage Services 38
4.1.3 Network Services 39
4.2 PaaS Layer Services 41
4.2.1 AWS Elastic Beanstalk 41
4.3 SaaS Layer Services 41
4.4 Service control and APIs 42
5 Google Cloud Platform 44
5.1 GCP Services in the IaaS Layer 45
5.2 PaaS Layer Services 47
5.3 SaaS Layer Services 47
6 Using the Google Cloud Platform 48
6.1 The GCP Project 48
6.2 Ways to access GCP resources 48
6.3 Using the Compute Engine 50
6.4 Using Cloud Storage 51
7 Large-scale job execution in the cloud 53
7.1 The emergence of Big Data 53
7.2 Data processing software frameworks 54
7.2.1 Batch and stream data processing 55
7.2.2 Apache Hadoop/MapReduce 55
7.2.3 Apache Spark 59
7.3 Executing MapReduce and Spark jobs on GCP 62
7.3.1 Running MapReduce jobs on GCP 62
7.3.2 Running Spark jobs on GCP 63
8 Data stream processing 65
8.1 Stream algorithms 65
8.2 Popular stream data processing frameworks 70
8.2.1 Spark Streaming 70
8.2.2 Apache Flink 72
8.2.3 Apache Storm / Trident 73
8.2.4 Apache Kafka 77
8.2.5 Apache Beam/GCP Dataflow 80
8.3 Comparison of stream frameworks 83
9 Cloud Functions 86
10 Service communication: Pub/Sub 91
10.1 Google Pub/Sub overview 91

10.2 Pub/Sub message flow 93
10.3 Message delivery example 94
10.4 Publishing and receiving messages in code 95
11 GCP service examples 99
11.1 Vision AI 100
11.2 Cloud Translation 101
11.3 Text-to-Speech 101
12 Cloud Service Orchestration 104
12.1 A complex Optical Character Recognition application example 104
13 Conclusions 111
Bibliography 112

Preface

Cloud technology is a relatively new but exciting area of computing. Several decades of research and development effort of academia and industry into distributed systems, networking, virtualisation, service-oriented computing turned into a practical reality providing virtually limitless computing and storage resources on which new type of applications and services can be provided to millions of users.

The purpose of this book is to provide an introduction to the programming aspects of cloud systems. As cloud systems are increasingly operated by highly-specialised data centres of large tech companies, it is the software were the real opportunities lie for the individual developers and small companies. The amount of material that we could possibly cover in this book is enormous; therefore, the focus is only on introducing the most fundamental concepts, methods and programming techniques to students, which should prepare them for further, more specialised study of selected specific areas.

The content of this book is strongly related to the structure and content of the Cloud Programming course offered to MSc students by the Faculty of Information Technology at the University of Pannonia in Veszprem, Hungary. While the book is meant to be introductory, working knowledge of Java programming, fundamental concepts of computer architecture, operating systems, distributed systems is a pre-requisite for this course. The chapters do not necessarily need to be read in the order they are presented in the book, but for readers not familiar with the topic, this might be the most suitable sequence.

While every care has been taken to create a correct manuscript, errors and omissions might be present. Any of these is the sole responsibility of the author. In hope of further improving this book, readers are welcome to send their feedback and comments to the author.

The support of the European Union and the Hungarian Government under Grant No EFOP-3.4.3-A.2.3 is gratefully acknowledged.

Dr Zoltan Juhasz

1 Introduction

Over the past decade, the "cloud" has become a household name. We store our data in the cloud, use cloud services such as Gmail, Amazon, Netflix, Spotify, various web shops, online news services, etc. every day without thinking about where they are exactly in the physical sense, how they are implemented and operated. We take their existence and availability for granted.

It took the computing and IT industry several decades of research and development, thousands of person-years of effort to create these systems, and above all, the cloud itself. We all know the buzzwords about the cloud: unlimited storage and computation, on-demand use, low cost, scalability, high-availability, elasticity or scaling, integration of services, and the like. We also know that somewhere in the world, there are large datacentres that are the backbone of the cloud infrastructure. These are mainly operated by large global corporations, due to the extremely high level of initial investment required to become cloud providers, as well as to be able to provide services at a truly global scale.

This book is about the cloud through the eye of a software developer. We will not be interested in the operational or daily management aspects of the cloud. What we will focus on is how this new 'medium' affects the software development practice, the way we create and deploy applications. From the following common cloud-related terms: Cloud Computing, Cloud Architecture, Cloud Programming, we will be looking at the last one, Cloud Computing in detail.

The aim of this book – and the course – is not just simply to provide practical programming tips; I hope that by the end of the course, students will have a perspective, a deeper understanding of the theoretical and technological drives that pushed our industry towards the Cloud. To appreciate the main challenges of modern computing and software development, Chapter 2 will provide a brief overview (or recap) of key developments that can be viewed as precursors or pre-requisites to the birth of Cloud Technology. We will touch topics such as resource sharing, developments in networking, various high-performance computing systems, the programming challenges of distributed systems, and so on. The maturity of these fields provided the foundations on which the Cloud could be built.

The next part of the book will look at the Cloud in a generic manner. Their key characteristics, architectural, operational and software-related aspects will be discussed. This will be followed by an overview of the key cloud providers, their short history, market share and relevance, as well as a brief list of their service offerings.

Then we turn to the programming aspects. We will start with the infrastructure components: storage, computation and data transfer (networking), as these are the core building blocks of any cloud system. This will be followed by higher-level services that make developers' life easier. Examples to these are application services, specific application platforms that are typically used in web-based system development.

Building on the concept of scalability, we will explore how to execute large computational tasks by using multiple execution services. Popular execution patterns, such as Map/Reduce will be explored. For the execution examples, but throughout the book, in general, practical examples will be given based on the Google Cloud Platform (GCP). The reason for this is the generous support of Google for several years in providing free access for our students to their cloud system, and the level of support and documentation available for the interested students. In the final chapters of the book, streaming data processing, Cloud Function, Publish/Subscribe communication, AI services and service orchestration will be discussed.

The main goal of the book is to help develop a general understanding and appreciation of cloud computing and to equip students with a sufficient level of knowledge of technologies and software techniques on which to build upon in their future career.

In our MSc course, the material is accompanied by practical, hands-on programming workshops during which students can execute and modify existing sample cloud applications as well as gain experience in developing programs on their own. Readers who work on the material independently should try to find sample programs relevant for the given chapter.

2 Technological developments of the past: the road to Cloud Computing

The Holy Grail of computing is a system that has exceptionally high performance, it is free or very inexpensive to use, accessible by anyone, reliable and pervasive, it provides fast, effective and simple ways for application development and deployment, and at the same time provides maximum user experience and usability for end users. We have not found this magic system yet but the Cloud is probably the one among the existing candidates that is the closest to it.

The goal of this chapter is to introduce the problems typically found in large computing systems and overview various technological solutions invented in the past decades for solving them. It is also our aim to illustrate that Cloud Computing is not entirely new, unique concept, but rather the result of decades of systematic research and development in distributed computing systems.

The overview of technological developments in this chapter will focus no so much on the technical details of each particular technology (we assume the readers are already familiar with these fundamental topics) but on the problems they tried to solve, the way they did it, and to draw up a conceptual path (by "joining the dots") that leads to the emergence of cloud computing. At the end of each subsection, we will emphasise how a given piece of technology or its fundamental concepts find their place in modern cloud computing systems.

2.1 Access to and sharing computational resources

The first central problem area that we examine is what we call the *problem of resource sharing*. A computing system should be owned and operated in a way that it minimises its cost, maximises its utilisation and the performance observed by the users. Unfortunately, these are conflicting needs that are hard to satisfy simultaneously. Let us look back to the history of computing and see in what form and level the issue of resource sharing appeared in various computing systems.

2.1.1 Limited resource systems – mainframes

In the early days of computing, computers were large, expensive and scarce, typically located in specific computing facilities. These *mainframe computers* (see Figure 2-1) required specially trained technical personnel for operating and programming the machines. The computers executed a limited number of special-purpose programs dedicated to particular research or business problems. Most systems could only execute one program at a time, so *execution time slots had to be reserved* in advance. Since the computers were centrally hosted and managed, the system administrators were able to create optimal program execution schedules that kept the utilisation rate of the computer high (consequently, its effective cost low). Note that the *unit operating cost* of computer system depends on its utilisation rate. If a system whose initial cost is, say, 10 000 USD operates 100 000 hours (approx. 11 years) at 100% utilisation, the cost of 1 hour program execution is 0.1 dollar. If the same system is idle 90% of the time, the effective program execution cost increases to 1 USD/h. Therefore, keeping utilisation rate high is an important goal in system operation.

Later technological improvements resulted in time-shared or time-sliced operating modes, in which users could use the computer at the same time via *terminals*. Despite the continuous improvements, these centralised mainframe systems had several disadvantages for end users:

- The shared nature of the computer meant that users had to wait for one another; programs could not be executed before the previous running instance had finished.

- In time-sliced multiplexed mode, each user program executed at a reduced speed. The performance decrease was proportional to the number of simultaneously connected users.
- The central administration resulted in strict rules to follow, which affected how and when the system could be accessed and what programs could be executed.

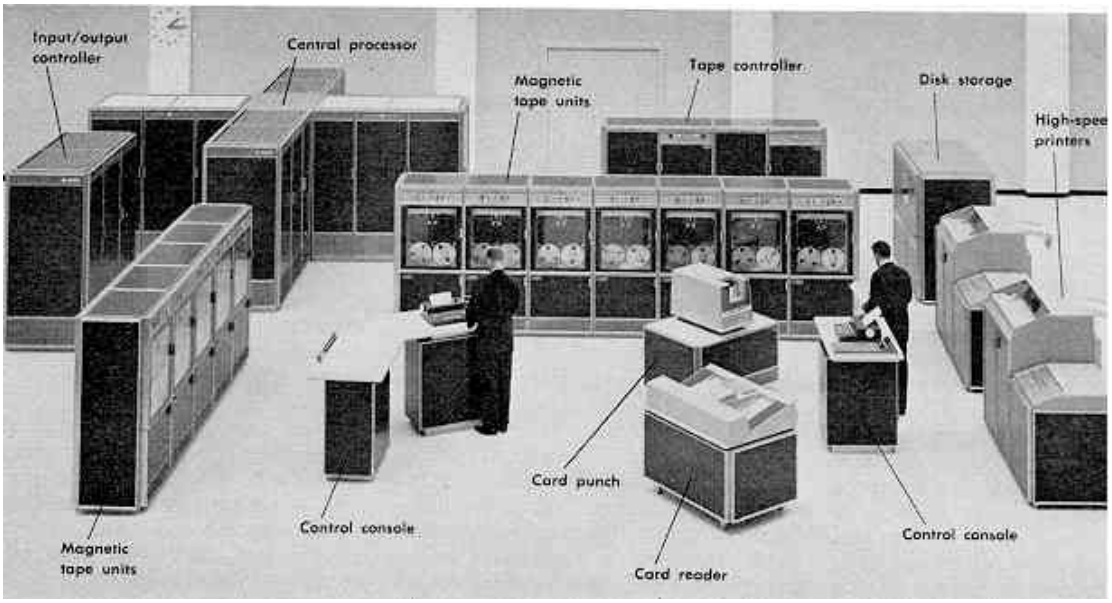

Figure 2-1 Typical mainframe computer system in the 1960s.[1]

Note the architecture diagram of the system (Figure 2-2) showing how terminals were connected to the central server. It resembles a modern networked computer system. The difference is that the terminals are not real computers; they are only devices with input and output capabilities attached to the central computer facility.

2.1.2 Personal computers, Networking and Client-Server systems

The Personal Computer (PC) revolution starting in the 1980s brought a radical shift in attitude to shared computer systems. As a result of miniaturisation (mass manufacturing of integrated circuits), the cost of computers dropped to the level that allowed first small businesses and later individuals to own a computer. One of the main argument in favour of the PC was that users no longer had to share their computer with others; they could use it whenever they want and for whatever they wish. The fact that users had to train themselves to be able to operate, administer and manage the computers was not advertised loudly…

With the computers came the need for peripherals, too. Users started to buy small printers, tape backup devices, and so on. Typically, these devices were used relatively infrequently. In a business environment, the additional cost of employees using private printers was coupled with the problems of data sharing. Individual PCs stored their data on their local hard drive. People working on different stages of a larger project had to copy and carry around files required by other colleagues as input. Not much changed as technology improved, data transfer moved on from floppy disks to CDs.

[1] Image source: https://alness.gnomio.com/pluginfile.php/209/mod_resource/content/1/On-line%20Resources/C%20Systems%20Int2/page_25.htm

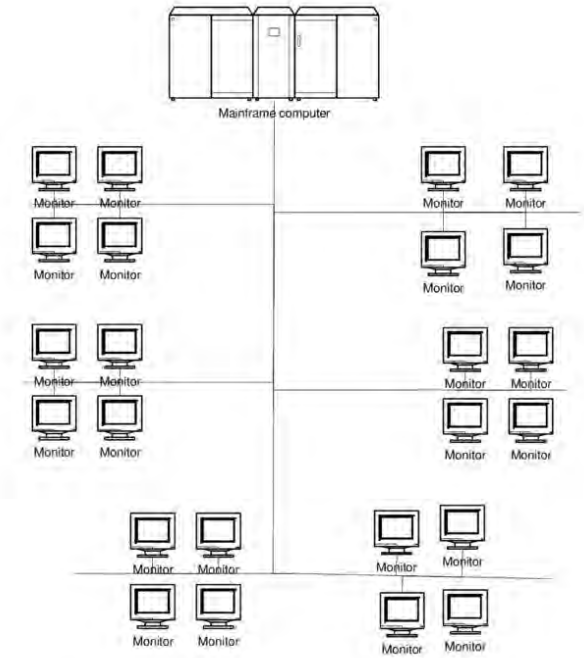

Figure 2-2 Mainframe system architecture with multiple terminals. [2]

Fortunately, along with the PC revolution, computer network technology became affordable, too. Local Area Networks allowed PCs to communicate with one another, which fuelled the development of middleware software packages facilitating data and peripheral device sharing. One successful such product was Novell NetWare that allowed small businesses to create print servers (use a smaller number of printers with much higher utilisation rate) and shared directories on which users could store data to be shared with others.

Since the 1980s, networking technology developed at a phenomenal speed. Ethernet networks have become commonplace, with large available bandwidth increases every couple of years, as shown in Figure 2-3. Following local area networks, geographically distant computers could be connected soon, using Wide Area Networks. Building on various segregated 'island' computer networks, and adopting the TCP/IP and Domain Name System technologies, the global Internet network was formed, onto which most countries connected by the end of the 1990s.

After a surge in non-shared computers, most institutes realised the need for sharing, both for financial and business efficiency reasons. Networking provided the underlying connection fabric for creating distributed computing systems. The simplest form of a distributed system is the client-server architecture, illustrated in Figure 2-4, in which a centralised server provides some well-defined functionality that multiple clients can access. Servers service client requests according to their implemented functionality. Common examples of server types include FTP, email, time, database and web servers. It must be emphasised that servers *specialise* for executing requests relating to a particular piece of functionality or application type. Clients must know the access details and usage

[2] Image source: E. Malmborg and B. Sundgren, "Integration of statistical information systems-theory and practice," *Seventh International Working Conference on Scientific and Statistical Database Management*, Charlottesville, VA, USA, 1994, pp. 80-89, doi: 10.1109/SSDM.1994.336959.

particulars of the server to be able to use a particular server. Client-server systems present interesting programming challenges, which we will examine in the following chapter.

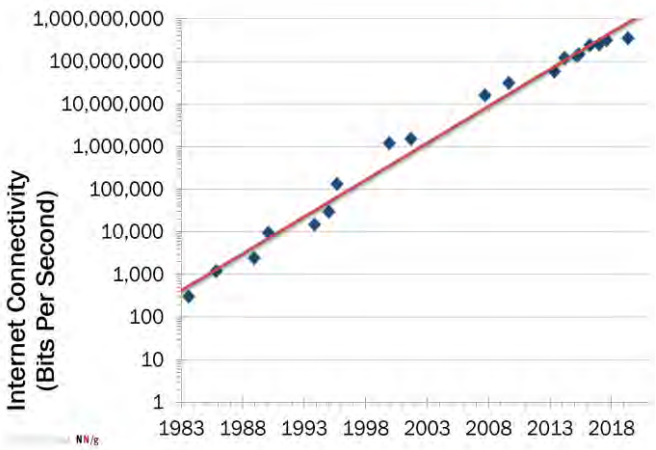

Figure 2-3 Increase of network bandwidth over the years.[3]

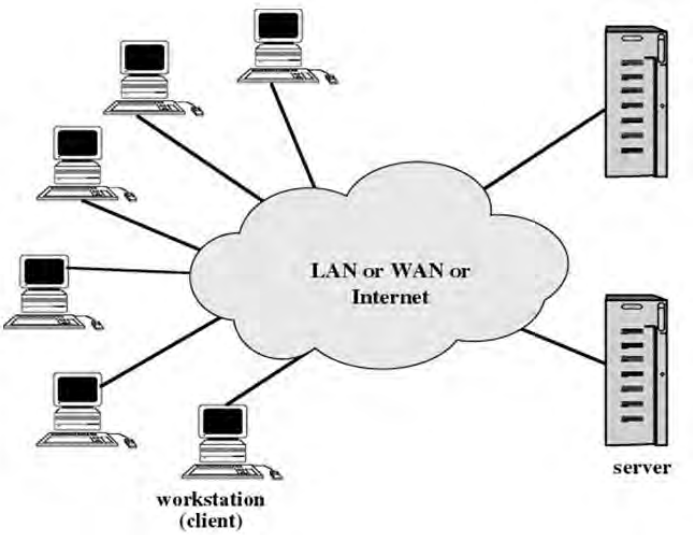

Figure 2-4 A generic client-server architecture[4]

2.1.3 Supercomputers

Although personal computers certainly destroyed the mainframe computer industry, it remained and fact that not every computing task could be solved with personal computers. For problems that were exceptionally demanding in a computational sense, special, much more powerful computers were created. These *supercomputers* took over the role of the former mainframe computers. Supercomputers, per definition, are the most powerful computers of the world at a given time. They are very expensive, consequently located at centralised supercomputer centres. Large countries have several supercomputer centres, small ones might have one or none at all. Access to supercomputers is limited, only a selected and approved set of users can run programs on them. Program execution is

[3] Image source: https://www.nngroup.com/articles/law-of-bandwidth/
[4] Image source: https://slideplayer.com/slide/5708626/

normally follows the *batch execution mode*, i.e. a program reads its input from files, executes then stores the results in output files. Interactive program execution is normally not supported. User programs (called *jobs*) are executed when all required resources (processors, memory, storage) are available for the task. Consequently, supercomputer systems employ queuing systems for job execution. Users *submit* their jobs to one of several execution queues and wait for the program to run and produce results. Note the similarity to early mainframe systems where users had to reserve computer time to run their programs. Sharing supercomputers is a necessity in order to achieve high utilisation rate that will keep operating costs at the minimum.

The performance of supercomputers is measured by the number of floating points operations (*flop*) they can execute in one second (*flop/s*). Performance is normally measured in multiples of thousand flops, such as Mflop/s (10^6), Glop/s (10^9), Tflop/s (10^{12}), Pflop/s (10^{15}), etc. The first computer that was considered as a supercomputer was the CDC 6600 manufactured by Control data Corporation, followed by the Cray-1 from Cray Research. The first system that reached the 1 Gflop/s level was the Cray Y-MP The next milestone, 1 Tflop/s, was reached by the ASCI Red system. Petaflops performance was first achieved by IBM Roadrunner computer. The key hardware properties of notable supercomputers are listed in Table 1. At the time of this writing, work is under way to create the first Exaflop/s supercomputer.

Table 1 Notable supercomputer systems and their system specifications

Supercomputer	Key specifications	
CDC 6600 (1964)	CPU: Memory: Performance:	60-bit processor @ 10 MHz Up to 982 kB (131000 x 60 bits) 2 MIPS (million instructions per second)
Cray-1 (1976)	CPU: Memory: Performance:	64-bit processor @ 80 MHz 8.39 MB (up to 1 048 576 words) 160 MFlop/s
Cray Y-MP (1988)	Processors: Memory: Performance:	8 vector processors @ 167 MHz 512 MB 2.144 GFlop/s
ASCI Red (1997)	Processors: Memory: Performance:	9298 x Pentium II Xeon Core Processors 1.212 terabytes 1.3 TFlop/s
IBM Roadrunner (2008)	Processors: Memory: Performance:	12,960 IBM PowerXCell 8i and 6,480 AMD Opteron dual-core processors 103.6 terabytes 1.042 PFlop/s
Tianhe-2 (2013)	Processors: Memory: Performance:	32,000 Intel Xeon E5-2692 12C @ 2.200 GHz and 48,000 Xeon Phi 31S1P 1345 terabytes 33.86 PFlop/s
Summit (2018)	Processors: Memory: Performance:	9,216 POWER9 22-core CPUs and 27,648 NVIDIA Tesla V100 GPUs 4,608 x 600 GB 200 PFlop/s
Fugaku (2020)	Processors: Memory: Performance:	158,976 Fujitsu A64FX CPU (48 cores) 4,866,048 GB 415 PFlop/s

Early supercomputer systems were sequential computers executing a single stream of instructions. Their speed came from special design, faster semiconductor technology and higher clock speed that was available in traditional computers. Later on, special hardware acceleration techniques were used,

such as vector operations and internal pipelining. Modern supercomputers, on the other hand, are parallel computers often containing hundreds of thousands of processor cores. Parallel supercomputers can only achieve high performance if a carefully written and optimised parallel program is executed on them. Typically, Fortran, C, C++ languages are used for these programs with suitable parallel programming extensions, such as OpenMP [1] or MPI (Message Passing Interface) [2][3], CUDA [4].

Figure 2-5 The Cray-1 supercomputer

Figure 2-6 The Tianhe-2 supercomputer

Figure 2-7 The Summit supercomputer

Figure 2-8 The Fugaku supercomputer

2.2 Programming client-server systems

As mentioned earlier, the simplest form of distributed systems is the client-server architecture. In this section, we will look at the programming aspects of these systems. The technologies we overview here were all created with the intention to raise the abstraction level of programming, increase programmer productivity and – at the same time – increase the quality and maintainability of the created code. As a reminder, Figure 2-9 shows the architecture of a generalised client-server system. The role of the client is to send various requests to the server and wait for its response. The server continuously listens for incoming requests, interprets them, then services the request, generates and sends the response to the client.

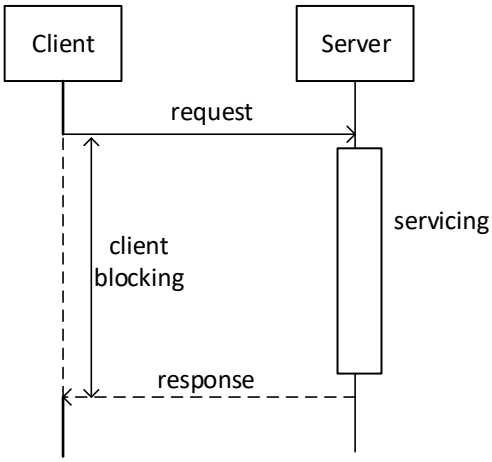

Figure 2-9 Request-response message exchange in a client-server system

The prototypical approach to programming such a system is by message passing using operating system functions that establish network connection to the server, and then send the assembled message and receive the response. An illustration of this, programmed in C, is shown in the following code section[5].

```
/****************** CLIENT CODE ****************/

#include <stdio.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <string.h>

int main(){
  int clientSocket;
  char buffer[1024];
  struct sockaddr_in serverAddr;
  socklen_t addr_size;

  /*---- Create the socket. The three arguments are: ----*/
  /* 1) Internet domain 2) Stream socket 3) Default protocol (TCP in this case) */
  clientSocket = socket(PF_INET, SOCK_STREAM, 0);

  /*---- Configure settings of the server address struct ----*/
  /* Address family = Internet */
  serverAddr.sin_family = AF_INET;
  /* Set port number, using htons function to use proper byte order */
  serverAddr.sin_port = htons(7891);
  /* Set IP address to localhost */
  serverAddr.sin_addr.s_addr = inet_addr("127.0.0.1");
  /* Set all bits of the padding field to 0 */
  memset(serverAddr.sin_zero, '\0', sizeof serverAddr.sin_zero);

  /*---- Connect the socket to the server using the address struct ----*/
  addr_size = sizeof serverAddr;
  connect(clientSocket, (struct sockaddr *) &serverAddr, addr_size);

  /*---- Read the message from the server into the buffer ----*/
```

[5] https://www.programminglogic.com/example-of-client-server-program-in-c-using-sockets-and-tcp/

```
  recv(clientSocket, buffer, 1024, 0);

  /*---- Print the received message ----*/
  printf("Data received: %s",buffer);

  return 0;
}
```

The same functionality implemented in Java looks like as follows.

```
import java.io.*;
import java.net.*;
import java.util.logging.Level;
import java.util.logging.Logger;

public class MyClient {
    private final int PORT = 10000;
    private Socket server;
    private BufferedReader in;
    private PrintWriter out;

    public static void main(String[] args) {
        new MyClient().start();
    }

    public void start(){
        try {
            System.out.println("Client is started...");
            // 1. establish connection with server
            server = new Socket("localhost", PORT);
            System.out.println("client connected... ");
            // 2. create stream instaces for communication
            in = new BufferedReader(
                    new InputStreamReader(server.getInputStream()));
            out = new PrintWriter(server.getOutputStream(), true);
            // 3. receive response
            System.out.println("Sending '" + msg + "' to server");
            out.println("HELLO");
            // 4. receive response
            String response = in.readLine();
            System.out.println("Response from server: " + response);
        } catch (IOException ex) {
            Logger.getLogger(MyClient.class.getName()).log(Level.SEVERE, null, ex);
        } finally{
            // 5. close communication channels
            try{
                if (in != null) in.close();
                if (out != null) out.close();
                if (server != null) server.close();
            }catch(IOException e){
                e.printStackTrace();
            }
        }
    }
}
```

Since message exchange is such a fundamental operational step of client-server systems, this sequence of instructions must be repeated many times in a large application. This leads to repeating

nearly identical code sections multiple times in the source code of an application. Developers would very quickly replace these by custom functions for handling message exchanges. Without standardisation though, each system would use an ad-hoc set of functions, parameter passing and error handling conventions that would make long-term code maintenance and interoperation very complicated.

2.2.1 Remote Procedure Call

It should come as no surprise that this problem was discovered very early by the distributed systems programming community and thus standardisation work began. The first proposal that became influential was the *Remote Procedure Call* (RPC) by Sun Microsystems [5]. The key philosophy of RPC is to present a programming interface for accessing and using a remote server that developers are already familiar with. This natural common abstraction is the procedure/function call well known for every programmer. If we can hide the details and complexities of connection setup, message packaging and transfer, programmers can focus more on program logic and simultaneously, the quality of the programs will increase. Such a standardised system can only work if there is a special "runtime" that performs these behind-the-scene tasks on behalf of the programmer. In distributed systems terminology, this "runtime" is called the *middleware*, a software layer sitting between the distributed application and the operating system. The role of the middleware layer is to create a common layer of abstraction known to all collaborating parties in the distributed system.

To access this middleware layer (the RPC runtime) we need to create special code sections, called *stub*, that connect the client and server code with the middleware [6]. Figure 2-10 illustrates the operation of the RPC call via the stubs. The stubs are generated by an RPC compiler.

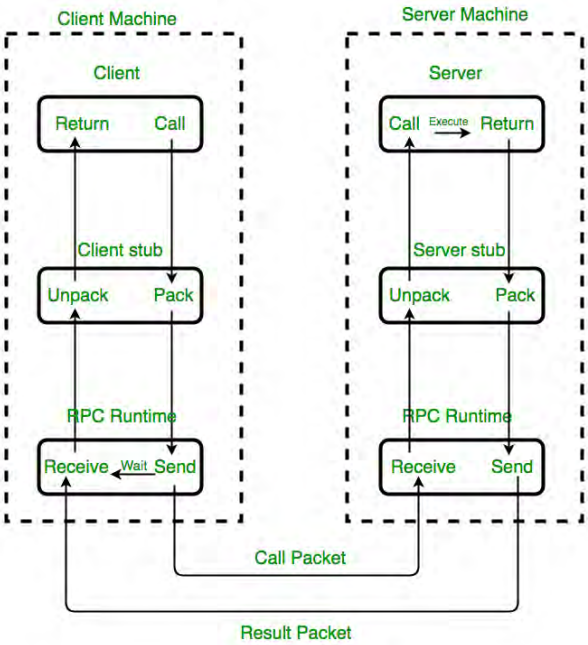

Figure 2-10 Implementation of the RPC mechanism

We ignore the implementation and compilation process here and only show a simplified code section illustrating the client server interaction.[6] The client connects to the server using the `clnt_create()` function, and on success executes a remote operation (printing a list of items) by calling the

[6] https://www.cprogramming.com/tutorial/rpc/remote_procedure_call_start.html

print_list_1() function. If we compare the previous socket-level implementation to this RPC version, the simplification is inevitable.

```
    CLIENT *cl;
    int *result;

    ...

    cl = clnt_create(argv[1], PRINTER, PRINTER_V1, "tcp");
    if (cl == NULL) {
        printf("error: could not connect to server.\n");
        return 1;
    }
    result = print_list_1(l, cl);
    if (result == NULL) {
        printf("error: RPC failed!\n");
        return 1;
    }
```

2.2.2 Remote Method Invocation

Java occupies a special position in distributed systems due to its platform-independence. The Java Virtual Machine provides an identical runtime environment to Java applications regardless of what hardware and operating system is used on a given computer. Since Java supports object persistence (serialisation), it is possible to send not only data but Java classes from one VM to another. This is the basis of the Java version (or re-implementation) of RPC, the Remote Method Invocation (RMI) technology [7].

The goal of RMI is to provide a remote method calling mechanism to Java programs. Since a Java program runs in the JVM and is isolated from the hardware, the standard Java method call mechanism is not suitable for accessing remote computers. The designers of RMI introduced a middleware layer that cooperates with the underlying operating system to build connections and transfer data to a remote party. In addition, since Java is a 100% object-oriented language, a special abstraction – a stub class – is introduced that represents the remote server to the client as a normal Java object during operation.

There is a bit of additional infrastructure required to get RMI working. Since the object persistence mechanism only transfers object state, an additional way must be provided to ship the compiled Java classes to the other party so that the remove VM's classloader can locate the class when loading. RMI uses HTTP servers to export class files. The last piece of machinery is the way clients and the server find each other in RMI. In contrast to the simple addressing method based on the IP address and port number, RMI uses a lookup mechanism based on interfaces. Remote servers are described by their interface that captures the functionality of the server. This means that clients must know the interface they want to use, as this interface will provide the public methods that a client can invoke. The use of interfaces provides an improved description of remote functionality than possible with port numbers. Now, a remote system can have arbitrary number of functionalities available to the clients, as long as they can be described by method signatures. Java's strong typing ensures that clients can only access and use services for which they were developed. The interaction of these components are shown in Figure 2-11.

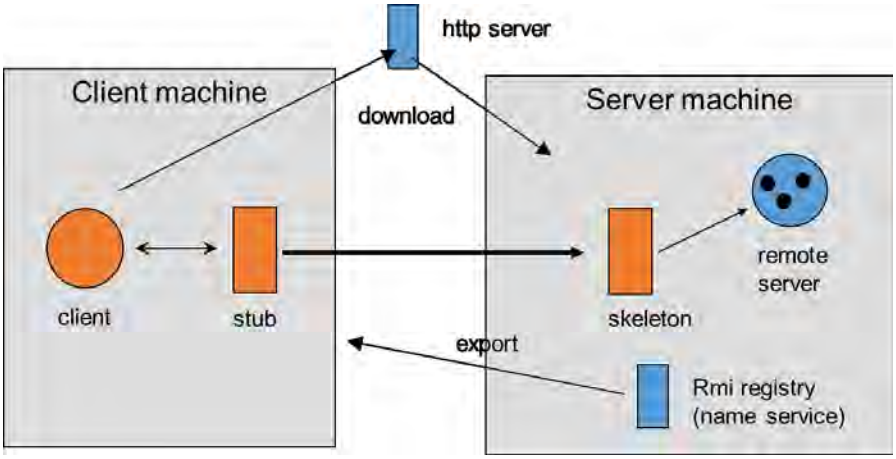

Figure 2-11 Components and architecture of the Java RMI system.

The following section shows the key steps of developing an RMI client-server application. Only the most important code sections will be shown. We use the example of a hypothetical remote bank that offers the client to deposit to, withdraw and transfer money from a bank account. The functionality of the server is described by a Java interface. Per specification, the remote server interface must be subclassed from the interface `Remote`, and each method must include a `throws RemoteException` in the declaration.

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface BankInterface extends Remote {
    int deposit(int amount) throws RemoteException;
    int withdraw(int amount) throws RemoteException;
    int getBalance() throws RemoteException;
}
```

The server will implement this interface with a Java class that is subclassed from `UnicastRemoteObject` and is registered into the RMIregistry on startup.

```
public class BankServer extends UnicastRemoteObject implements BankInterface {
    static int balance = 0;

    public BankServer() throws RemoteException {
        super();
    }

    public int getBalance() throws RemoteException {
        return balance;
    }

    // other methods are omitted

    public static void main(String[] args) {
        if (System.getSecurityManager() == null) {
            System.setSecurityManager(new RMISecurityManager());
        }
        String name = "BankServer";
        try {
            BankServer server = new BankServer();
            Naming.rebind(name, server);
            System.out.println("BankServer bound");
        } catch (Exception e) {
            System.err.println("BankServer exception: " + e.getMessage());
```

```
            e.printStackTrace();
        }
    }
}
```

The client will execute two major steps in the program. The first is to locate the remote server using the RMIRegistry and the second is to execute remote functionalities by local method calls on the stub/proxy object representing the server. The client code nicely illustrates the high level of abstraction and elegance of RMI in implementing client-server systems.

```
public class ATMClient{

    public static void main(String args[]) {
        if (System.getSecurityManager() == null) {
            System.setSecurityManager(new RMISecurityManager());
        }
        BufferedReader stdIn = new BufferedReader(new InputStreamReader(System.in));
        try {
            String name = "//" + args[0] + "/BankServer";
            BankInterface server = (BankInterface) Naming.lookup(name);
            ...
            boolean finished = false;
            while (! finished){
                ...
                balance = server.getBalance();
                ...
            }
        }catch(RemoteException ex){
            ex.printStackTrace();
        }
        ...
    }
}
```

2.2.3 The case for loosely coupled systems

The RPC/RMI programming abstraction is a great help in reducing complexity and program errors in distributed software development. It provides a natural extension of the well-known function/procedure/method call technique. It turns out, however, that it is a bit fragile in the presence of errors. Hiding the details of low-level communication, as well as 'pretending' the server is just another ordinary function of an object imply to many programmers that the system they work with behaves exactly the same way as a traditional locally executing program. Nothing could be further from the truth! Distributed servers can manifest various errors that are non-existing in sequential programs. The network connection can break down, the server can have a transient or fatal error, producing incorrect or no results. These situations are difficult to monitor and control with function calls. RMI uses the special Java exceptions (RemoteException) that signal the caller if something went wrong.

The real difficulty comes in long method call chains; when several distributed system components call each other to execute a complex task. Let's see an example in which a client C will call server A. Server A, in turn will call server B to help, which then will call server D. The client might be unaware of this implementation detail, as it only connected to server A. When no errors occur, the system works perfectly, but if an error happens, the state of the entire system can easily become corrupt. Think of a task that requires transactional integrity. How can you guarantee that a task executes only if all subtasks run correctly? If any member of the call chain generates an error, a domino effect starts and

exceptions (or error codes) will ripple through the system in a backward direction. In addition, if one method call hangs indefinitely (due to waiting or a deadlock situation), each caller method will be blocked as well. This mode of operation is called *strong coupling* in distributed systems. In other words, an error in one component directly affects the operation of another component. The complete failure of one server will block the entire system, and the tasks are no longer executable. As shown in Figure 2-12, there are many possible error locations in a distributed method call chain. It is rather difficult to locate the exact error cause from a single returned RemoteException instance; moreover, it is even more difficult to guarantee the state integrity of the system in such a situation.

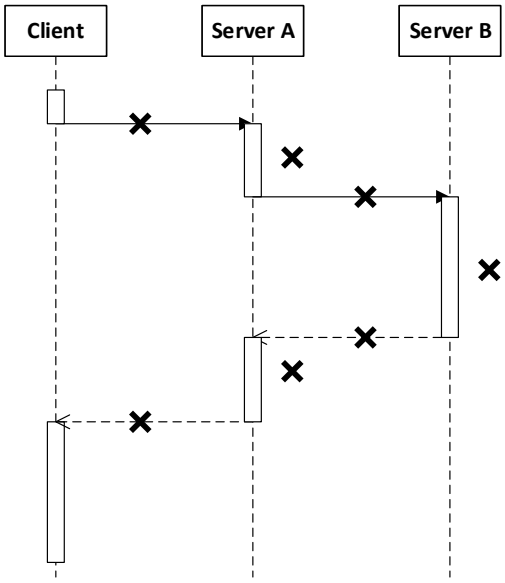

Figure 2-12 Possible error locations in an RPC/RMI call

As developer attempted to create larger and larger distributed systems, the scalability problems inherent to the strongly coupled method chaining abstraction became apparent. The industry decided to turn to loosely coupled communicating systems, relying on message-passing type communication and asynchronous execution. We will look these in more detail later.

2.3 Service oriented systems

The concept of service orientation grew out of client-server systems and the use of interfaces for describing server functionality. In order to make distributed systems more flexible, less tied to hardware, researchers were looking at ways to decouple the system executing the server code from the functionality the server represents. The concept of the service appeared as a suitable generalisation. A *service* is a piece of functionality implemented in hardware or software that provides a useful function for the client. A service is represented by a contract between the service provider and service customer. The key novelty in service-oriented software systems is the ability to find service providers based on their service instead of the server address that makes it physically executable. For instance, if I want to translate a document from, say, German to English, I am only interested in online services that can provide me with this translation. I am not concerned where they run (IP address), I only care that the translation will be done. How can this be achieved? We will see in the rest of this section.

Service-oriented systems are based on the following fundamental principles: (i) the service is described by an interface; (ii) there is a directory that mediates between clients and services (services register and clients look up services in the directory); (iii) Clients and services interact only for the duration of the service execution; (iv) clients and services use open, standardised technologies and protocols.

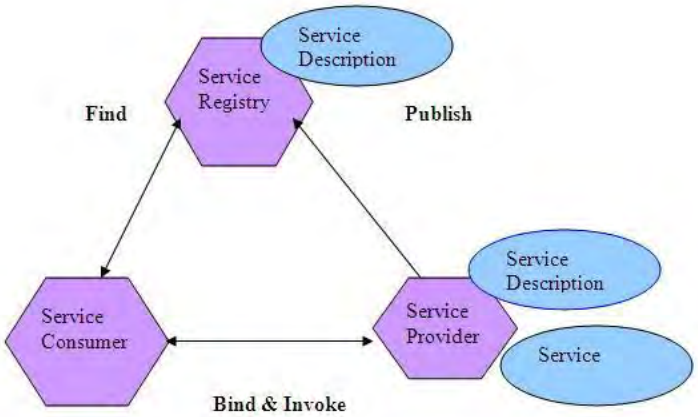

Figure 2-13 The generic architecture and operation of a service-oriented system.

Service-orientation started to become popular from the year 2000. As with any heterogeneous distributed system, the first key problem was to establish a platform-neutral communication and protocol framework. After this, the design and implementation of service-oriented systems could began. The promise of service-oriented computing was to create Internet-enabled "components" with well-defined functionality from which larger, applications that are more complex could be created by relatively straightforward integration processes.

The first candidate for middleware technology was XML. Interestingly, this is not a programming language but a mark-up language. XML, however, was always meant to be used by computers, not humans, carries a strict, rule-based approach to describing data and operations. The idea was to use XML to describe service functionality, plus use it in data communication and service invocations, too. For the latter, additional tools had to be developed that generated "stubs" for target programming languages from the service interfaces. Not surprisingly, the first XML-based service systems used an RPC-style service invocation.

2.3.1 XML based services – web services

When XML and service-orientation is used on top of the HTTP protocol, we talk about Web Service technology. Web services are services designed for invocation via the Internet using Web protocols. They were meant to create the backbone for new generation of business-to-business applications that will simplify performing business online and creating systems that can be easily maintained and modified in the future. Figure 2-14 illustrates the general architecture of a Web Service based service-oriented system.

Services are described by an XML-based WSDL (Web Service Description Language) interface descriptor that carries information about the location of the service, the functionality it offers and the way it can be accessed. This description is stored in a public directory, called UDDI (Universal Description, Discovery, and Integration). The directory is also XML-based. Clients use the UDDI directory to find suitable services, retrieve the necessary information for accessing the service, then

using XML-based SOAP (Simple Object Access Protocol) messages, connect to the service and execute chosen functions.

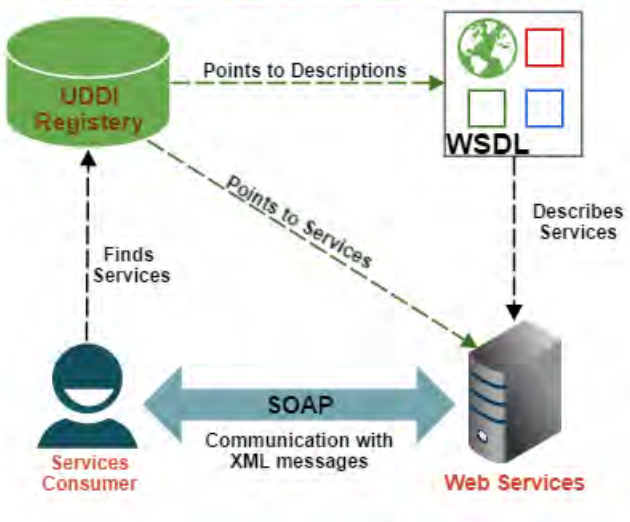

Figure 2-14 The Web Service architecture

Similar to web servers, the recommendation for web services is to implement stateless services. Stateless services are not dependent on the order of invocations, they do not alter their own state and always perform the same operation to any incoming request, which make them tolerant to many types of errors found in large distributed systems. Unfortunately, many services, especially client-dependent business services are stateful, which led to the support of state in services. The increasing complexities necessitated the extension of the simple WSDL-SOAP based client-service interaction technology. Several new layers and components appeared in the Web Service technology stack (as illustrated in Figure 2-15) that were responsible for various additional operational functionality (e.g. managing distributed transactions, providing context, orchestrating services, describing business workflows, etc.). This, unfortunately resulted in such a complex and large technology system that is almost impossible to understand. The set of Web Service specifications, colloquially called the "WS-* stack" contains over 70 specifications.

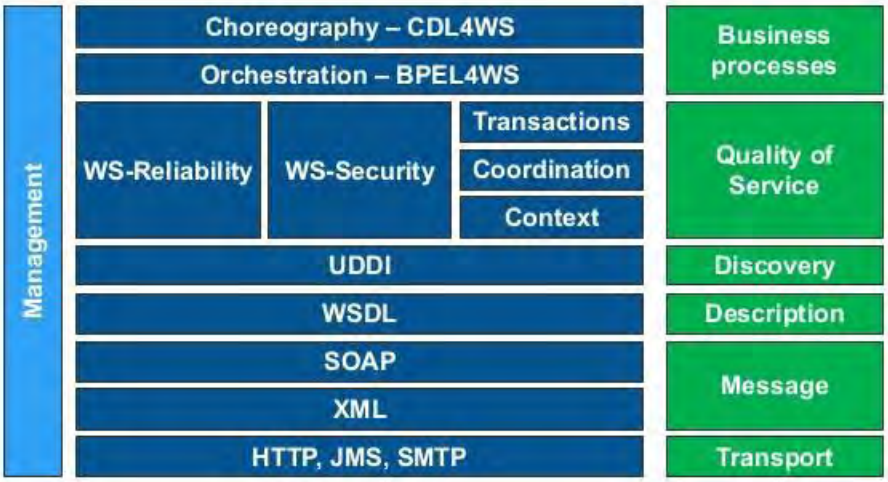

Figure 2-15 The Web Service technology stack

2.3.2 RESTful services

Representational State Transfer (REST for short) is the result of Roy Fielding's PhD work in 2000 and is the basis of an alternative approach to creating large distributed service systems. Unhappy with the complexity of mainstream service-oriented approaches, Fielding suggested to use a simple, resource-oriented way to describe services. Whereas traditional approaches were centred around the notion of computation (executing something on behalf of the client), REST looked at the problem from the data point of view. Fielding suggested that everything (even a single byte in memory) could have a URL through which its state can be queried and modified. For these, only a GET and POST HTTP commands are required along with the URL pointing to the resource. He developed a method to describing service interfaces using URLs including GET and POST commands.

The simplicity of REST-based or RESTful services made this approach popular very quickly, creating a real clash of opinions within the Web Service community. Nonetheless, REST seems to be the clear winner by now, leaving XML based Web Services used for new systems development only in exceptional circumstances. Details of REST can be found in Fielding's dissertation[7] or at the following Wikipedia entry: https://en.wikipedia.org/wiki/Representational_state_transfer.

2.4 Grid computing

Grid Computing emerged at the end of the 1990s as the result of two independent, simultaneous movements. Supercomputers in the US operated in an isolated fashion which resulted in large regional differences in supercomputer loads. One supercomputer centre could become overbooked resulting in user jobs waiting in work queues while some other centre at the same time were underutilised not receiving enough jobs to keep the system busy. Researchers proposed that by connecting these centres into a larger system which would automatically send jobs to the most suitable centre could increase the overall efficiency and utilisation of supercomputer centres and would reduce waiting time for users. About the same time, networking and distributed computing systems reached the level of maturity that allowed the construction of large compute clusters from local area connected computers. Researchers proposed to extend this concept to wide-area networks, i.e. connect distant computers together to create large virtual parallel computers. The first effort was proposing connecting supercomputers, while the second was planning the same with clusters of computers. Both were seeking to increase the efficiency and utilisation of existing compute resources by providing an efficient load-balancing software layer.

The term Grid Computing was introduced to compare access to computing to access to electricity. In the US, the electrical power distribution system is called the Power Grid. The idea was to provide computing and storage facilities for researcher regardless where they are geographically. If a computer is plugged into the Grid, it can access resources, just as we get electricity when we plug our device into the mains socket. Early efforts resulted in various software systems aiming at providing a high-level programming interface for creating and running parallel programs on distributed clusters. Notable systems include the Parallel Virtual Machine (PVM)[8], Legion [9], and the Globus Toolkit [10]. These early systems were initially focusing on creating small cluster based systems looking at ways to efficiently use idle computers in university clusters but, as experience accumulated, larger and larger wide-area systems were attempted. By 2005, nearly all developed countries were creating national grid systems. In Europe, the DataGrid project [10] led by CERN (with the participation of several

[7] https://www.ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

European countries) was a successful demonstration of this concept, although this was a system created only for one specific task, processing data generated by the Large Hadron Collider.

The most successful middleware software development project was the Globus toolkit that concentrated on using existing systems and software components and standard protocols (such as FTP, etc) instead of creating new programming abstractions (such as PVM or Legion). We will use an example from a Globus publication [11] that will help understanding the ideas behind Grid Computing.

Let us assume that a researcher wants to run a simulation program. This particular program is a distributed simulation and in interactive mode. Interactive mode is notoriously difficult in batch execution environments, even more problematic in a multi-site case. Each site has to provide sufficient resources for the program to run and at the same time with the others. It would be nice to have this executed for the user by the system itself. As Figure 2-16 illustrates, in a grid resource management system, we would expect to have a distributed simulation specific broker who takes over the configuration task from the user. It will translate the user's request into computer specific terms and will start searching for supercomputer sites ready for servicing the request. It will gather real-time information from a second level broker, the supercomputer resource broker, and with its help co-allocate resources at several sites for the execution of the simulation. Once the co-allocation is complete, the simulation will start. The user will only see the initial request and the running simulation program. All the technical details are hidden.

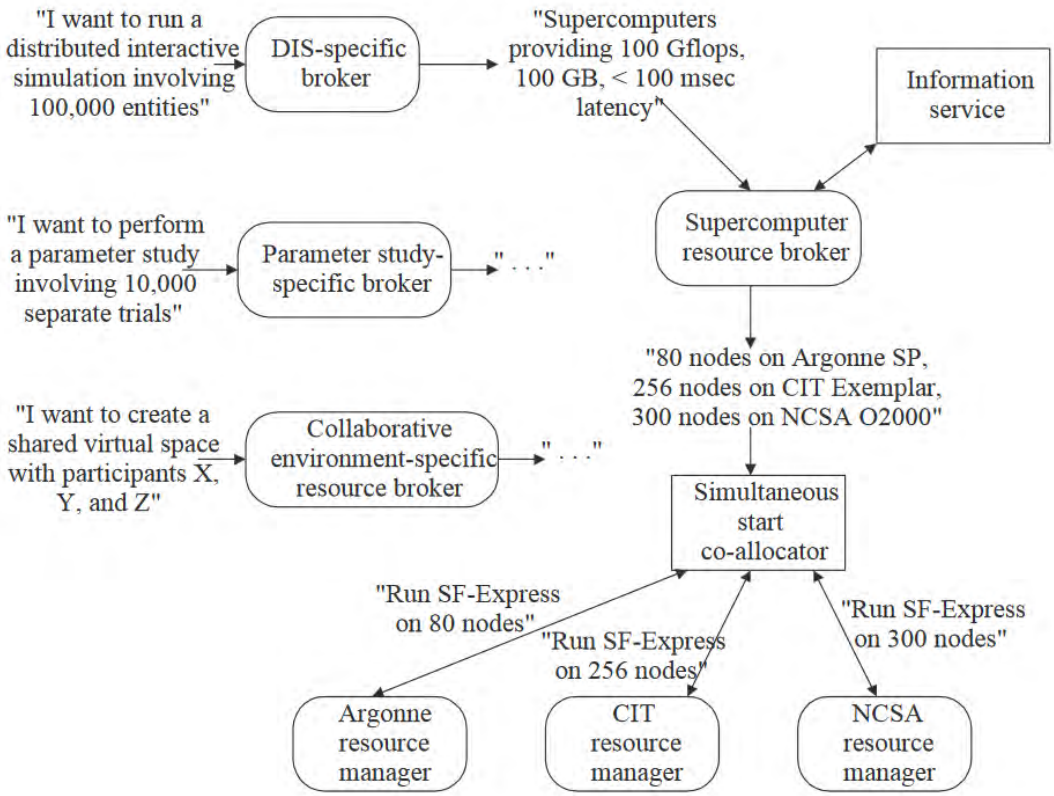

Figure 2-16 A high-level view of the execution of a user-generated resource query.

As the saying goes, "the devil is in the details". How can this be achieved? What kind of software components are required for the management of the program execution? Figure 2-17 shows the operation of this resource management system with more technical details. The system uses specifications described using a Resource Specification Language to express resource and scheduling requirements. At each site, a local Globus Resource Allocation Manager (GRAM) is running that

collects and transfers resource information to the Information Service, processes RSL requests accepting or denying them, and is responsible for monitoring and managing an executing task.

Resource brokers read grid information from the Information Service and during processing a client request will generate specialised queries and may contact other, more specific brokers with specialised queries. When the required resources requirement is satisfied, the Broker will work together with the co-allocator to reserve, allocate and execute the necessary software components.

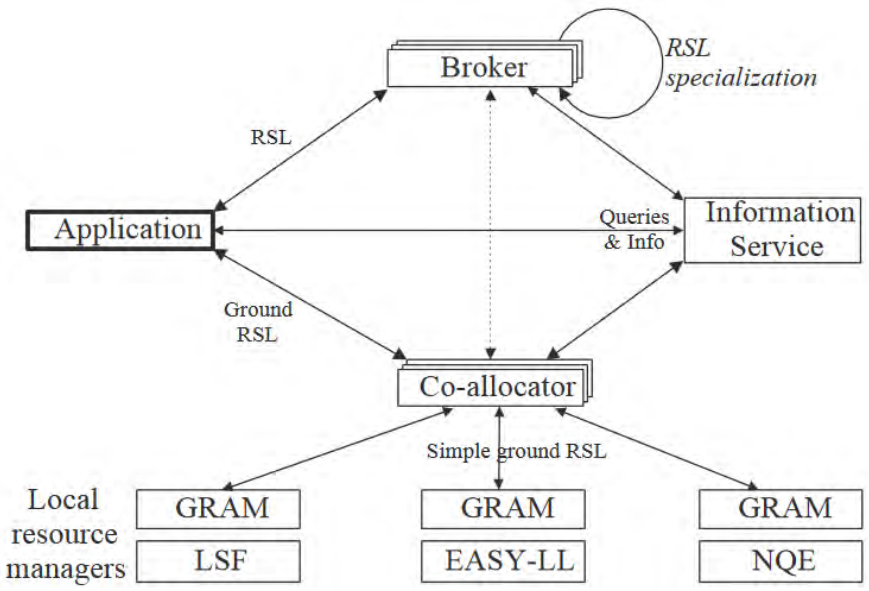

Figure 2-17 The architecture of the Globus resource management system.

As Grid Computing evolved, the notion of service-orientation appeared in the Grid as a useful abstraction. As a result, the Globus Toolkit underwent a major restructuring process and turned into a service-oriented version using Web Services as the underlying technology. The new system was called Open Grid Service Architecture (OGSA). This effort was governed by an international committee with representative members of academic and business communities.

2.5 Voluntary resource sharing

An alternative of the push-style Grid systems, in which users send their program for execution to remote sites, is voluntary computing; in which computer owners donate their computers for program execution. Programs are deployed in a pull-style by connecting a central task database.

The first and most well-known example for such a computational system is the SETI@home project. The project started in 1999 at the University of Berkeley with the idea to use idle computer resources to find signs of extra-terrestrial life by analysing radio telescope data. In this system[8], users installed a program that would run as a screensaver but instead of drawing random graphics on screen, it would connect to the SETI server, download a segment of telescope data and analyse it. The simplified architecture of the system is shown in Figure 2-18. It is interesting to note that in this system the client who wishes to run analysis programs is represented by the server, and users, who provide computational resources are clients. A further important feature of the system is that work allocation is fully automatic and the operation is fault-tolerant; the operation is continuous even in the presence

[8] https://setiathome.berkeley.edu/sah_papers/cacm.php

of cline failures or disconnections. The project was a major success; millions of users donated their computer time and the aggregate compute performance of the project exceeded top supercomputers. Technical details of the project are discussed in [12].

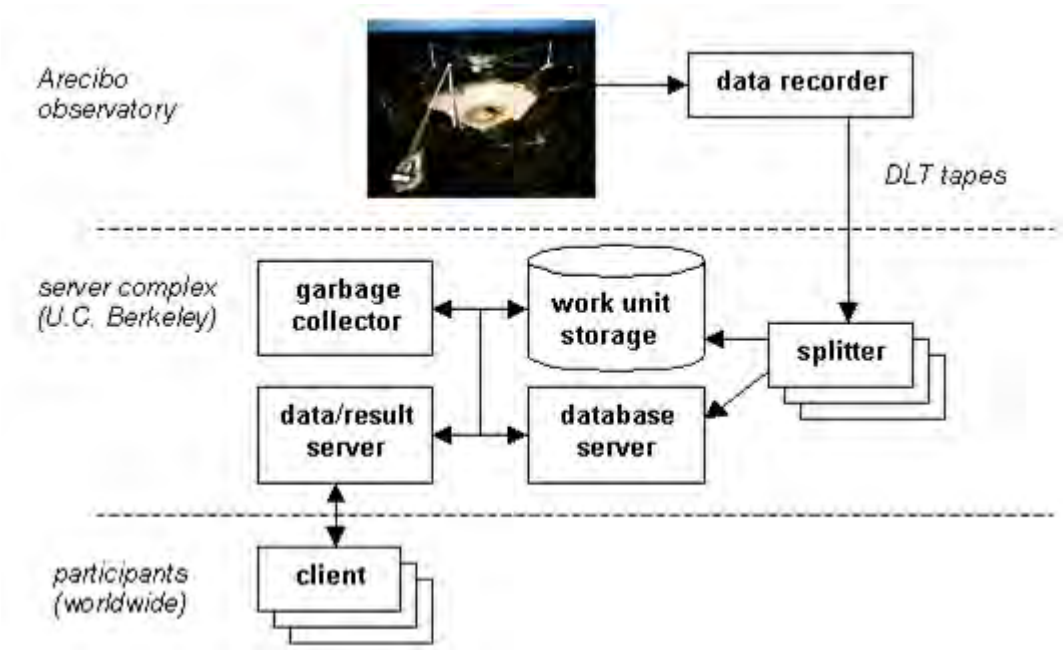

Figure 2-18 Architecture of the SETI@home system

The success of the project resulted in a new sub-project, the development and support of the runtime infrastructure. A generalised software system called Berkeley Open Infrastructure for Network Computing (BOINC)[9] was created and made available for projects looking for similar distributed computing resources. There are over 30 science projects using the BOINC infrastructure with nearly 800 000 active participating computers and over 40 petaFlop/s aggregate compute performance.

A similar project but running outside the BOINC project umbrella is Folding@home which is protein structure research platform. At the time of writing, the key focus of the project is to find molecules that can be used as potential drugs for combatting the COVID-19 virus.

2.6 Hosting and the emerging utility computing model

By the late 1990s, early 2000s computer have become indispensable for the daily operation of companies. More and more computers were deployed at businesses and it became obvious that these systems need professional support and maintenance. Initially, companies started to employ IT specialists for this task but this has become a real problem for small and medium enterprises whose primary focus is not on IT. For small companies in sales, manufacturing, etc., it was not viable and efficient to pay full time IT staff to look after the company website or in-house computers. Within a few years a new industry emerged, outsourced web hosting. It was more economical to operate a server farm with high-speed interconnect at one location and host web sites for several small businesses. Economy of scale resulted in lower prices, companies could forget about the details of website operation, and could focus on their primary role. Web hosting was soon followed by general hosting, where complete computers were managed and operated for a client, running arbitrary

[9] https://boinc.berkeley.edu/

applications. Companies liked this approach as they could pay a flat monthly fee for the services that was easy to plan for in the yearly budget. This was the beginning of what we now call “utility computing”.

SUN Microsystems was the first major IT company that saw business potential in this type of computing. Fitting nicely with their motto “The Network Is The Computer” they created the Sun Container, a portable server container (see Figure 2-19), initially designed to be deployed at large customers needing extra computational power. In 2005, SUN started to operate these systems on their own and offered access to the resources at a flat pay-per-use fee. For the Sun Grid compute utility, the price was 1$ per CPU-hour; for the Sun Grid storage utility, the price was $1 per gigabyte per month. The system was a real success but the unfortunate 2008 financial crisis left SUN bankrupt and ended their existence.

Figure 2-19 The SUN Grid container

2.7 Virtualisation

Virtualisation as a technology has been known for a long time and was used routinely in mainframe computers. The goal of virtualisation is to abstract away or disconnect a software system from its hardware environment by creating a virtual environment, most of the time in the form of a virtual machine. Virtualisation has gained attention and popularity with the emergence of hosting providers and data centres.

In the past, a software could only run on a hardware onto which it was compiled. This resulted in a brittle infrastructure and vendor “lock-in”. For instance, if a computer became obsolete, it became more and more difficult to maintain the required hardware infrastructure and keep the software operational. Also, developing a system for a particular operating system became a “life-time” choice, as it was impossible to move to a different platform once the application was developed.

The use of virtual machines enables the creation of a more flexible and efficient computing infrastructure. With virtualisation, it is now possible to virtualise a hardware resource and run any operating system on top of it (Figure 2-20). Similarly, it is possible to virtualise an operating system and run applications on it although the underlying platform is running an incompatible operating system.

As shown in Figure 2-21, there are various areas of virtualisation (hardware, network, storage, memory and so on). In *hardware virtualization*, a virtualisation layer called Virtual Machine Monitor (VMM) is used that is responsible for emulating the hardware to the operating system. The goal of hardware virtualisation is to be able to run complete operating systems on any hardware without

modification. The virtualisation layer will emulate an instruction set architecture and hardware view that is required by the guest operating system.

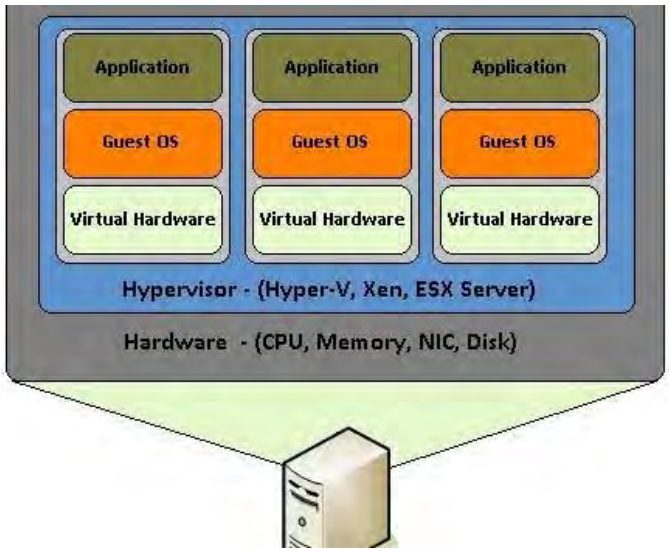

Figure 2-20 Virtualisation allows running different operating systems on the same hardware

In case of *Full Virtualization*, the complete simulation of the actual hardware takes place to allow software to run an unmodified guest OS.

Paravirtualization represents an interesting approach to hardware virtualisation. In this type of virtualization, the virtualisation layer present a special API to the operating system that allows some functionality of the operating system to be executed in the host hardware instead in the emulated layer. This increases performance and efficiency in execution. Unfortunately, this requires modification in the operating system code. Some operating systems have paravirtualised distributions.

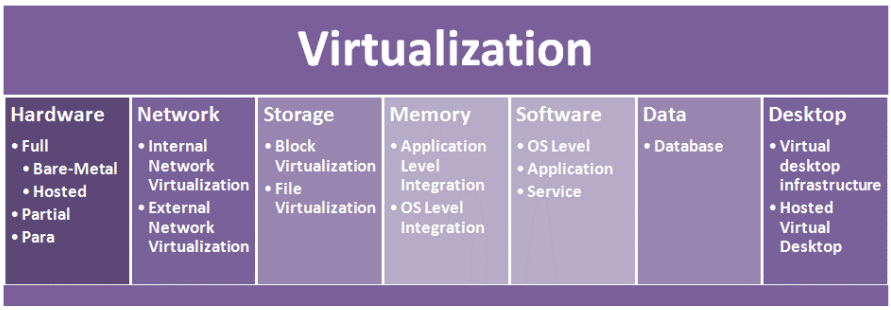

Figure 2-21 Various forms of resource virtualisation[10]

Full and paravirtualization methods are software-based approaches. The instructions area translated in binary form the guest to the host versions, which translation has serious performance penalties. The most common strategy for hardware virtualization today is *hardware-assisted virtualisation*, which relies on special hardware support from CPU manufacturers. In this mode, many functions of the OS or application software can use the physical hardware resources of the host computer (mainly

[10] Source of virtualisation figures: https://www.znetlive.com/blog/virtualization-in-cloud-computing/

its CPU) that results in much higher performance than the full software emulation based virtualisation techniques.

In summary, the key advantages of virtualisation are as follows.

- Virtualisation provides ability to manage resources effectively. Applications, systems can be started and stopped at will using snapshots, can be migrated to new hardware if necessary.
- Virtualisation increases efficiency of IT operations. It is possible to run several virtual machines on one computer and maximise the utilisation of the processors instead of the traditional one app-on server configuration that results in many idle CPU cycles.
- Virtualisation provides for easier backup and disaster recovery. Virtual machine images and state snapshots can be easily created and stored/archived.
- Virtualisation increases cost savings with reduced hardware expenditure. This is the result of better utilisation of resources.

There are also disadvantages of virtualisation that we should mention.

- There is some performance penalty of virtualisation due to the extra layers. This may result in slightly lower execution speed for applications.
- For commercial software packages, virtualised execution may be explicitly prohibited, or result in increased licensing costs.
- An organisation using virtualisation must train IT staff in virtualization.

3 Cloud technology fundamentals

Cloud technology is part of our modern everyday life. In layman's terms, we could define it as a large, virtual and scalable compute/storage infrastructure operated by cloud service providers that can be used on an on-demand basis for business entities or individuals for a variety of purposes. The key advantage of the cloud is that it can free up IT users from looking after and administering hardware and software components. The most remarkable feature of the cloud is its simplicity of use and on-demand scalability. We will look at these latter property in more detail.

For a modern enterprise relying on computers in its day-to-day operation, there are two conflicting goals in the operation of the IT infrastructure. The first goal is to minimise the size of the infrastructure to keep the upfront and operational cost at minimum. The second one is to have enough resources to perform the tasks required for profitable operation. If the business can be characterised by a constant and steady computational need (running a constant number of applications in a 24/7 fashion) the required capacity can be planned easily. For fluctuating loads, the situation is more complicated. The worst case is the unpredictable, burst-like load pattern.

Since companies must be able to operate even under extreme computational loads, technology officers normally over-specify their system; they plan for the maximum possible system load. In fluctuating or burst usage patterns, this strategy leaves the IT system underutilised most of the time (see Figure 3-1). Underutilised periods of use represent wasted resources and operational costs. It would be more economical to plan for the average system load level but this would leave the infrastructure and the company vulnerable, failing to operate in the most crucial cases, i.e., during heavy load intervals.

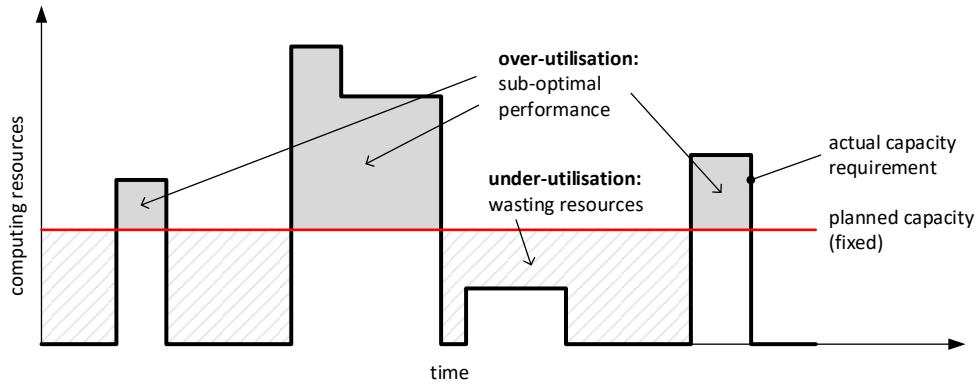

Figure 3-1 Varying computational load and its consequence on capacity planning.

Any reliable external infrastructure that is accessible for companies to bring in extra resources in peak load situations is a welcome opportunity. This allows businesses to cut down on the cost of their IT infrastructure, service only normal load situations with the internal resources and rely on external resources at peak times. Cloud technology is the first technology that can offer this at a global scale and at commercial grade quality.

This characteristics of the cloud alone would be enough to make it an attractive and successful service for the business world. What makes it special is that it is also economically attractive and represents an infrastructure that provides a platform for many, even more interesting possibilities.

3.1 Core concepts

The cloud service providers operate very large data centres consisting of thousands of servers. This server pool is the backbone of the cloud. Thanks to virtualisation technology, various operating systems can run on these systems efficiently in a way that makes them easy to monitor, manage and migrate.

If required, the low-level resources (servers, storage and networking) can be used by organisations or individuals as "systems" as if provided by a server hosting company. This type of service is called *Infrastructure-as-a-Service* (IaaS) and is displayed as the lowest layer in the service pyramid of Figure 3-2. This type of service gives maximum flexibility to the user in what and how to run on the system but with the expense of user responsibility in the management of the resources.

Not all clients require such flexibility. Many of them need only a web site with embedded functionalities (e.g. web-shops, gas, electricity, water utility companies, etc). These applications require standardised application frameworks, and their operation can be largely simplified if the cloud provider runs and manages the required application runtime environment. Examples could be an ASP.Net, Spring or NodeJS web application with some suitable backend database. In this case, the runtime platform is provided to and operated for the user (*Platform-as-a-Service*, PaaS). The user is required only to supply the application program that could be deployed on the application platform.

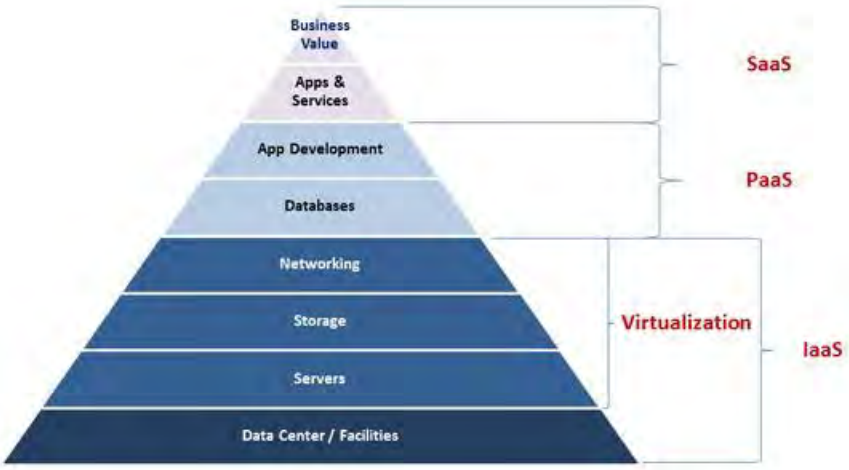

Figure 3-2 Different service layers in the Cloud

At an even higher level of service, a complete software system could be made available in the cloud. The simplest example of this is the cloud-based email service. Users are not interested in the operation details of this service, nor are they interested in the location of the servers or managing the software. They only want to send and read electronic mail. *Software-as-a-Service* (SaaS) is a very nice example of how a useful piece of functionality can be abstracted out as a service and offered to millions of users.

As we move higher on the service pyramid, the required user participation or involvement in operating, managing the system becomes less and less. This is nicely summarised in Figure 3-3.

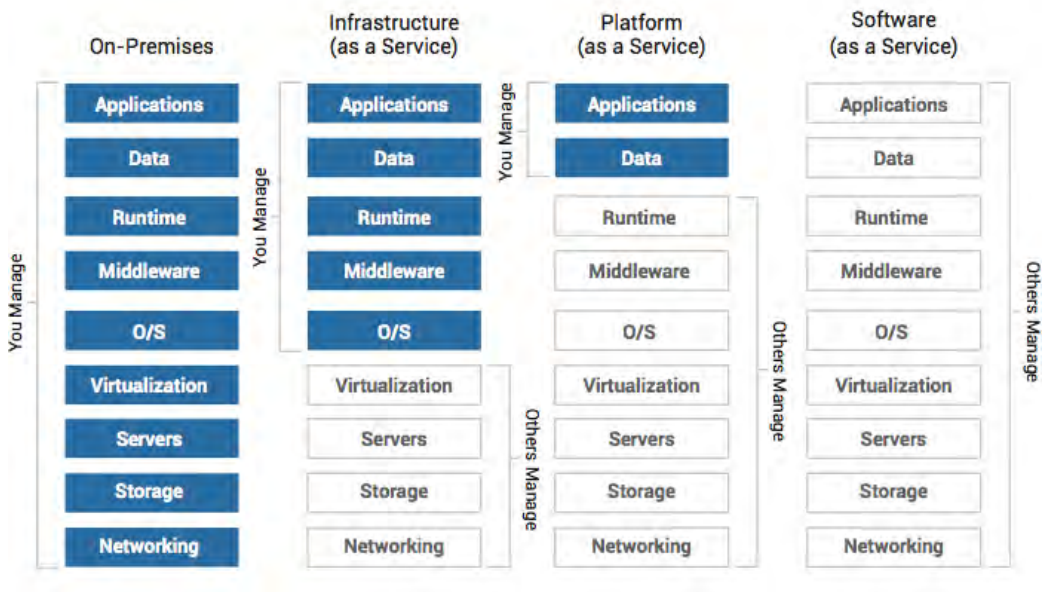

Figure 3-3 Change of management responsibilities at different cloud abstraction layers

Several other types of services appeared recently, complementing the traditional Iaas, Paas and SaaS layers; namely, Container-as-a-Service (CaaS) and Function-as-a-Service (FaaS). These will be discussed in more detail later in the programming chapters.

3.2 The economy of cloud

The key selling point of the cloud is its affordable cost, efficiency and reliability. These are all due to the economy of scale. Cloud data centres are very large, centralised server parks, with very high server and storage density. Figures Figure 3-4 and Figure 3-5 show examples of Google and Amazon data centres. Due to the scale of the data centres, vendors can purchase components with discounted rate. This may also be true for the electricity price as well. Centralisation also helps in providing top-class physical and IT security measures. Since systems in the data centre are very similar, many administrative tasks can be fully automated hence IT staff cost can be kept relatively low.

Figure 3-4 One of the Google cloud data centres

Figure 3-5 One of the Amazon cloud data centres.

The centralised design allows cloud service providers to operate their centres very efficiently. The level of operation efficiency and the available security solutions would be very difficult to achieve in a traditional company IT infrastructure.

Naturally, the responsibility of the cloud provider is not only to offer an IaaS, PaaS or SaaS service instance. There are many other factors of operation that a service provider must focus on. As shown in Figure 3-9, some of these are related to internal management, regulatory compliance (auditing) some are to offering high-level services (service brokering).

Figure 3-6 Inside an Amazon data centre

Figure 3-7 Inside a Google data centre

Figure 3-8 Google Data centre cooling system engine room

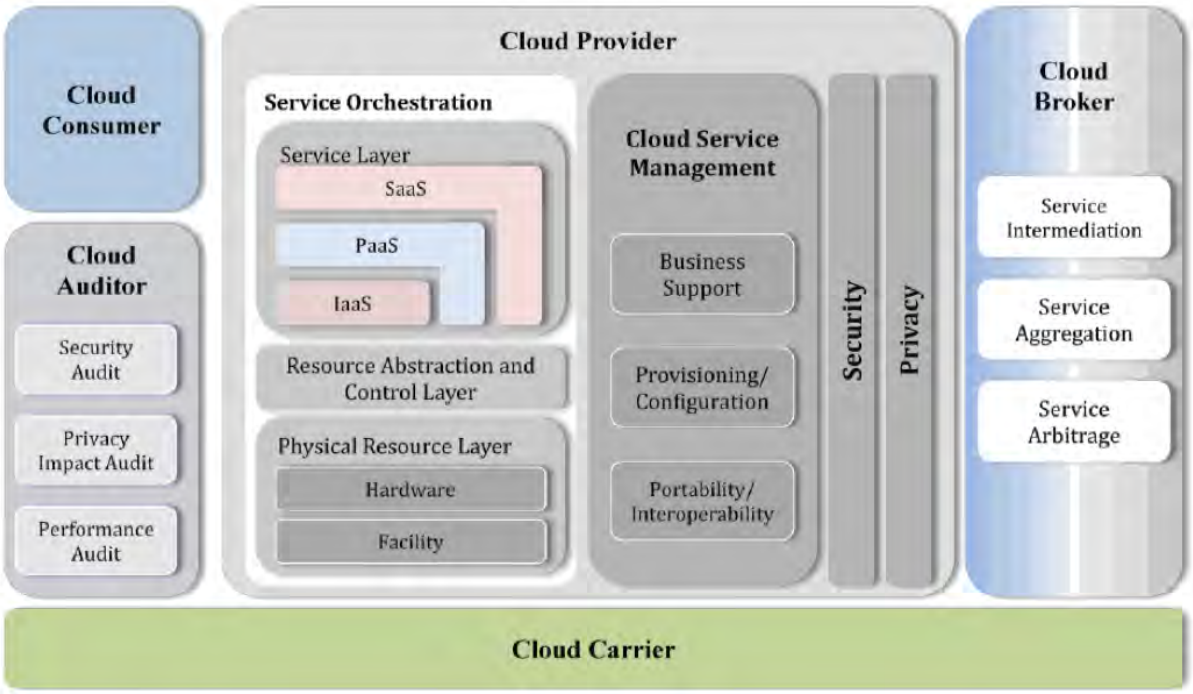

Figure 3-9 A generic cloud technology stack. Note that services (IaaS, PaaS, SaaS) represent a small part of the true scale of a cloud system

3.3 Cloud user and usage support

So far, we looked at user support in terms of management involvement. There are other ways to differentiate among cloud services, based on users' functional requirements. Software developers working in the cloud require developer support components (source code repository, test environments, etc.). Industry end users might require specific technologies relevant to their sector, e.g. Internet-of-Things (IoT) support. Data-centric users could require a wide range of data storage and analysis alternatives. Some of these are illustrated in Figure 3-10

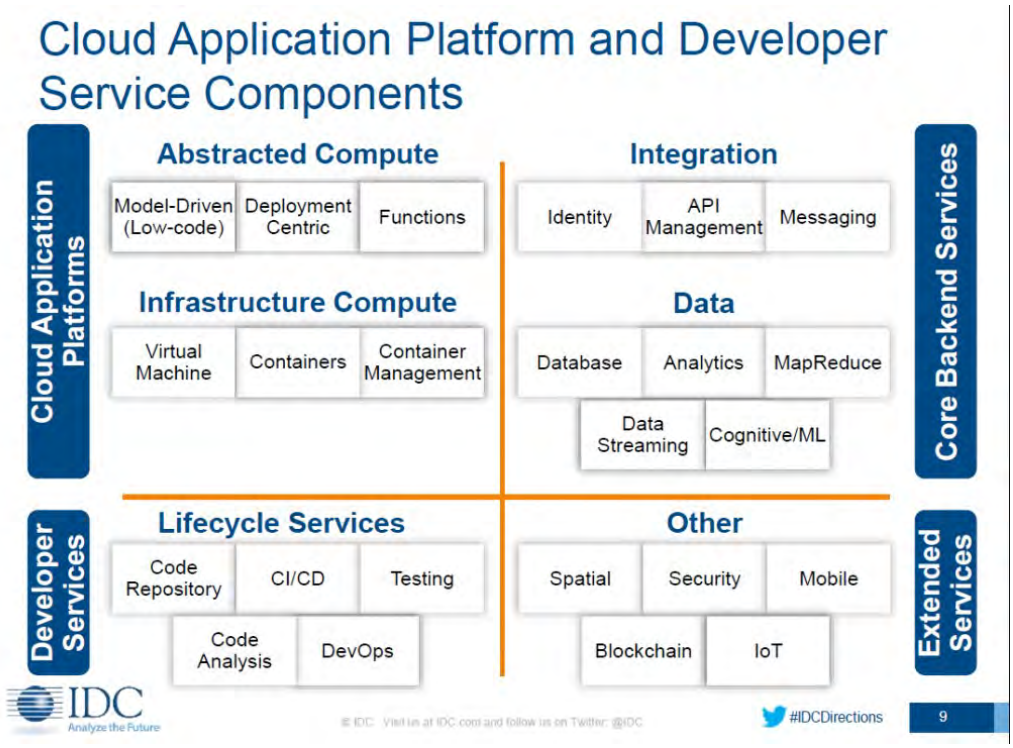

Figure 3-10 Different services of a cloud platform targeted to different user communities.

3.4 A bit of cloud history

Before diving into the detailed overview of the major cloud service providers and their services and their programming, it is probably useful to overview the developments of the past two decades and understand how modern cloud providers emerged, and understand the motivations, reasons, opportunities that led to the current status quo.

The largest cloud providers today are Amazon, Microsoft and Google followed by Alibaba, Oracle, IBM and others. As shown by the analysis diagram in Figure 3-11, in the coordinate system of Worldwide market share – Annual growth rate, Amazon is the dominating the global market. Microsoft and Google are growing fast but may or may not reach Amazon in market share. Oracle and IBM seem to be small players with moderate growth potential.

Another analysis provided by the Gartner Group divided the cloud vendors in a four-quadrant chart based on their overall vision and ability to execute their vision, labelling them as Leaders, Challengers, Visionaries, Niche Players. Figure 3-12 includes four snapshots of the results, from 2015 to 2020. It is

clear that Amazon is the global leader, but it is interesting to watch how Google is growing and closing the gap with Microsoft.

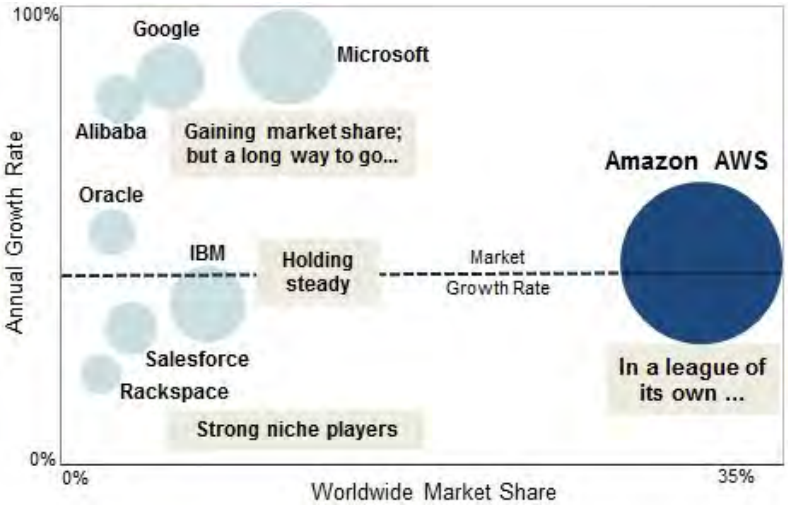

Figure 3-11 The position of global cloud players in terms of their market share and annual growth rate

3.4.1 Origins of the Amazon cloud service

The Amazon company was founded in 1994 to sell books, music and electronics online. How did Amazon ended up in the cloud world and became a leader? The story is interesting and educational. The original concept of Amazon was to sell items online, at large scale and at very low price. Achieving low prices required to leave out the middlemen (intermediate sales points) from the sale transactions (i.e. to sell directly to the clients from the warehouse) and make placing and processing orders and shipping very fast using cutting edge computerised processes. In the first few years, internal software development was performed in a rather ad-hoc basis, without much planning for long-term operation and growth. By the end of the nineties, the company found it more and more difficult to develop new systems efficiently. This has become a serious issue when they were developing online shopping technologies for Target and Marks & Spencer. It turned out that despite the experience in developing such systems, they could not reduce the overall development time, mainly because many infrastructure and backend components had to be created from scratch.

Recognising this problem, the company introduced an internal rule around the year 2000. Every internal development group had to adopt the service-oriented design methodology and create well-defined interfaces (APIs) for the systems they designed. Another rule related to the use of these services. The company ruled that every group must use service developed by the other groups inside the company, build upon those components, if it was possible. New services could only be developed it they were not yet available. Within few years, this has change the company a service company. The service-oriented view appeared in their infrastructure layer as well, using virtual machines to make their servers flexible and easy to manage. The first result of this effort was the publication of the SOAP/XML API in 2002 that enabled programmatic access to Amazon's shop catalogue for their partners. In 2003, the management identified their IT hosting expertise and service-components as a key strength and decided to create a branch in the company called Amazon Web Services (AWS) that offered resources and services for third-party clients.

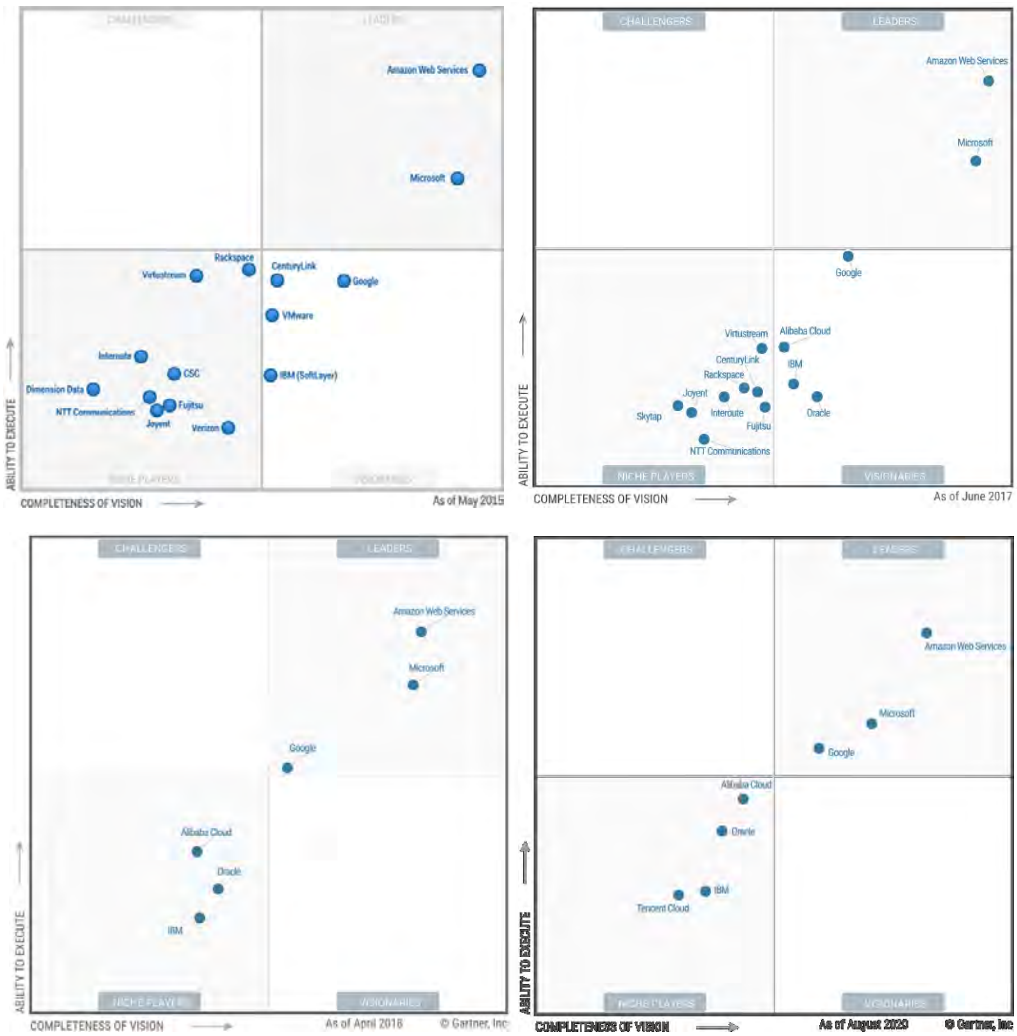

Figure 3-12 The evolution of cloud vendors within the Gartner Group's Challengers-Leaders-Niche players-Visionaries four-quadrant analysis from 2015 to 2020

After three years of development, Amazon launched a few fundamental services in 2006; the Simple Storage Service (S3), the Elastic Compute Cloud (EC2), and the Simple Queue Service (SQS). Very few thought that this had been the first step of creating the largest cloud service provider of the world. The success and subsequent growth was unexpected. In 2009, EC2 services were launched in Europe, too. The development has been continuous ever since, competitors started to response with several year's delay. Since then, Amazon became not only a cloud provider but one of the largest and most valuable company in the world selling practically everything one can imagine. Behind the scenes, the company developed over the years considerable expertise in machine learning and artificial intelligence too, and have become a key player in the AI field as well.

3.4.2 Microsoft and the cloud

The development of Microsoft's cloud platform started in 2008. The system, originally called Windows Azure, was launched in 2010. Later it was renamed to Microsoft Azure. The original focus of Microsoft was on its business partners, offering them a much simpler IT infrastructure operation alternative and the possibility to scale existing applications to millions of users without changing the code. Later on,

this focus has widened resulting a global system offering a wide range of services to end users and companies alike. The measure of success of Microsoft's effort is its second place among the cloud providers.

3.4.3 Google Cloud

Google, as a web search company relied upon large serve farms servicing millions of search queries per minute therefore had built up significant expertise in efficient server operation and management. It is not surprising that they became interested in providing cloud services to their users. The company launched its first service, the Google App Engine that provides a platform for web applications in 2008. This was followed by other services in succession; the Google Cloud Storage and BigQuery in 2010, the Google Cloud SQL in 2011, and the Google Compute engine by 2013. The company had a very aggressive strategy for cloud development and growth and today is the fastest growing cloud provider. Inside Google, the cloud division has become the fastest growing and most profitable branch of the company.

3.4.4 IBM and Oracle

Two relative latecomers in the cloud market are IBM and Oracle. IBM started to develop its own cloud system in 2013 first focusing on infrastructure-as-a-service offerings, In 2016, it introduced platform services and by 2017 it covered all layers of the service pyramid. Oracle launched its cloud service in 2016, and steadily expanded its service set to cover a wide range of use cases and support various application areas.

3.5 Pricing

Remember the utility computing model and Sun's container that use a flat 1$/CPU-hour price? This pricing scheme is called the *fixed-priced on-demand* or *pay-as-you-go* pricing. Amazon used this fixed-price scheme for its services in 2006 charged by the hours. Charge by the hour means that even if 5 minutes is used for a resource, the full hour fee has to be paid by the user. Several years later this scheme improved, first charge unit reduced to 5 minutes, one minute, then to one second. Minute and second based charges were especially useful for burst loads with short but heavy computational demand. The fixed price scheme, in general, is favoured by businesses, as it makes planning, budgeting and accounting resource use at the company very straightforward.

Further developments in the pricing strategies include the appearance of discount and dynamic pricing. *Reserved pricing* requires the user to pay an up-front reservation fee for a certain amount of resources which then will be available for at a lower, discounted rate. This is quite similar in concept to the resource brokering mechanism in Grid computing systems where user had to reserve resources for simultaneous execution on different computers. Another pricing alternative is *spot pricing*, which uses a dynamic, auction-based pricing mechanism. Spot pricing is typically used for idle resources and the low starting price is used to attract users to the otherwise idle resources. There are several subtle variants of these three basic pricing schemes. We will look at some of them in the detailed overview of a selection of cloud services.

3.6 Programming

Since the cloud environment is mainly for performing computational tasks, whether these are web applications, data processing and machine learning workflows, the question of programming

languages naturally arises. Which programming language or languages can be used for creating cloud applications?

There are two conflicting requirements here. Users would like to run their programs in the widest possible range of resources. Ideally, this means complete platform independence. At the same time, users want to keep and use their existing source code, components and applications and expect them to be readily usable in the cloud without code modification.

Naturally, service providers would also like to support one language or a very small set of languages since each supported language results in the replication of the entire cloud programming API in that given language, increasing code base size, testing, maintenance and future development efforts.

The result of this clash to date is that most cloud providers try to support as many languages as possible. The most popular contenders are JavaScript, Java, Python, C# but many others can be found at certain vendors.

4 Amazon AWS

Amazon Web Services is the largest and leading cloud system in the world. It offers the widest selection of services in the form of component services or complete application to users with the widest availability. The categories of services AWS offers is listed on their website [13] (screenshot is shown in Figure 4-1). Each category represents several services.

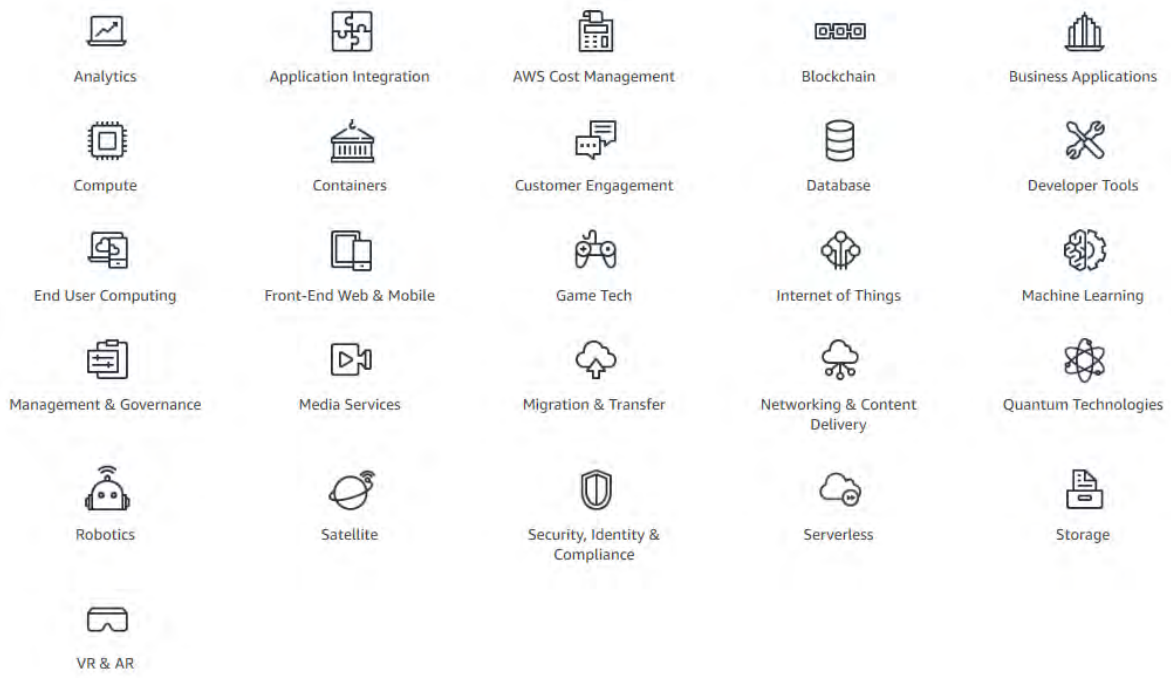

Figure 4-1 Service categories at Amazon Web Services

AWS has many solutions for different industry too that make the adoption of cloud simpler for these users. A short list of industrial sectors and illustrative cloud solutions offered is as follows.

Advertising and Marketing	solutions for marketing and advertising (data analytics, personalisation, messaging and marketing campaign)
Automotive	solutions for connected vehicles, autonomous driving development, manufacturing and supply chain management
Consumer Packaged Goods	solutions for smart factory, supply chain, intelligent forecasting, connected home & smart products, personalisation
Education	solutions for primary and high schools, universities for online learning, distance education, online labs and classes, streaming, contact management, etc.
Energy	solutions for seismic acquisition, storage and analysis, reservoir simulation & field development planning, subsurface data management
Financial Services	financial services for institutions across banking, payments, capital markets and insurance, including scalable processing infrastructure data analytics and machine learning
Gaming	database, compute, analytics and machine learning resources for game developers and online game providers

Government	commercial cloud services for public administration (from government to municipalities) available at all security classification levels: Unclassified, Sensitive, Secret, and Top Secret
Healthcare and Life Sciences	solutions for healthcare providers and insurance companies, pharmaceutical and biotech industry, genomics (data storage, scalable computing, analytics and machine learning)
Manufacturing	solutions for product & production design (parallel simulations and data processing), smart factory, smart products & services (IoT & BigData technologies)
Media and Entertainment	solutions for digital content creation and delivery (video applications, studio in the cloud, secure content streaming)
Nonprofit	solutions for websites, marketing campaigns, IoT based monitoring
Retail	solutions for websites and online shopping, customer relation management, data analytics and machine learning
Telecom	solutions for billing, content delivery, customer service
Travel and Hospitality	solutions for personalisation, reducing IT costs, improving customer service

Amazon provides services based on the world's largest cloud infrastructure. AWS is available in 245 countries and have data centres in 24 geographic regions (see Figure 4-2). Each region has several Availability Zones that are connected with low-latency, high throughput and redundant networks to provide high availability and fail-over capabilities. Zones are considered as the single failure units.

In the following subsections, we will overview some of the key AWS services.

Figure 4-2 Existing and planned AWS global regions.

4.1 IaaS Layer Services

The fundamental services in the AWS infrastructure layer abstract out the key elements of any computing systems; compute, storage and network capabilities.

4.1.1 Compute services

The Amazon **Elastic Compute Cloud** (Amazon EC2) is one of the first services Amazon rolled out and it provides virtual machines accessible as a web service. Virtual machines can be run over a variety of hardware resources belonging to one of the following instance type groups: *general purpose*, *compute*, *memory* or *storage optimised*, and *accelerator* (GPU, FPGA) based systems. The general-purpose instance group is designed for business critical applications, small and mid-sized databases, web tier applications. The compute optimised instance group is ideal for high performance computing, batch processing, video encoding, etc. Memory optimised instances are designed to support high performance databases, distributed web scale in-memory caches, real time big data analytics, and the like. Storage optimised instances are particularly suited for NoSQL databases, data warehousing, distributed file systems. Finally, accelerator instances are recommended for compute intensive machine learning, graphic intensive applications, high-performance computing (HPC) algorithms and gaming. Each instance group contains several instance type classes that characterise the hardware platform. For instance, the AWS general-purpose instance group contains the following classes: A1, M4, M5, M5a, M5n, M6g, T2, T3, T3a and T4g. The A1 class uses the Amazon ARM-based 64-bit Graviton processor; the M4 and M5 classes contain Intel Xeon CPUs; the M5a uses AMD EPYC processors. The M5n represent higher network bandwidth for the M5 class. The T classes (T2-4) are so-called burstable instance types that offer and interesting usage model. The instances offer a baseline performance but are capable to provide increased performance under burst load situations. Underutilised instances collect credits (i.e. save money) that can be used to cover the extra resource requirements of the burst mode. The number of cores, the type and size of memory and network bandwidths can vary in wide range, e.g. core count from 1-96 cores (virtual CPUs, vCPUs), memory size from 8 GB to 24 TB, network bandwidth from 1 to 100 Gbps. Table 2 lists the available configurations of the General Purpose M4 instance type.

Table 2 General Purpose M4 instance type configurations

Instance	vCPU*	Mem (GiB)	Storage	Dedicated EBS Bandwidth (Mbps)	Network Performance
m4.large	2	8	EBS-only	450	Moderate
m4.xlarge	4	16	EBS-only	750	High
m4.2xlarge	8	32	EBS-only	1,000	High
m4.4xlarge	16	64	EBS-only	2,000	High
m4.10xlarge	40	160	EBS-only	4,000	10 Gigabit
m4.16xlarge	64	256	EBS-only	10,000	25 Gigabit

There are several further variants of the EC2 compute service. The *Amazon EC2 Autoscaling* service can automatically add or remove compute capacity to meet changes in demand. The *Amazon EC2 Spot* service uses the spot-pricing scheme and result in up to 90% cost savings for fault-tolerant workloads. The *AWS Batch* service provides support for use cases where multiple compute jobs have to be executed. The batch service simplifies the process, as there is no need to create a fixed-sized cluster and install a job scheduler and manager software. The service will schedule and execute the jobs automatically and use as many resources by dynamic scaling as is required for the task. This service is

ideal for scientific studies such as molecular dynamics simulations, genetics, and physics parameter sweep studies or in the media sector, e.g. making animated movies or performing post-production special effect processing.

4.1.2 Storage Services

Amazon provides three major types of cloud storage options, namely the Object, File, and Block Storage services. The difference among the three are in the intended use and data access interface.

Amazon Simple Storage Service S3

Data in the S3 storage service is stored as *object*s within resources called "buckets". A single object can be up to 5 terabytes in size. The service can scale storage resources up and down to meet fluctuating demands automatically. Data durability on S3 is designed for 99.999999999%. To achieve this, S3 automatically creates and stores copies of all S3 objects across multiple systems.

S3 can also append metadata tags to objects, manage data access controls, secure data against unauthorized users, run big data analytics, and monitor data at the object, bucket levels. Objects can be accessed through S3 *Access Point*s or directly through the bucket hostname. The storage service uses different storage classes that differ in data access time, redundancy and storage size. These S3 Storage Classes are as follows: *S3 Standard*, *S3 Intelligent-Tiering*, *S3 Standard-Infrequent Access*, *S3 One Zone-Infrequent Access*, *Amazon S3 Glacier*, *Amazon S3 Glacier Deep Archive*, and *S3 Outposts*. An interesting feature of S3 that it automatically moves data objects across storage classes based on the access pattern of the data, as illustrated in Figure 4-3. This is performed without user intervention and can result in significant cost savings.

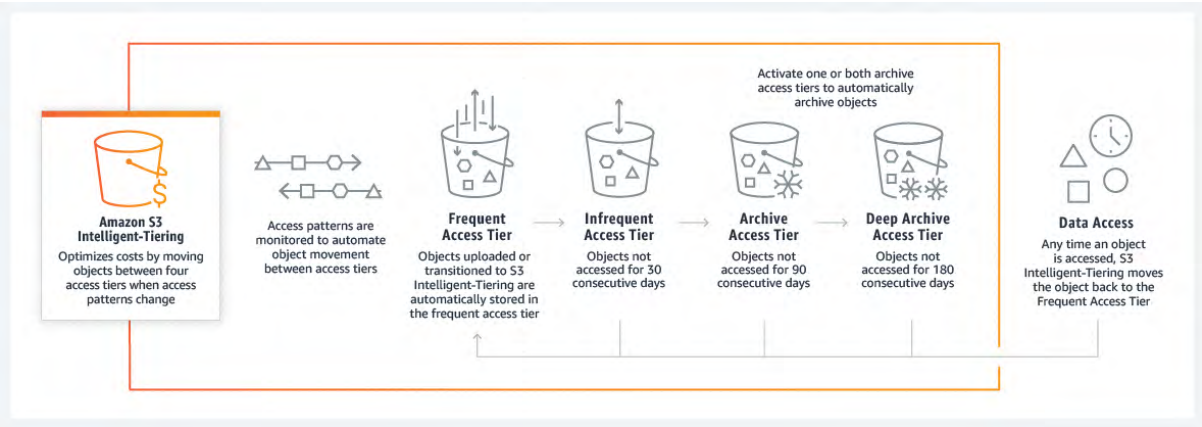

Figure 4-3 Intelligent data tiering in the AWS S3 service[11]

Amazon Elastic File System

The *Amazon Elastic File System* (Amazon EFS) is a service for a more traditional style data storage. It provides a simple, scalable, fully managed elastic NFS file system that can be used with AWS Cloud

[11] Image source: https://aws.amazon.com/s3/?c=s&sec=srv

services and on-premises resources. A key feature of EFS is that it can scale -- on demand -- to petabytes without disrupting applications. There are two storage classes available in EFS: the Standard storage class, and the Infrequent Access storage class (EFS IA). The Infrequent Access provides price/performance cost-optimized for files not accessed every day. EFS can use the Lifecycle Management to move automatically and transparently those files not accessed frequently into EFS IA files.

Amazon Elastic Block Store

The *Amazon Elastic Block Store* (EBS) is created for both throughput and transaction intensive workloads, such as relational and non-relational databases, enterprise applications, containerized applications, big data analytics engines. The service is an easy to use, high performance block storage service designed for use with Amazon Elastic Compute Cloud (EC2). There are five different volume types to choose from to balance optimal price and performance. At one end, one can choose a single digit-millisecond latency type for high performance database workloads, or at the other end, gigabyte per second throughput for large, sequential workloads such as Hadoop. The volume types can be tuned in performance, or increased in volume size without disrupting applications. EBS volumes are replicated within an Availability Zone (AZ) to support mission-critical systems.

Data Security

One of the biggest fear when storing data in the cloud is security. How safe is our data? There are users who could greatly benefit from using the cloud but refuse to do so because they feel their data is not safe. Naturally, there are situations, sometimes due to legal regulations that prohibit data storage outside a business entity or institute. You can think of financial and health care institutions as good examples. In all other cases, the fear of data loss is largely unjustified.

By default, data is always encrypted in the Cloud. It is encrypted when in transit and it is stored encrypted on storage devices. Amazon S3 supports both server-side encryption (with three key management options) and client-side encryption for data uploads. Even if data is accidentally leaked, it will be encrypted that makes it practically unusable for attackers. In most know cloud data leak incidents, the cause of the data leak was the result of human error. Cloud vendors, such as Amazon, Microsoft, Google employ the best security experts in the world and operate at a much more strict security level than it is found in on-premises IT infrastructures. Besides encryption, Amazon S3 also offers flexible security features to block unauthorized users from accessing your data. Any data is only accessible by default to the user who created it. Sharing is only possible if set explicitly and controlled through AWS access points under the supervision of the Identity and Access Management (IAM) system. In cases when further isolation is required, *virtual private cloud*s (VPC) can be created. Amazon Virtual Private Cloud (Amazon VPC).

4.1.3 Network Services

The third component of the IT infrastructure after compute and storage services is the network. One may ask, what is so special about the network that is should appear as a service. Isn't it just an automatic connection between the system components?

It is surprising how different network requirements can appear in cloud applications. Users vary from individuals who perhaps only use the cloud as a backup device for family photos to large multi-national companies running their entire operation on the cloud. To support these range of requirements, AWS provides several network-related services that can be used in combination to provide the required connection layer. These services are the *Amazon Virtual Private Cloud*, *AWS Transit Gateway*, *AWS PrivateLink*, *Amazon Route 53*, *Elastic Load Balancing*, *AWS Global Accelerator*, *AWS Shield*, *AWS WAF*,

AWS Firewall Manager, and the *AWS Virtual Private Network*. These services can be grouped by their role as services used in building, scaling, and securing networks. We will highlight the key features of each below.

Amazon VPC is used to build a cloud network, define and provision a logically isolated network for AWS resources. The AWS Transit Gateway can be used to create a central connection for Virtual Private Clouds and on-premises networks. If required, a private – dedicated, non-shared – connection can be create to the cloud using AWS PrivateLink. Amazon Route 53 is a managed DNS service that can be used to route users to Internet applications, while AWS Virtual Private Network (VPN) allows clients to connect to cloud resources using VPN as if they were on-premises resources using encrypted communication channels.

Amazon also offers services for scaling networks, such as Elastic Load Balancing that can be used to automatically distribute traffic across a pool of resources, such as instances, containers, IP addresses, and Lambda functions. A number of solutions are provided for securing network traffic. These include AWS Shield to safeguard applications running on AWS against DDoS attacks, AWS WAF protecting web applications from common web exploits, and AWS Firewall Manager to centrally configure and manage firewall rules. The application of these concepts and services is illustrated in Figure 4-4.

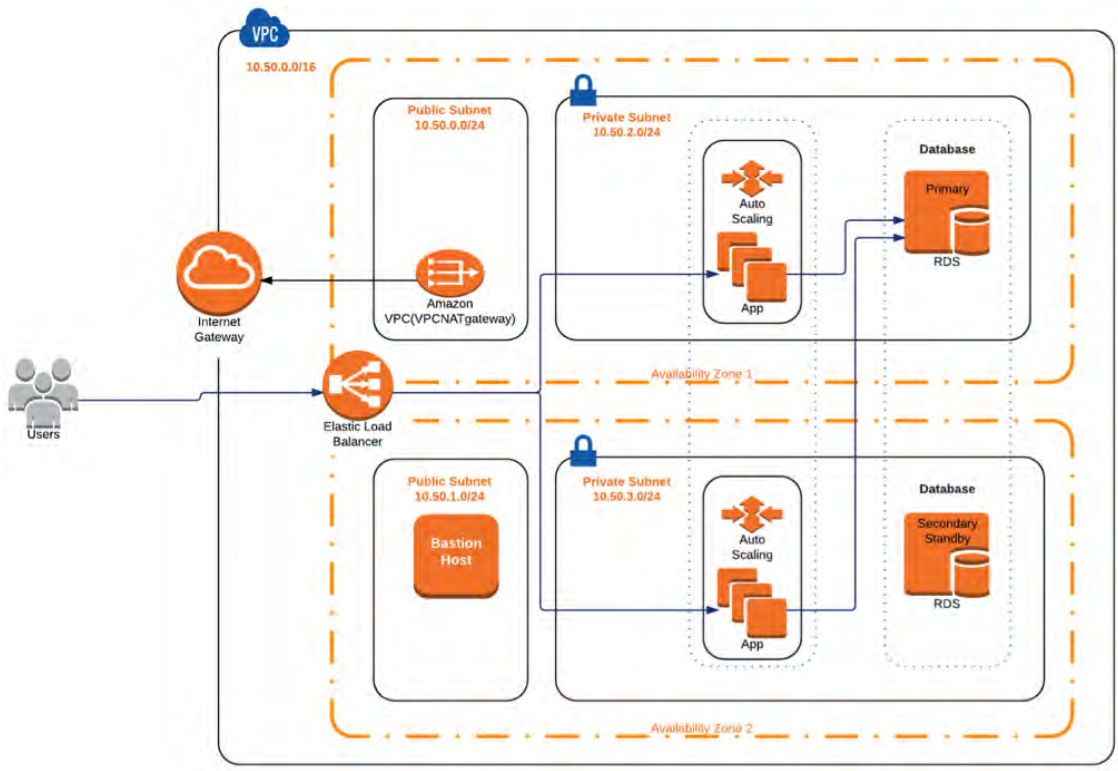

Figure 4-4 A combination of private and public subnets, load balancing and auto scaling in a cloud application network configuration[12]

[12] https://aws.amazon.com/blogs/startups/launch-your-app-with-the-aws-startup-kit/

4.2 PaaS Layer Services

The different infrastructure layer services discussed above offer great flexibility to cloud application developers. The downside is that as the application and its user base grow, more services are required to handle complexity which, in turn, complicates applications and their management.

Platform-as-a-Service was designed to alleviate this problem. Low-level infrastructure details are shielded from the user/developer, only the application layer is visible. Amazon offers the Elastic Beanstalk as a universal cloud application runtime platform.

4.2.1 AWS Elastic Beanstalk

AWS Elastic Beanstalk is an easy-to-use service for deploying and scaling web applications and services. Elastic Beanstalk supports applications developed in Go, Java, .NET, Node.js, PHP, Python, and Ruby. When an application is deployed to Elastic Beanstalk (e.g. a Java *.war file), it builds the selected supported platform version and provisions one or more AWS resources, such as Amazon EC2 instances, to run the application. Elastic Beanstalk automatically handles the details of capacity provisioning, load balancing, scaling, and application health monitoring.

Once the application is launched, the application can be managed and monitored by the user. The workflow of application deployment, lunch and management is show in Figure 4-5. In the case of a Java application, the runtime environment is an Apache Tomcat web container supporting Java Servlets and Java Server Faces technology.

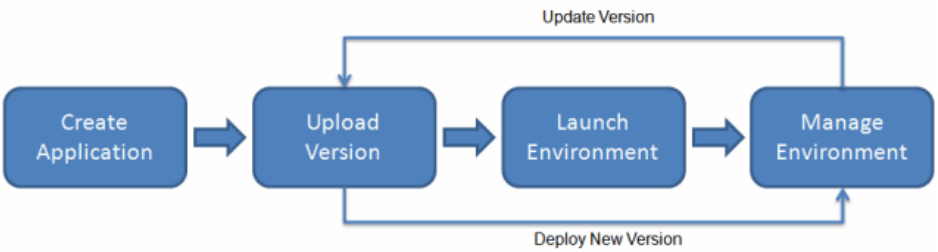

Figure 4-5 Application management workflow

4.3 SaaS Layer Services

In certain cases, a piece of functionality can be present to users as a stand-alone service. In these cases users are not required to know either the implementation or management details of the service. They are only using then using knowledge about the user-level functionality, e.g. how to send or receive an email, open and edit a photo, and similar. These type of services represent the highest level of abstraction in the cloud and are called Software-as-a-Service (SaaS).

The highest level of AWS SaaS offerings include a variety of services spanning various application areas, such as infrastructure, business, industry, machine learning and IoT. Besides the hundreds of third-party SaaS applications powered by AWS, Amazon has their own SaaS offerings, e.g. WorkDocs WorkMail, and the Chime online meeting for office users or the QuickSight data analytics service. In addition, there are several SaaS service components that can be used to add sophisticated features to existing applications (e.g. text analytics “Comprehend”, translation “Translate”, image and video analysis “Rekognition”, speech generation “Polly” and understanding “Transcribe”, and so on).

4.4 Service control and APIs

AWS services and applications can be programmed in various languages. Each language is supported in the form of an SDK (Software Development Kit). AWS supported SDKs are: JavaScript, Python, PHP, .NET, Ruby, Java, Go, Node.js and C++.

Most services can also be controlled from the Command Line Interface (CLI). The generic interface is AWS CLI, which provides access to all services from the command line. In addition, specialised command line interfaces are also available for ECS, Elastic Beanstalk, Amplify, Amazon Machine Image, and serverless applications. Using the CLI commands, users can control multiple AWS services. It is also possible to write scripts to automate common tasks. The AWS CLI offers features including improved installers, new configuration options such as AWS Single Sign-On (SSO), execution of OS shell commands, and various interactive features, such as fuzzy auto-completion and dynamic in-line documentation. A small set of examples are shown below:

```
$ aws ec2 describe-instances

$ aws ec2 start-instances --instance-ids i-1348636c

$ aws sns publish --topic-arn arn:aws:sns:us-east-1:546419318123:OperationsError -
-message "Script Failure"

$ aws sqs receive-message --queue-url https://queue.amazonaws.com/546419318123/Test

$ aws help
```

Java SDK

The AWS Java SDK provides a comprehensive set of classes to control and program various AWS services. The API documentation can be found at: https://sdk.amazonaws.com/java/api/latest/. Note that the AWS Java SDK contains over 73000 Java classes and interfaces. This is roughly 10 times the number of classes of the Java SE platform! This illustrates the extent and richness of the AWS service infrastructure.

There are various code examples provided for in the Java SDK documentation that illustrate the use of these classes and show through various use cases the integration and control of services. The examples can be found at https://github.com/awsdocs/aws-doc-sdk-examples/tree/master/javav2. In this chapter, we will look at a few representative examples only. The first sample creates an Elastic Compute instance.

```
        String name = args[0];       // name of the instance
        String amiId = args[1];      // OS image ID

        Region region = Region.US_WEST_2;
        Ec2Client ec2 = Ec2Client.builder()
                .region(region)
                .build();
        // create an instance
        String instanceId = createEC2Instance(ec2,name, amiId) ;
        System.out.println("The Amazon EC2 Instance ID is "+instanceId);
        ec2.close();
        RunInstancesRequest runRequest = RunInstancesRequest.builder()
                .imageId(amiId)
                .instanceType(InstanceType.T1_MICRO)
                .maxCount(1)
                .minCount(1)
```

```
                .build();
        // once created, start the instance
        RunInstancesResponse response = ec2.runInstances(runRequest);
        String instanceId = response.instances().get(0).instanceId();

        Tag tag = Tag.builder()
                .key("Name")
                .value(name)
                .build();

        CreateTagsRequest tagRequest = CreateTagsRequest.builder()
                .resources(instanceId)
                .tags(tag)
                .build();

        try {
            ec2.createTags(tagRequest);
            System.out.printf(
                    "Successfully started EC2 Instance %s based on AMI %s",
                    instanceId, amiId);
        } catch (Ec2Exception e) {
            System.err.println(e.awsErrorDetails().errorMessage());
            System.exit(1);
        }
```

The second sample will access a running EC2 instance identified by its instanceID and enable monitor the service.

```
         String instanceId = args[0];

        Region region = Region.US_WEST_2;
        Ec2Client ec2 = Ec2Client.builder()
                .region(region)
                .build();

        MonitorInstancesRequest request = MonitorInstancesRequest.builder()
                .instanceIds(instanceId)
                .build();

        ec2.monitorInstances(request);

        System.out.printf("Successfully enabled monitoring for instance %s",
                              instanceId);
```

5 Google Cloud Platform

Google is probably the fastest growing cloud provider in the world. The Google Cloud Platform (GCP) provides a wide range of services from the lowest, infrastructure level to the most abstract software and serverless service types. The Google Cloud Platform is available in over 200 countries. Google operates a number of data centres around the globe (see Figure 5-1).

Figure 5-1 Location of GCP data centres

Resources in GCP are arranged to *regions* consisting of multiple *zones*. At the time of writing, GCP operates in 24 regions using altogether 73 zones. Physical and virtual GCP resources can be defined as of global, regional or zonal type. Resources with global scope (available everywhere) include preconfigured disk images, disk snapshots, and networks. Regional resources (e.g. Central US, Western Europe, East Asia) typically include static external IP addresses. Most resources are of zonal scope, e.g. virtual machine instances, disks. Zones represent a domain of single failure. Using the zonal system has several benefits. It provides locality for resources of a particular application (required resources can be accessed faster if in the same zone, <1 ms round-trip network latency is typical); isolate resources from others (increased security); using several zones by replicating applications can provide redundancy that improves service reliability, availability, performance (users can choose services that are closest to them) and disaster recovery. A graphical illustration of the zonal system and the corresponding resource is shown in Figure 5-2.

GCP contains over 100 products; a small set of them is shown in Figure 5-3. The products include Iaas, PaaS, SaaS services, Container and Function as a Service solutions as well as developer, security and management products. In the remaining part of this chapter, we will look at the most important products in the different service categories. After this overview, we will investigate a selected set of services in more detail and focus on how they can be programmed and integrated into cloud applications.

Figure 5-2 Illustration of the Global, Regional and Zonal resources concept

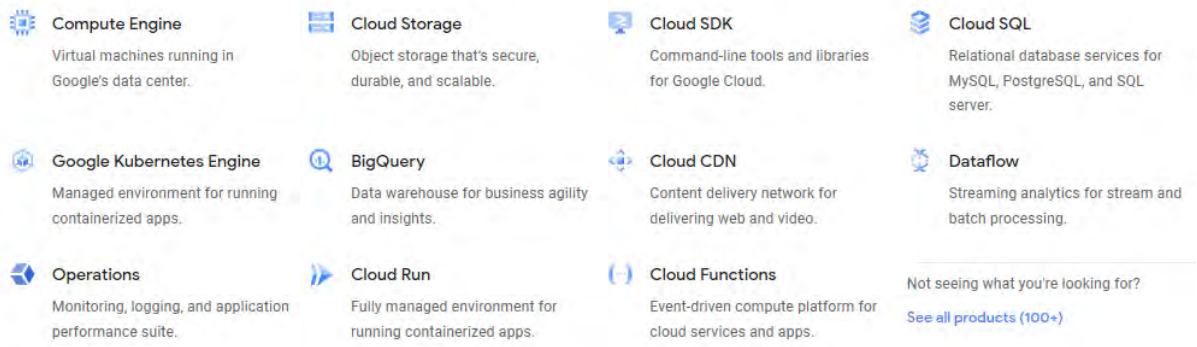

Figure 5-3 A representative subset of the available Google Cloud Platform products

5.1 GCP Services in the IaaS Layer

Infrastructure services include compute, storage and network components. The fundamental service in the IaaS layer is the **Compute Engine** that represents a Virtual Machine instance running on a particular type of hardware/OS platform. As found at other cloud providers, the underlying GCP hardware are grouped by their intended use. In the case of GCP, these groups represent General Purpose (E2, N1, N2, N2D), Compute (C2), Memory (M2) and Accelerator (A2) optimised machines.

General purpose machines are meant to be used for web server and database applications. They provide a good balance of price and performance. For high-performance and gaming applications, engineering design, Compute optimized machines should be used. Memory optimized machines offer large sets of memory and are ideal for data-analytic applications and for in-memory databases. If we want to run artificial intelligence applications required GPUs or have massively parallel implementations for HPC tasks, we can use the Accelerator optimized machines that offer different types of NVIDA GPUs.

The number of cores in these virtual CPUs can range from 1 to 96 cores. Memory can be increased up to 1.4 TB per VM. The configuration of the VM obviously affects the price. GCP Compute Engine instances are charged by the seconds at a flat rate depending on the number of cores and RAM size configured. There are several ways of reducing the price. GCP operates an automatic *sustained use discount*. If GCP detects that our application is running a considerable time each month, it will automatically apply a discount. If our application has a steady and predictable usage pattern, it is possible to use a *committed use discount*. This can result an up to 50% discount. Finally, if our program can run on an interruptible VM, the *pre-emptible VM*s are a good choice offering up to 80% discount.

There are further interesting features of the GCP Compute Engine. If required, VMs can be run on *sole-tenant node*s, which means that a physical server node will only run our Compute Engine instances. The node will not be shared with VM instances of other users. This is equivalent to having a dedicated machine in our office running our applications but located in the cloud. Another form of security guarantee is the option to use *Confidential VM*s. These VMs are using Confidential Computing, a new technology that allows user data to be encrypted during computation. Traditional execution in the cloud is based on data encrypted in storage (encryption at rest) and during data transfer (encryption in transit) but decrypted when loaded to memory and manipulated. With Confidential Computing, it is now possible to do the calculations in the CPU chip without decrypting the data! With using the *Live migration* feature, GCP can guarantee that our VM will remain operational even during system maintenance periods. 24/7 operation does not mean that a server is running non-stop. Every now and again the servers must stopped, e.g. for system software updates, hardware component change, etc. Live migration allows VMs to be migrated to another node in the same cloud zone without user intervention.

The next key service in the infrastructure layer is the data storage. In GCP, this service is simply called **Cloud Storage**. Cloud Storage provides storage for data objects. Storage size is unlimited. Cloud Storage is accessible worldwide at a low latency and provides high durability (99.999999999% annual durability). If required, geo-redundancy as an option is available. In this case data is stored in a multi-region or dual-region manner. There are various storage classes based on the intended use of Cloud Storage.

- *Standard Storage* is the best choice for data that is accessed frequently, including websites, streaming videos, and mobile apps.
- *Nearline Storage* is a low cost option for data that can be stored for at least 30 days without access, such as data backup and long-tail multimedia content.
- *Coldline Storage* is a very low cost storage option if the data can be stored for at least 90 days without retrieval. This class is ideal for disaster recovery.
- *Archive Storage* is designed for store-only storage. This has the lowest cost of all storage classes but requires the data is stored for at least for 365 days between read operations. This storage is ideal for long-term (regulatory) data archival purposes.

The GCP network service cover the type of functionality that we discussed earlier in the Amazon AWS chapter. **Cloud Interconnect** can connect any infrastructure to the Google Cloud. Use **Cloud VPN** to create virtual private networks or **Virtual Private Cloud** to create regional or global cloud systems. The **Cloud Load Balancing** service can help distributing service requests among cloud resources and **Cloud Armor** can help protect those services from external DoS and web attacks.

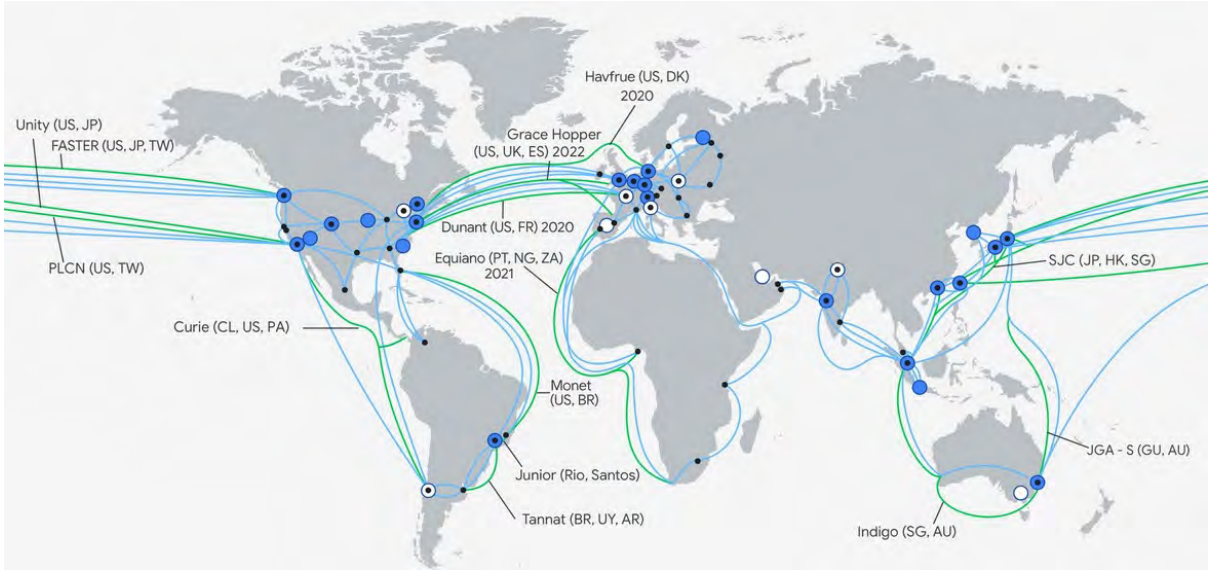

Figure 5-4 Worldwide network connections of the Google Cloud Platform

5.2 PaaS Layer Services

To simplify the deployment and execution of scalable cloud applications without the complexity of the configuration and management of the cloud infrastructure, Google Cloud Platform – similarly to other cloud providers – also offers a platform as a service product, the GCP **App Engine**. App Engine provides most of the management of the resources such as automatic scaling, security update, and detailed management (app deployment, launch, application and resource monitoring).

App Engine supports the execution of applications written for any of the following languages/platforms: Go, Java, .NET, Node.js, PHP, Python, or Ruby. It is also possible to use custom runtimes to support programs written in other languages.

App Engine will automatically connect with storage products, such as Cloud Storage, Cloud SQL, Firestore, MongoDB, or third-party databases running on-premises, or with a third-party vendor.

5.3 SaaS Layer Services

Google is famous for its innovative applications and services available for the entire world. Gmail, Google Calendar, Google Docs and Google Maps are excellent examples of these. These applications are cloud applications on their own right (software as a service) but there are many more software services (components) that can be accessed in the Google Cloud as building blocks for larger applications, including components such as vision, media, machine translation, and data analytics. We will meet and use some of these later in this book.

6 Using the Google Cloud Platform

6.1 The GCP Project

In order to use the Google Cloud Platform, users must create a GCP account then create a GCP project. The GCP project is the fundamental unit of GCP usage as it groups resources for a particular application and forms the basis for billing.

Each project must have (i) a *name*, (ii) an *ID*, and (iii) a unique *project number*. A project can be created through the GCP **console** (https://console.cloud.google.com/), which is a web-based interface to GCP. The home page is shown in Figure 6-1. The cloud **Console** provides a simple and intuitive web interface to the GCP, allowing project and resource management, monitoring, configurations, providing direct access to specific GCP services and to extensive documentation.

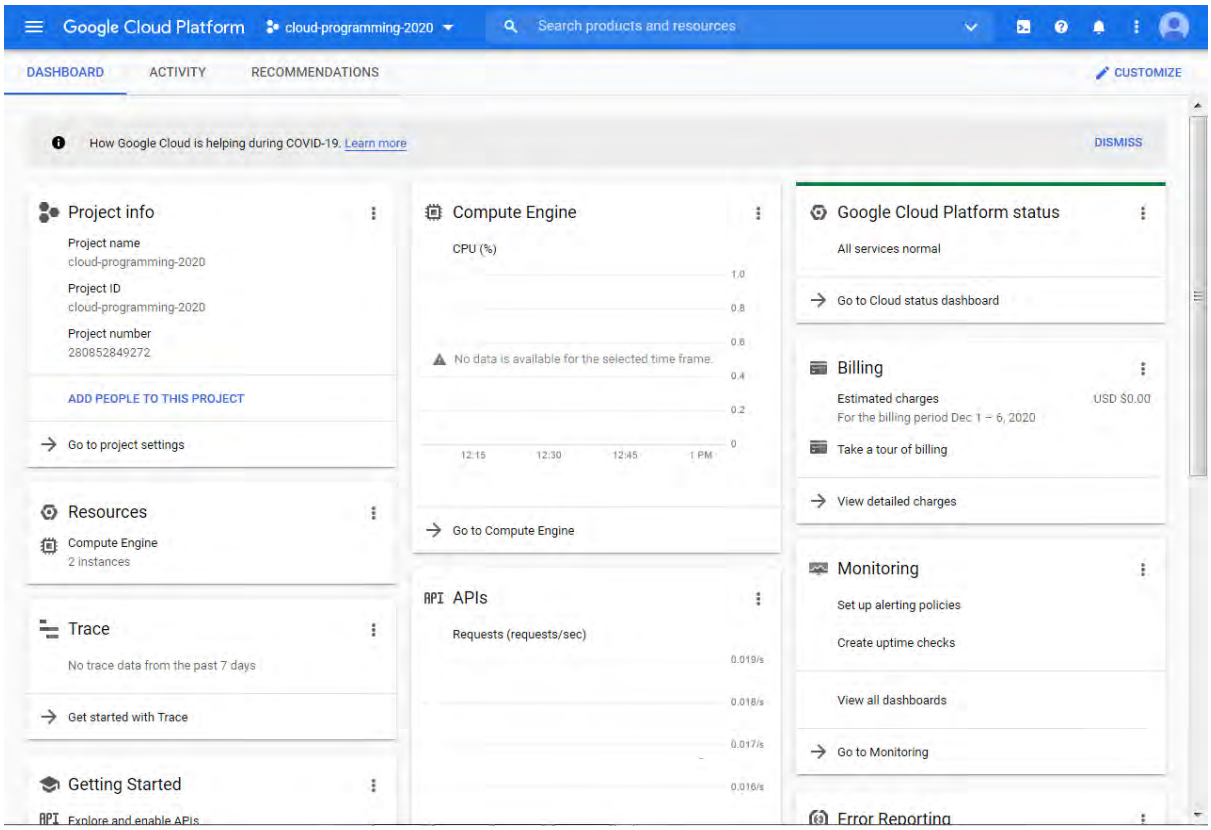

Figure 6-1 The Google Cloud Platform Console

6.2 Ways to access GCP resources

The cloud console (shown earlier) is not the only way to access GCP resources. There are various alternative ways of accessing GCP and its services. One of them is the **`gcloud`** command line interface that is most useful for expert cloud administrators. `gcloud` provides various commands arranged into a hierarchy for accessing and managing different GCP services. It is part of the Google Cloud SDK, which must be installed before `gcloud` can be used. Two simple examples below illustrate its use. More examples will be shown later when specific services and their use are discussed.

The first example lists all Compute Engine instances created in GCP zone *us-central1-a*:

```
gcloud compute instances list --filter="zone:us-central1-a"
```

The first keyword 'compute' in the command indicates that we send the command to the compute engine service. The next two keywords specify that we intend to list the instances available in the zone specified in the filter clause.

The next command lists all projects belonging to a user in a table format including project creation date and time expressed in local time zone:

```
gcloud projects list \
    --format="table(name, project_id, createTime.date(tz=LOCAL))"
```

Again, the first keyword specifies that the command will be sent to the projects while the second provides the command to list these projects.

A similar command line interface is also available within the GCP Console homepage. After clicking on the "**>_**" icon in the top right corner of the Console, the Google **Cloud Shell** window is activated at the bottom of the page (see Figure 6-2), in which `gcloud` commands can be issued to the resources. The Cloud Shell creates a temporary Compute Engine virtual machine instance in the background that will execute the commands. The VM comes with 5 GB of persistent disk storage, pre-installed Google Cloud SDK and other tools and language support for Java, Go, Python, Node.js, PHP, Ruby and .NET in the form of a built-in code editor. It also has built-in authorization for access to GCP Console projects and resources.

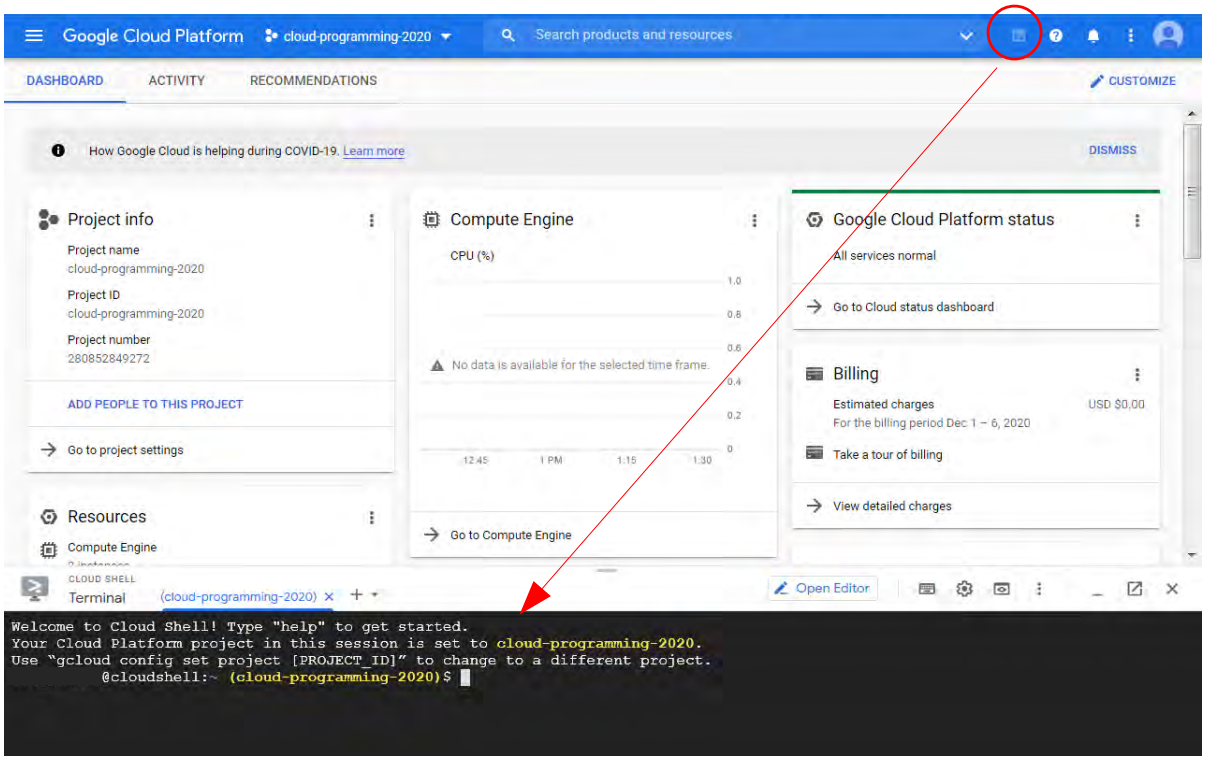

Figure 6-2 Using the Google Cloud Shell from the GCP Console

Yet another form of accessing and controlling services is the programmatic access from a system/service or application level program. In the Google Cloud Platform, software access is provided

via the Cloud Client Libraries that provides an API in the form of an SDK for software developers. The Client Libraries are available in the following programming languages: C#, C++, Java, Node.js, Go, Python, Ruby and PHP.

6.3 Using the Compute Engine

In this section, we will illustrate how we can list, start and stop Compute Engine instances programmatically using the Java Client Library. Only code snippets will be shown, the full sample program can be found at the following address: https://github.com/alfonsof/google-cloud-java-examples/tree/master/gcloudcomputeengine/src/main/java/example

Listing instances

```
Page<Instance> instancePage;

// Instantiate a client
Compute compute = ComputeOptions.getDefaultInstance().getService();

System.out.println("Listing VM instances ...");

// List instances
instancePage = compute.listInstances();
System.out.printf("Instances:\n");

Iterator<Instance> iterator = instancePage.iterateAll();
while (iterator.hasNext()) {
   Instance instance = iterator.next();
   System.out.printf("Instance: %s\n", instance.getInstanceId().getInstance());
   System.out.printf("Data instance:\n%s\n\n", instance);
}
```

Starting an instance

```
// Instantiate a client
Compute compute = ComputeOptions.getDefaultInstance().getService();
System.out.println("Starting VM instance ...");

// Start instance
Operation operation = compute.start(instanceId);
if (operation == null) {
   System.out.printf("Error: Instance \"%s\" does NOT exist.\n", instanceId);
   return;
}

// Wait for operation to complete
try {
   operation = operation.waitFor();
} catch (InterruptedException e) {
   e.printStackTrace();
} catch (TimeoutException e) {
   e.printStackTrace();
}
```

Stopping an instance

```
// Instantiate a client
 Compute compute = ComputeOptions.getDefaultInstance().getService();
 System.out.println("Starting VM instance ...");

 // Stop instance
 Operation operation = compute.stop(instanceId);
 if (operation == null) {
    System.out.printf("Error: Instance \"%s\" does NOT exist.\n", instanceId);
    return;
 }

 // Wait for operation to complete
 try {
    operation = operation.waitFor();
 } catch (InterruptedException e) {
    e.printStackTrace();
 } catch (TimeoutException e) {
    e.printStackTrace();
 }
```

6.4 Using Cloud Storage

We finish the Google Cloud Platform usage chapter with a few code snippets illustrating the programmatic access and control of the Google Storage service. First we will create a new bucket, then we will list all available buckets in the Storage.

Creating a bucket

```
import com.google.cloud.storage.Bucket;
import com.google.cloud.storage.BucketInfo;
import com.google.cloud.storage.Storage;
import com.google.cloud.storage.StorageClass;
import com.google.cloud.storage.StorageOptions;

public class CreateBucketWithStorageClassAndLocation {
  public static void createBucketWithStorageClassAndLocation(
             String projectId, String bucketName) {
    // The ID of your GCP project
    // String projectId = "your-project-id";

    // The ID to give your GCS bucket
    // String bucketName = "your-unique-bucket-name";

    Storage storage = StorageOptions.newBuilder().setProjectId(projectId)
             .build().getService();

    // See the StorageClass documentation for other valid storage classes:
    // https://googleapis.dev/java/google-cloud-
clients/latest/com/google/cloud/storage/StorageClass.html
    StorageClass storageClass = StorageClass.COLDLINE;

    // See this documentation for other valid locations:
    // http://g.co/cloud/storage/docs/bucket-locations#location-mr
    String location = "ASIA";
```

```
    Bucket bucket =
        storage.create(
            BucketInfo.newBuilder(bucketName)
                .setStorageClass(storageClass)
                .setLocation(location)
                .build());

    System.out.println(
        "Created bucket "
            + bucket.getName()
            + " in "
            + bucket.getLocation()
            + " with storage class "
            + bucket.getStorageClass());
  }
}
```

Listing buckets

```
import com.google.api.gax.paging.Page;
import com.google.cloud.storage.Bucket;
import com.google.cloud.storage.Storage;
import com.google.cloud.storage.StorageOptions;

public class ListBuckets {
  public static void listBuckets(String projectId) {
    // The ID of your GCP project
    // String projectId = "your-project-id";

    Storage storage = StorageOptions.newBuilder().setProjectId(projectId)
             .build().getService();
    Page<Bucket> buckets = storage.list();

    for (Bucket bucket : buckets.iterateAll()) {
      System.out.println(bucket.getName());
    }
  }
}
```

7 Large-scale job execution in the cloud

As we discussed earlier, the cloud emerged as a solution to the scalable and large-scale processing needs of organisations. The central question is how one can perform large-scale processing in the cloud. It is clear how to create and start a single virtual machine instance, but how does one cope with the management of hundreds or thousands of such instances? Moreover, the computing jobs must be managed too. Jobs have to be mapped onto VM instances, started, monitored for errors, and re-started in the presence of failures. The overall process of creating the required underlying compute infrastructure and executing a large number of jobs can easily become a daunting task. It is no surprise that various software frameworks had been developed to simplify this process.

7.1 The emergence of Big Data

Large compute facilities were initially used for large computational (scientific and engineering) problems. Over the last decade, however, we could witness a full digital transformation of our global society. The age of analogue data storage (paper-based documents, magnetic audio and video tapes, vinyl records, etc.) is over. Digital data generation and storage is now mainstream. Huge amount of corporate and public data are generated continuously. Documents are generated and stored in digital formats (see Figure 7-1). All sorts of data, from manufacturing to public administration, are waiting to be processed in every minute.

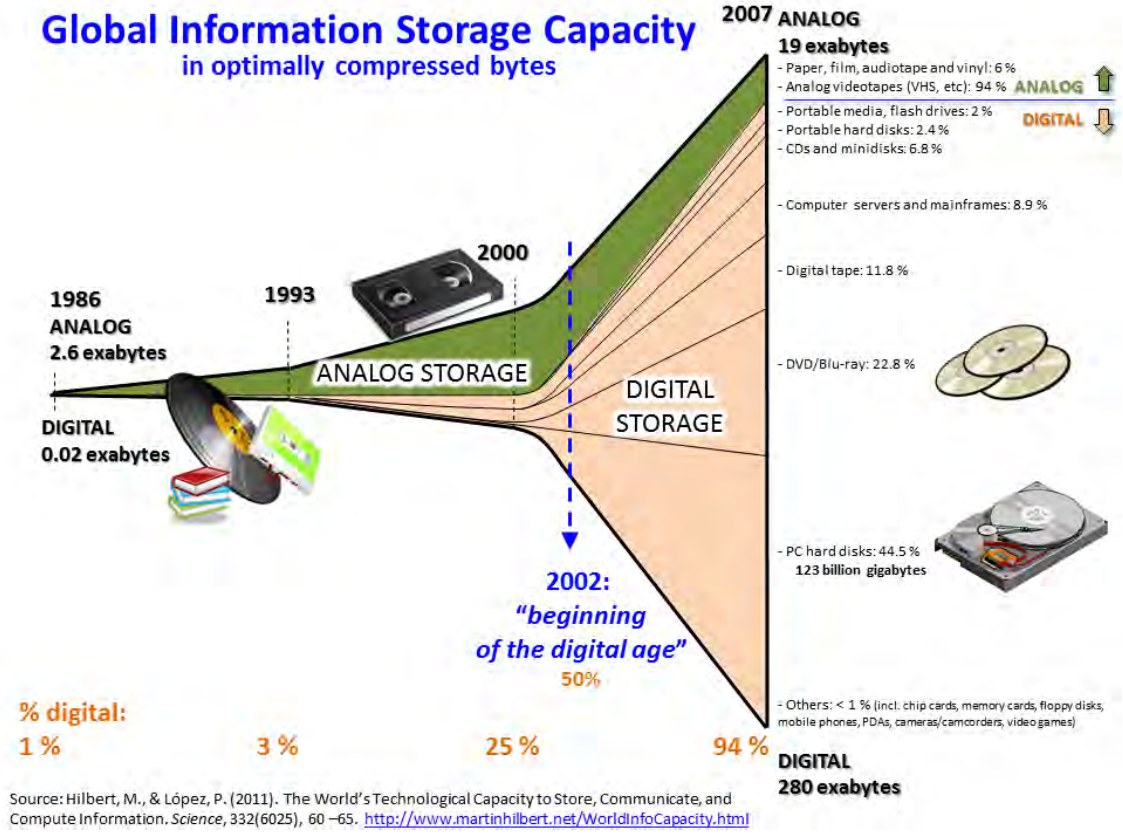

Figure 7-1 Analogue to digital transformation of data generation and storage over the past decades.

The key characteristics of this new digital data revolution is that data is generated and typically stored in raw format. We can only retrieve information form the data set through computer-based data processing; information is embedded in the data. Processing can vary from simple retrieval as in the

case of database systems to complex feature extraction and data analysis, the latter including artificial intelligence assisted approaches to discovering new patterns and trends in the data.

This new, large-scale data processing technology is called Big Data and Big Data processing. A more precise definition specifies that we describe a systems as Big Data if:

- it consists of large enough datasets that typical systems are incapable of handling, and
- it requires new processing methods to cope with this amount of data processing in a timely manner

Big Data systems are also characterised by the "V" model, based on three keywords: *volume*, *velocity* and *variety* with the following meaning:

- **Volume**: data size is orders of magnitude larger than in conventional systems
- **Velocity**: the speed with which information moves around in the system is significantly higher than in traditional systems
- **Variety**: the data originates from a wide range of data sources, and the quality and types of data varies.

The natural consequence of Big Data systems is the need for large-scale computational platforms, which implies – due to the digital data volume explosion discussed above – that these platforms will dominantly perform data processing, transformation and analysis tasks. It seems that large systems are no longer the exclusive "toys" of scientists running numerical simulations.

It is also important to note that data processing is different in many ways from traditional high-performance computing. In HPC, most applications are based on unique, individual algorithms optimised for the fast solution of a specific scientific computation problem. Big Data processing is different from this as the data processing follows a rather standard series of steps. Let us call this the life cycle of Big Data processing:

Stage 1 Ingest /collect/ data -- this stage includes extraction, sorting, labelling, and transformation of data

Stage 2 Persist data in long term storage – requires a distributed filesystem or database

Stage 3 Process and analyse – batch vs. real-time processing including query processing, statistical and data analysis algorithms

Stage 4 Visualise results – detect trends, changes, give insights

7.2 Data processing software frameworks

In the past, large-scale data processing applications were executed on large compute clusters. With the advent of the cloud, many of these systems migrated to the cloud in the form of data processing frameworks that simplify the complex execution and resource management steps found in these applications. In this section, we will overview and compare the most popular and well-known cloud data processing frameworks based on their functionality, capabilities and efficiency. We will cover the following software packages: Apache Hadoop, Apache Spark, Storm, Kafka, Flink, Beam. We will focus on open-source frameworks because vendor lock-in is especially dangerous in the cloud. If a user relies on a service or software framework that is only available or executable in one particular cloud, the possibility of changing the service provider in the future is practically zero.

Before engaging in the framework overview, however, we need to discuss a fundamental concept related to data processing, namely the batch and stream-oriented approaches to data processing, as this will be crucial for understanding the differences among the frameworks.

7.2.1 Batch and stream data processing

The traditional way of processing data is the following. Input data is stored in multiple input files of in a database system. A program is launched to process the input data and generate results. The program will access and read the entire data set into memory, perform the required computation, and – once the results are available – save the results into output files or store them in a database. The key characteristics of this execution model is that input data is finite (bounded), static and persistent; the program has access to the complete sets of data records and execution is staged (i.e. data input –> data processing –> data output). This style of execution is called *batch processing*.

There are situations when the batch execution model is not applicable or practical. In many modern applications, data is generated in real-time. Examples include live media (video, audio, text, messaging), IoT systems (traffic data – autonomous driving, traffic jams; manufacturing – process and assembly data and logs), or physical and social networks (live data, various logs, security data). In these applications, one cannot wait for the entire input data set to become available for processing. A continuous real-time data stream will never finish! The alternative is to process small chunks or records of data as soon as they become available. Besides the obvious advantage that our system can now process the data, there are other benefits as well. Since only a small portion of the entire data set is held in memory, the resource requirements of these applications are lower. If we process data by records, the reactiveness and latency of our system will improve. Consequently, this mode of data processing, which we call *stream-oriented data processing* or simply *stream processing*, is beneficial to improve performance and system utilisation and can be used even when the whole data is available for processing. Figure 7-2 illustrated the concept of stream processing in comparison to batch processing.

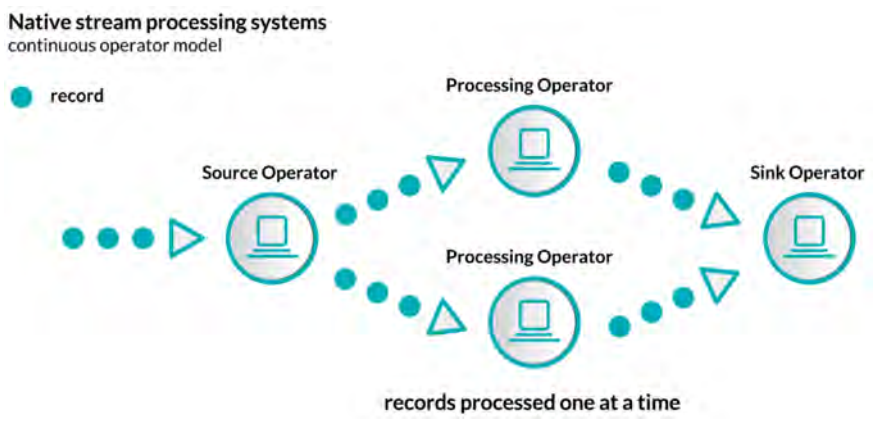

Figure 7-2 The concept of stream processing on an unknown size data set

7.2.2 Apache Hadoop/MapReduce

The Hadoop/MapReduce framework is an Apache open-source project [14] created to support the execution of large-scale batch data processing applications based on the Map-Reduce programming

model. The underlying infrastructure is a large cluster of commodity servers. On top of this runs Hadoop, a software system written in Java providing a fault-tolerant and platform independent execution environment. The system has the following main components:

- HDFS - the Hadoop Distributed File System
- YARN - a cluster coordinator/scheduler
- MapReduce – the batch processing engine
- Hadoop common – a set of common utilities

The philosophy of Hadoop/MapReduce is that large data processing tasks are executed in parallel. The input data is divided into smaller chunks that are assigned to different servers for simultaneous execution (the "Map" phase), than the partial results are collected to form the final results ("Reduce" phase). The input data is stored in a large fault-tolerant distributed filesystem. Hadoop is built on the assumption that in a distributed system failure is not the exception but the norm. If we use thousands of servers, there will always be one that is not working; therefore, fault-tolerance has to be an essential feature of the runtime system. Hadoop provides application-level fault tolerance support.

The system consist of *compute* and *storage* nodes; these are normally are the same, which allows job execution without unnecessary extra data transfer. Jobs are submitted to the system from clients, then the Hadoop execution environment takes over the scheduling and management of the jobs.

Execution is controlled by the YARN engine that consists of a single master ResourceManager, one worker NodeManager per cluster-node, and MRAppMaster per application (see Figure 7-3). The job (JAR file or executable) will be received by the ResourceManager, which schedules the subtasks to the nodes and provide monitoring and feedback to clients using information received from the NodeManagers.

A MapReduce application has a simple structure that can be followed with a programming model provided by interfaces. The execution model is as follows:

1. read data set from filesystem
2. divide data into chunks (key-value pairs)
3. apply computation to subsets on different nodes: **Map process**
4. **reduce** value of each key: **Reduce process**
5. write result back to filesystem

In general, a MapReduce program will create a set of key-value pairs while executing the various stages of the program, as shown here

(input) `<k1,v1> ->` **map** `-> <k2,v2> ->` **combine** `-> <k2,v2> ->` **reduce** `-> <k3,v3>` (output)

K1, K2, K3 and V1, V2, V3 are different key and value classes, which must all be serializable for execution. The execution of the program involves several phases, such as

- Input phase: the input phase reads and parses input data and forwards it to the Mapper in the form of key-value pairs
- Map phase: user-defined processing implemented as a function that processes incoming key-value pairs and generate new output key-value pairs. Intermediate keys and values are used to forward the results of the Mapper to the next phase.
- Combiner phase: this is an optional step used to do local reduce operations on a single node (Mapper process).

- Shuffle and Sort phase: this is the starting step of the Reducer process. The Reducer will sort the input key-value pairs by the key values to make the reduction process more efficient.
- Reducer phase: in this phase, the user-defined reduction function is executed to aggregate, filter or combine the data produced by the map processes. The Reducer will output zero more key-value pairs as the result of the program.
- Output phase: the final phase of the program which takes the result key-value pairs and saves them in the required format to the file system.

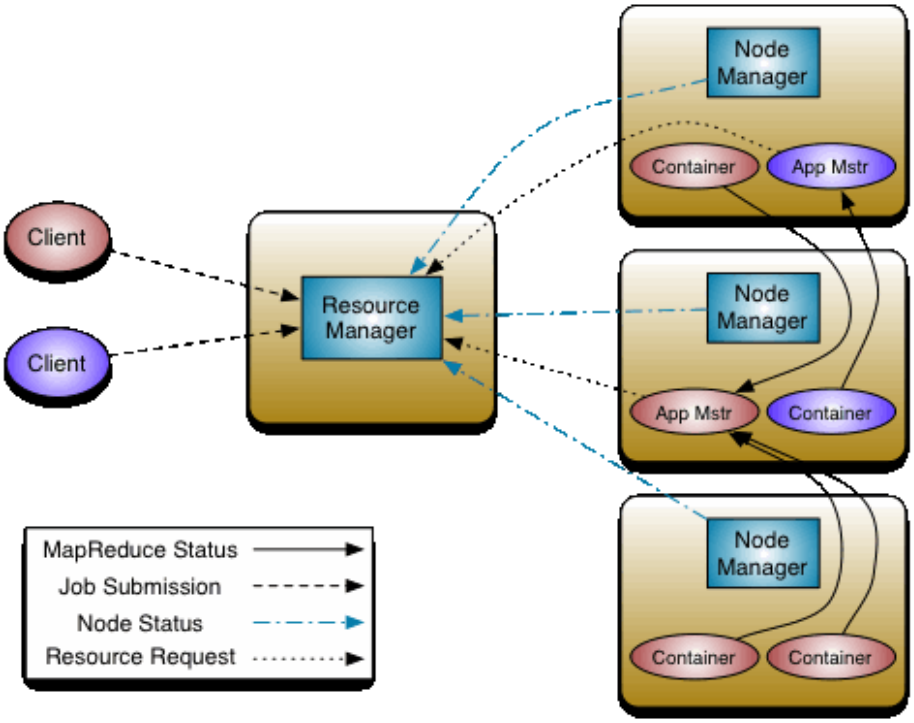

Figure 7-3 Hadoop architecture and job execution mechanism.

To make the execution easier to understand, a simple program is used as illustration. The example is a simple Word Count application that counts the number of occurrences of each word in a given input set. The phases of the execution are illustrated in Figure 7-4. The example will use two classes to implement the required functionality. The class `TokenizerMapper` implements the Mapper interface and split the input file into words and emit a <<word>, 1> as result. The class `IntSumReducer` implements the Reducer interface and will calculate the sum of the values of a given text key. This class will be used as a local (node-level) reducer and as a global (system-level) reducer as well. The main method of the program configures the program; first we obtain the job configuration, set the executable JAR file, specify the mapper, combiner and reducer classes, and finally the output Key and Value classes and the input/output paths. The program will execute when the waitForCompletion() method is called.

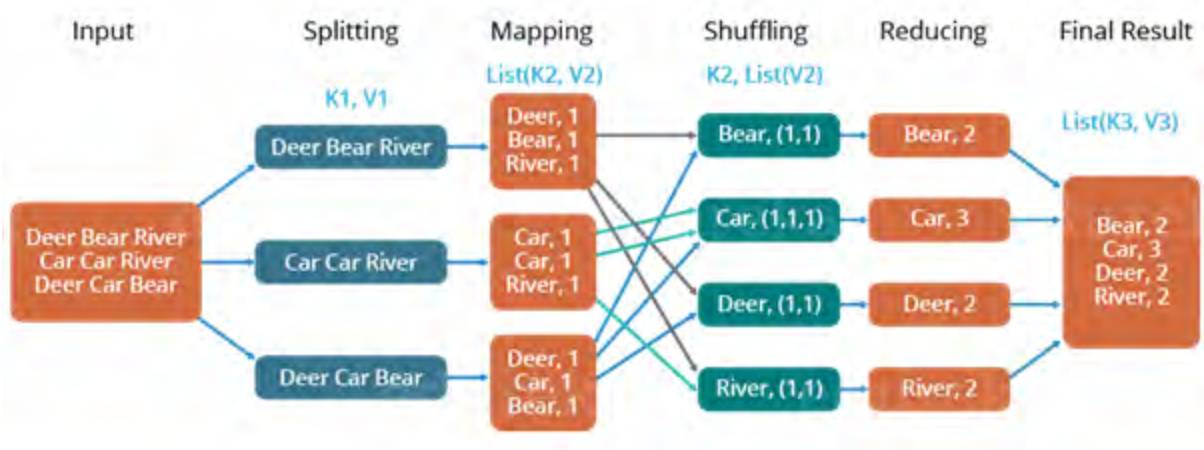

Figure 7-4 Phases of the WordCount example program execution.

Wordcount Example

```
import java.io.IOException;
import java.util.StringTokenizer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

  public static class TokenizerMapper extends Mapper<Object, Text, Text, IntWritable>{

    private final static IntWritable one = new IntWritable(1);
    private Text word = new Text();

    public void map(Object key, Text value, Context context) throws IOException,
                            InterruptedException {
      StringTokenizer itr = new StringTokenizer(value.toString());
      while (itr.hasMoreTokens()) {
        word.set(itr.nextToken());
        context.write(word, one);
      }
    }
  }

  public static class IntSumReducer extends Reducer<Text,IntWritable,Text,IntWritable> {
    private IntWritable result = new IntWritable();

    public void reduce(Text key, Iterable<IntWritable> values, Context context) throws
                            IOException, InterruptedException {
      int sum = 0;
      for (IntWritable val : values) {
        sum += val.get();
      }
      result.set(sum);
      context.write(key, result);
    }
  }
```

```
  public static void main(String[] args) throws Exception {
    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "word count");
    job.setJarByClass(WordCount.class);
    job.setMapperClass(TokenizerMapper.class);
    job.setCombinerClass(IntSumReducer.class);
    job.setReducerClass(IntSumReducer.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));
    System.exit(job.waitForCompletion(true) ? 0 : 1);
  }
}
```

7.2.3 Apache Spark

The Hadoop/MapReduce framework is a widely used system to process extremely large datasets. It has one fundamental limitations though, which is that it uses the file system for intermediate data transfer. This is serious performance bottleneck, since file system bandwidth is several orders of magnitudes lower than memory bandwidth. If a system is used for processing large datasets, this may become a prohibitive problem. Identifying this deficiency, a MapReduce side project was started in 2009 to create an in-memory version of MapReduce. Few years later, this project became an independent Apache project named Spark [15].

Spark is a very fast alternative framework executing MapReduce style programs. When using in-memory operations, it can be up to 100 times faster than a Hadoop/MapReduce program. If it is running on disks, it can still be up to 10 times faster due to the reduced number of I/O operations required. Besides speed, Spark has other advantages. It provides APIs for several languages, e.g. , and provides richer functionalities than the 'Map' and 'Reduce' steps: it supports SQL queries, machine learning and graph analytics operations as well as streaming data processing capabilities.

Spark has its own built-in cluster management system, which means it can be executed without the help of other cluster execution environments. However, if required, it can make use of third party cluster runtimes. The execution modes of Spark are as follows:

- Standalone mode: Spark will run as an independent cluster-based environment.
- Hadoop mode: Spark will run on top of the Hadoop YARN cluster management system.
- Spark in MapReduce mode: Spark jobs will be executed as Hadoop/MapReduce jobs. This mode helps in integrating legacy MapReduce programs with Spark.

Spark has become the leading data processing/analytics framework. Its rich functionality and high performance made it successful in a large variety in application areas. Figure 7-5 illustrates the Spark layered architecture as well as its support to various application programming technologies, execution environments and data sources.

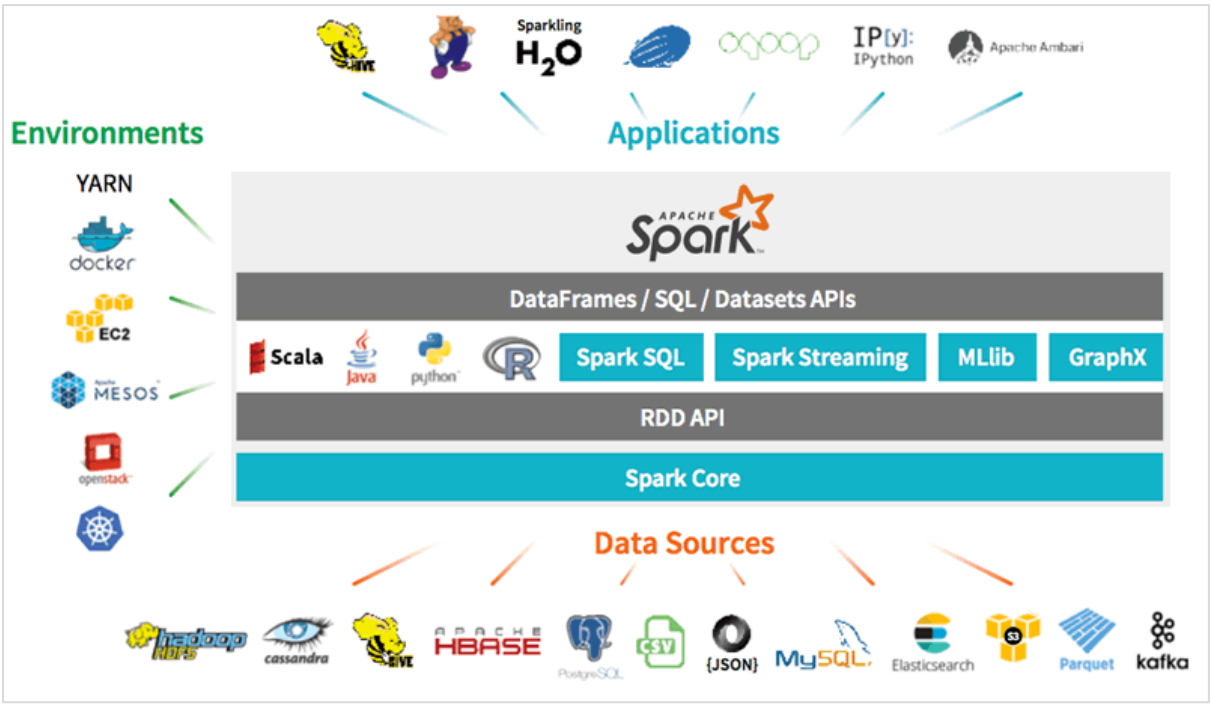

Figure 7-5 Illustration of Spark's rich support for third party applications, runtime environments and data sources.

As before, we show the Word Count example but this time implemented in Spark Java. The code illustrates the key steps of the program. First a Spark session is created that will be used to access input and stop the program. The input is translated into Spark's internal data representation structure, Resilient Distributed Datasets (RDD in short). This distributed data structure allows Spark to execute operations in parallel on different nodes with high efficiency. Figure 7-6 shows how data is stored between the different stages of computation in MapReduce. Tuples are stored on disk in the file systems. In contrast, as seen in Figure 7-7, Spark uses the distributed memory (RDD) for intermediate storage. The file system is only accessed at the beginning and the end of the program.

Once the list of words is produced in the program, key-value pairs are created and stored in the `ones` data structure as the result of the *map* step. The next line performs the *reduce* operation on the ones, and results in `counts` data structure. Finally, the `counts` is transformed into a List and the results are printed to the console.

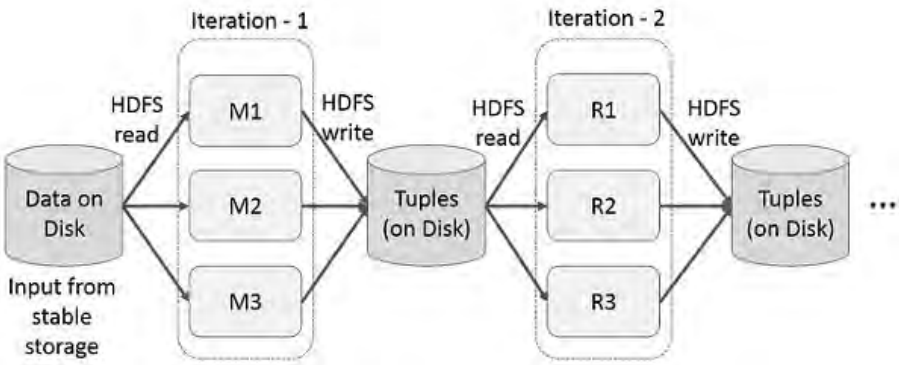

Figure 7-6 Intermediate data communication in Hadoop/MapReduce

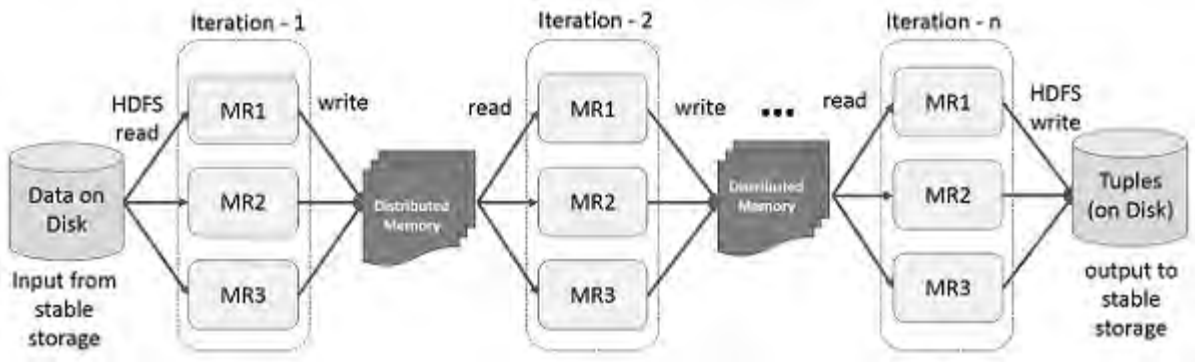

Figure 7-7 Intermediate data communication in Spark

WordCount Spark example[13]

```
package org.apache.spark.examples;

import scala.Tuple2;
import org.apache.spark.api.java.JavaPairRDD;
import org.apache.spark.api.java.JavaRDD;
import org.apache.spark.sql.SparkSession;
import java.util.Arrays;
import java.util.List;
import java.util.regex.Pattern;

public final class JavaWordCount {
  private static final Pattern SPACE = Pattern.compile(" ");

  public static void main(String[] args) throws Exception {

    if (args.length < 1) {
      System.err.println("Usage: JavaWordCount <file>");
      System.exit(1);
    }

    SparkSession spark = SparkSession
      .builder()
      .appName("JavaWordCount")
      .getOrCreate();

    JavaRDD<String> lines = spark.read().textFile(args[0]).javaRDD();

    JavaRDD<String> words = lines.flatMap(s -> Arrays.asList(SPACE.split(s))
                                  .iterator() );

    JavaPairRDD<String, Integer> ones = words.mapToPair(s -> new Tuple2<>(s, 1));
    JavaPairRDD<String, Integer> counts = ones.reduceByKey((i1, i2) -> i1 + i2);

    List<Tuple2<String, Integer>> output = counts.collect();
    for (Tuple2<?,?> tuple : output) {
      System.out.println(tuple._1() + ": " + tuple._2());
```

[13] https://github.com/apache/spark/blob/master/examples/src/main/java/org/apache/spark/examples/JavaWordCount.java

```
    }
    spark.stop();
  }
}
```

7.3 Executing MapReduce and Spark jobs on GCP

In this section we look at how MapReduce and Spark jobs can be executed on the Google Cloud Platform. GCP supports native Hadoop and Spark job execution using the GCP **Dataproc** service. Dataproc is a managed GCP service enabling fast creation of compute clusters and efficient execution of data processing jobs. In addition to being able to run MapReduce or Spark jobs, Dataproc is tightly integrated with other GCP services (e.g. BigQuery, Cloud Storage, Cloud Bigtable, Cloud Logging, and Cloud Monitoring) providing a full data platform (see Figure 7-8).

There are several ways of accessing Dataproc, through (i) the REST API, (ii) the Cloud SDK, (iii) the Cloud Dataproc UI, or (iv) via the Cloud Client Libraries.

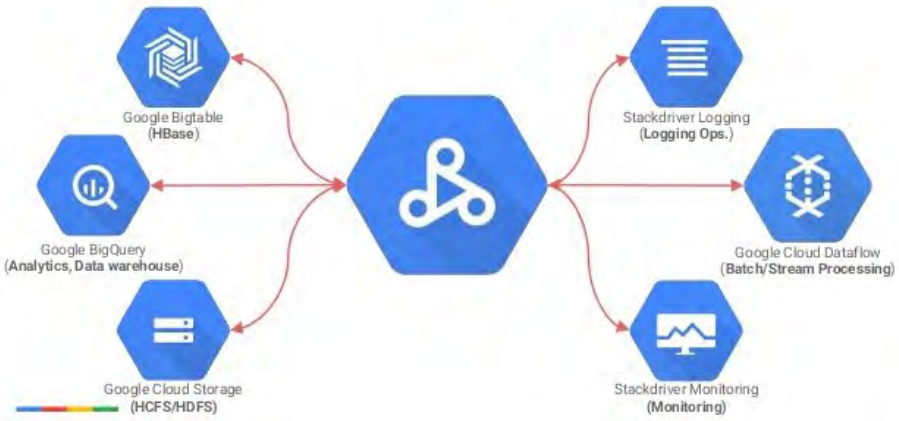

Figure 7-8 Dataproc service (in centre) and its possible connections to other services.

We will use the WordCount example program to illustrate execution on the Google Cloud Platform. We assume that a MapReduce and Spark version of the sample is already compiled and placed in JAR files. As a side note, more information on the operation of Dataproc can be found at the following link: https://cloud.google.com/dataproc/docs.

7.3.1 Running MapReduce jobs on GCP

The traditional execution command for the example on and installed and running Hadoop cluster would be as follows:

```
>hadoop jar hadoop-mapreduce-example.jar WordCount /sample/input /sample/output
```

Since we are using the Google Cloud, we will need a slightly modified command line and additional setup steps. Executing the program on GCP will involve the following steps:

1. Compiling the sample Java program
2. Creating a Storage bucket, and uploading the file to be processed
3. Creating a Google Cloud Dataproc cluster
4. Submitting the job for execution
5. Inspecting the result

Step 1 is skipped as we assume the program is already compiled. The program will us the Google Cloud Client Libraries to access services.

Creating storage and uploading input file

For the example, we will use Google Storage as the data storage platform. We create a unique bucket e.g. word-count-bucket

`gs://test-bucket22/*`

The input file is then uploaded to the Storage.

Create an input file `words.txt` with the following content and upload it to the test-bucket22:

```
apple pear apple microsoft pear door window table chair chair
```

Create the cluster and execute program

Once the previous steps are complete, we can create a Hadoop cluster from the Google Shell

```
>gcloud dataproc clusters create hadoop-cluster --region europe-west4
```

The option `create hadoop-cluster` will give the name `hadoop-cluster` to the created cluster and will be used by Dataproc as a native Hadoop cluster. Starting the cluster should be fast. Once the cluster is up, the program is executed with the following command.

```
>gcloud dataproc jobs submit hadoop --cluster hadoop-cluster --jar
    gs://test-bucket22/WordCountMapReduce.jar -- wordcount.WordCount
    gs://test-bucket22/words.txt gs://test-bucket22/results
```

As specified, the results of the program will be listed in `gs://test-bucket22/results.` The content of the file can be listed as:

```
>gsutil cat gs://test-bucket22/results/*

microsoft       1
window  1
pear    2
chair   2
apple   2
door    1
table   1
```

7.3.2 Running Spark jobs on GCP

Executing a Spark job is not more complicated than the previous example. We create the cluster the same way (or) resuse the existing cluster and submit a spark job. Input and output files remain the same. We assume again, that the program containing the Spark example is already compiled and stored in a JAR file.

Once the previous steps are complete, we can create a Hadoop cluster from the Google Shell

```
>gcloud dataproc clusters create spark-cluster --region europe-west4
```

Once the cluster is up, the program is executed with the following command.

```
>gcloud dataproc jobs submit spark --cluster spark-cluster --jar
    gs://test-bucket22/WordCountSpark.jar -- wordcount.WordCount
    gs://test-bucket22/words.txt gs://test-bucket22/results
```

As before, the results of the program will be listed in `gs://test-bucket22/results`.

In both cases, execution logs can be found and viewed in the Cloud console under the Dataproc service. If Cloud logging is required, the following properties must be set when creating the clusters.

```
dataproc:dataproc.logging.stackdriver.enable=true
```

If we want to enable job resourcing of YARN container logs, the following property is required when creating the cluster:

```
dataproc:dataproc.logging.stackdriver.job.yarn.container.enable=true
```

GCP also provides monitoring capabilities. The execution of the program and the properties of the cluster can be monitored as shown in Figure 7-9.

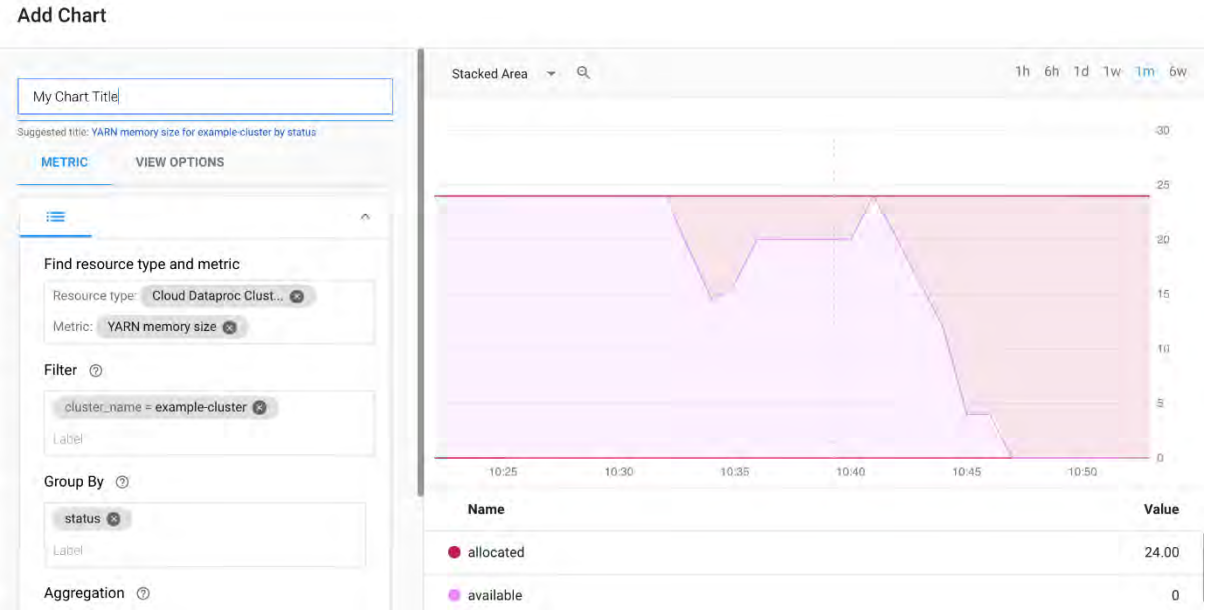

Figure 7-9 An example for the Hadoop cluster monitoring window

8 Data stream processing

As discussed in Section 7.2.1, not all data processing tasks can be performed in the batch execution style. Continuous data streams, e.g. real-time measurement or monitoring data, cannot be processed with the traditional input—process—output paradigm. In these cases, streaming data processing and stream algorithms can help us.

In addition to providing an execution mechanism, stream processing can improve system performance (decrease latency) and reduce resource consumption (less memory is used as only a portion of data is used at any one time during processing).

In this section, we briefly overview the key characteristics of stream processing, then overview and compare the most popular stream data processing software frameworks. We will finish this chapter with an introduction to the Beam framework and its use in the Google Cloud Platform.

8.1 Stream algorithms

The most important feature of stream data processing that makes it different from batch processing is the continuous, unbound incoming data stream. The algorithms must be able to process data as data elements arrive without having access to the entire, complete input data set.

A simple statistical computational problem can shed light on this distinction nicely. Assume that we need to calculate the variance of N data samples x_i, $i = 1, \dots, N$. The variance of the data is defined by the following well-known formula:

$$Var(X) = \frac{1}{n}\sum_{1}^{n}(x_i - \mu)^2$$

where μ is the arithmetic mean or expected value of x_i

$$\mu = \frac{1}{n}\sum_{1}^{n} x_i$$

When we have access to the entire data set, i.e. all N elements in the memory or in an input file, one option for implementation is to use a two-pass algorithm. In the first pass, the mean is computed; in the second pass, the variance is calculated. The Java implementation is as follows.

```
double varianceTwoPass(double[] data){

    // first pass: calculate mean
    double mean = 0;
    for (int i = 0; i < data.length; i++) {
        mean += data[i];
    }
    mean /= data.length;

    // second pass: calculate sample variance
    double variance = 0.0;
    for (int i = 0; i < data.length; i++) {
        variance += (data[i]-mean)*(data[i]-mean);
    }
    return variance/data.length;
}
```

It can be tempting to avoid the two passes over the data elements by merging the implementation into one loop using the following alternative definition

$$Var(X) = \frac{1}{n}\sum_{1}^{n}(x_i - \mu)^2 = \frac{1}{n}\sum_{1}^{n}{x_i}^2 - \mu^2 = \frac{1}{n}\left[\sum_{1}^{n}{x_i}^2 - \frac{1}{n}\left(\sum_{1}^{n}x_i\right)^2\right]$$

The first term of the final formula is the sum of squares, while the second term is the sum of the elements squared divided by the number of elements. This can be implemented in Java as follows.

```
    double varianceSinglePass(double[] data){

        double sum = 0;
        double sumOfSquares = 0;
        double mean = 0;

        double variance = 0.0;
        for (int i = 0; i < data.length; i++) {
            sum += data[i];
            sumOfSquares += data[i] * data[i];
        }
        return (sumOfSquares – (sum * sum)/data.length) / data.length;
    }
```

The problem with this implementation is that the values of sumOfSquares and `(sum * sum)/data.length` can be very similar, their differences resulting in so small values that cannot be represented with the available precision of floating point arithmetic. This problem is called "catastrophic cancellation" [16] and leads to incorrect results. Consequently, this algorithm should never be used in practice!

A one-pass algorithm that provides correct results and does not suffer from catastrophic cancellation was presented by Welford [17]. This algorithm is a one-pass – also called online – algorithm that updates the mean and variance as new samples are visited. Let us first see how this works for the calculation of the mean.

Let us have the mean values, m_k, $k = 1, \dots, N$, defined as: $m_1 = x_1/1$, $m_2 = (x_1 + x_2)/2$, $m_3 = (x_1 + x_2 + x_3)/3$, and so on. It can be seen that m_k can be expressed using m_{k-1} as

$$m_k = \frac{(n-1)m_{k-1} + x_n}{n}$$

which can be further simplified as

$$\begin{aligned} m_k &= \frac{(n-1)m_{k-1} + x_n}{n} \\ &= \frac{[(n-1)m_{k-1} + m_{k-1}] - m_{k-1} + x_n}{n} \\ &= m_{k-1} + \frac{x_n - m_{k-1}}{n} \end{aligned}$$

This shows that the mean for any number of values can be computed iteratively by using only the current value and the mean from the previous iteration.

The interesting question is whether the variance can be computed by a similar recurrence relation. We will use the following identities $x_k - m_n = x_k - m_{n-1} - (k_n - m_{n-1})/n$ and $x_n - m_n = (n -$

$1)(x_n - m_{n-1})/n$ and use S_n to represent the sum term in the variance formula: $S_n = \sum_1^n (x_i - \mu)^2$ with this the variance can be expressed as

$$Var(X) = \frac{1}{n} S_n$$

We want to arrive at a recurrence formula for S_n similar to the one for the mean calculation, in the form of

$$S_n = S_{n-1} + M_n$$

We will derive M_n by taking the difference $S_n - S_{n-1}$.

$$M_n = S_n - S_{n-1} = \sum_{i=1}^{n} (x_i - m_n)^2 - \sum_{i=1}^{n-1} (x_i - m_{n-1})^2$$

Splitting the first summation term as

$$\sum_{i=1}^{n} (x_i - m_n)^2 = \sum_{i=1}^{n-1} (x_i - m_n)^2 + (x_n - m_n)^2$$

we can rearrange M_n as

$$M_n = (x_n - m_n)^2 + \sum_{i=1}^{n-1} ((x_i - m_n)^2 - (x_i - m_{n-1})^2)$$

Using the identity $(a^2 - b^2) = (a + b)(a - b)$, we term inside the summation can be rewritten as

$$\begin{aligned} \left(x_i - m_{n_1}\right)^2 - (x_i - m_{n-1})^2 &= (x_i - m_n + x_i - m_{n-1})(x_i - m_n - x_i + m_{n-1}) \\ &= (x_i - m_n + x_i - m_{n-1})(m_{n-1} - m_n) \end{aligned}$$

In the subsequent steps of the manipulation of the above term we will use the fact that

$$\sum_{i=1}^{n} (x_i - m_n)^2 = 0$$

Since

$$m_n = \frac{1}{n} \sum_{i=1}^{n} x_i$$

and

$$n \cdot m_n = \sum_{i=1}^{n} x_i$$

$n \cdot m_n$ is identical to $\sum_1^n m_n$, hence

$$\sum_{i=1}^{n} m_n = \sum_{i=1}^{n} x_i$$

Rearranging the equation, we get

$$\sum_{i=1}^{n} m_n - \sum_{i=1}^{n} x_i = \sum_{i=1}^{n} (x_i - m_n) = 0$$

Which is also zero if we square the bracketed term. Returning to the formula for M_n, and manipulating it further, we get

$$\begin{aligned} M_n &= (x_n - m_n)^2 + \sum_{i=1}^{n-1} ((x_i - m_n)^2 - (x_i - m_{n-1})^2) \\ &= (x_n - m_n)^2 + \sum_{i=1}^{n-1} (x_i - m_n + x_i - m_{n-1})(m_{n-1} - m_n) \\ &= (x_n - m_n)^2 + (m_{n-1} - m_n) \sum_{i=1}^{n-1} (x_i - m_n + x_i - m_{n-1}) \end{aligned}$$

Rewriting the term in the summation as

$$\begin{aligned} &\sum_{i=1}^{n-1} (x_i - m_n + x_i - m_{n-1}) = \sum_{i=1}^{n-1} \big((x_i - m_n) + (x_i - m_{n-1})\big) \\ &= \sum_{i=1}^{n-1} (x_i - m_n) + \sum_{i=1}^{n-1} (x_i - m_{n-1}) \end{aligned}$$

Based on the summation identity above, the last summation term becomes zero leaving us with

$$\sum_{i=1}^{n} (x_i - m_n + x_i - m_{n-1}) = \sum_{i=1}^{n} (x_i - m_n)$$

We will manipulate this term a little bit further to simplify it even more.

$$\begin{aligned} \sum_{i=1}^{n-1} (x_i - m_n) &= \left(\sum_{i=1}^{n-1} (x_i - m_n) \right) + (x_n - m_n) - (x_n - m_n) \\ &= \left(\sum_{i=1}^{n-1} (x_i - m_n) + (x_n - m_n) \right) - (x_n - m_n) \\ &= \left(\sum_{i=1}^{n} (x_i - m_n) \right) - (x_n - m_n) \end{aligned}$$

The first term of the right hand size becomes zero gain due to the identity, leaving us with

$$\sum_{i=1}^{n-1} (x_i - m_n) = (m_n - x_n)$$

Returning to our original equation for Mn and substituting the above result, we get

$$\begin{aligned} M_n &= (x_n - m_n)^2 + (m_{n-1} - m_n) \sum_{i=1}^{n-1} (x_i - m_n + x_i - m_{n-1}) \\ &= (x_n - m_n)^2 + (m_{n-1} - m_n)(m_n - x_n) \end{aligned}$$

$$= (x_n - m_n)[(x_n - m_n) - (m_{n-1} - m_n)]$$
$$= (x_n - m_n)(x_n - m_{n-1})$$

With this, we arrive at the final recurrence formula for S_n:

$$S_n = S_{n-1} + M_n$$
$$= S_{n-1} + (x_n - m_n)(x_n - m_{n-1})$$

from which the variance can be computed as $Var(X) = Sn/n$.

The importance of Welford's algorithm is that besides providing a numerically stable one-pass method for calculating the mean and variance, it can be used without having the entire data set known. It can be used for a data sequence of unknown length. It also demonstrates that streaming algorithms often require algorithmic changes sometimes going back to the fundamental mathematical basis to find a solution that can be executed in a streaming version. The calculation of variance is a nice example of this.

Assume a producer-consumer system, where the producer generates a series of data elements and sends them to the consumer that will calculate the mean and variance of the data. The Welford algorithm can be used for the calculation of this data stream without the need to store the data elements in memory. The Java implementation of the consumer is below. We further assume that the producer and consumer communicate via an `ArrayBlockingQueue` instance.

```
private class Consumer extends Thread{
        private ArrayBlockingQueue data;
        private long n;
        private double mean;
        private double sn;
        private double variance;

        public Consumer(ArrayBlockingQueue data) {
            this.data = data;
        }

        public void run(){
            while (true) {
                try {
                    int x = data.take();
                    n++;
                    double oldMean = mean;
                    mean = mean + (x - mean)/n;
                    sn = sn + (x - mean)* (x - oldMean);
                    variance = sn/n;
                } catch (InterruptedException ex) {
                   ex.printStacktrace();
                }
            }
        }

    }
```

Naturally, not all problems can be expressed using single data elements only. If an application needs. e.g. averages over a certain period, say daily, weekly and monthly mean temperature or precipitation, stream algorithms operate on *data windows*. These windows can be overlapping or non-overlapping. If the windows do not overlap, we often refer to them as data segments. Overlapping windows are often referred to as sliding windows. Sliding windows are routinely used in time series analysis to compute various statistical metrics and data trends.

Streaming data systems are typically implemented as a pipeline of processes following the pattern of a general producer-consumer system. Processes are background threads that consume, process and publish data continuously until stopped. Processes are connected by bounded channels, as shown in Figure 8-1, normally implemented by message queues.

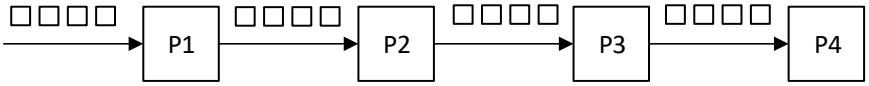

Figure 8-1 A generic stream processing system consisting of four stages. Incoming data elements are processed in sequence by P1 and the results are forwarded to P2 (and to P3 and P4) that will process and communicate in a similar manner.

Implementing streaming data processing systems is a complex endeavour. Efficiency, reliability, fault-tolerance, scalability and ease of programming are crucial requirements. Since a large portion of the in these systems is concentrated on the runtime system, it make sense to try to separate the execution logic and the runtime from each other. In the part of this chapter, we will look at the result of such runtime framework development efforts, and how they can help the development of streaming applications.

8.2 Popular stream data processing frameworks

The 24/7 style availability and operation of applications in the cloud brought with it the emergence of data stream applications. If we can compute continuously in the cloud, why not use this capability for processing continuous data streams. The proliferation of these applications then, naturally, led to the development of stream processing frameworks. We will look first at Spark that offers support for stream data processing in the form of microbatches. Many users found that this abstraction and the achieved performance unsuitable for their needs, hence several new stream frameworks appeared in the past decade. We will continue with the overview of these.

8.2.1 Spark Streaming

Spark Streaming [18] is an extension to Spark to support processing of live data streams. It can receive data from a variety of data sources and store results after processing in file system of databases (see Figure 8-2). The implementation is based on the concept of microbatches. As shown in Figure 8-3, Spark Streaming receives continuous input data and segments it into small batches of data (microbatches) that are subsequently sent to the regular Spark system for execution as a sequence of individual batch jobs. These microbatches are abstracted out internally as *discretised streams* or *DStreams* in short. The DStream data structure in Spark Streaming is a sequence of RDDs.

Figure 8-2 Supported data connections of Spark Streaming to third party data sources and sinks

Figure 8-3 Implementation of stream execution on top of the Spark batch engine[14]

The batch Word Count example implementation must be changed to support stream operations. The stream version[15] uses a socket based stream to receive words in a continuous fashion. Each unit will be a line of text stored in a `JavaReceiverInputDStream` type variable. On this variable, we execute the familiar map operation to produce a sequence of words (of type `JavaDStream`), on which we execute the reduce operation resulting in key-value pairs (of type `JavaPairDStream`). Note that these were only configuration steps. The actual stream processing starts when the `start()` method is invoked on the `ssc` streaming context.

```
...
// Create the context with a 1 second batch size
SparkConf sparkConf = new SparkConf().setAppName("JavaNetworkWordCount");
JavaStreamingContext ssc = new JavaStreamingContext(sparkConf, Durations.seconds(1));

// Create a JavaReceiverInputDStream on target ip:port and count the
// words in input stream of \n delimited text (eg. generated by 'nc')
// Note that no duplication in storage level only for running locally.
// Replication necessary in distributed scenario for fault tolerance.
JavaReceiverInputDStream<String> lines = ssc.socketTextStream(
        args[0], Integer.parseInt(args[1]), StorageLevels.MEMORY_AND_DISK_SER);

JavaDStream<String> words = lines.flatMap(x ->
                                Arrays.asList(SPACE.split(x)).iterator());
JavaPairDStream<String, Integer> wordCounts =
           words.mapToPair(s -> new Tuple2<>(s, 1)).reduceByKey((i1, i2) -> i1 + i2);

wordCounts.print();
ssc.start();
ssc.awaitTermination();
...
```

[14] Image source: https://spark.apache.org/docs/latest/streaming-programming-guide.html

[15] Code source: https://github.com/apache/spark/blob/v3.0.1/examples/src/main/java/org/apache/spark/examples/streaming/JavaNetworkWordCount.java

8.2.2 Apache Flink

The most disturbing feature of streaming data processing is that requires new implementations for our well-known traditional batch algorithms. In other words, it is a disruptive technology. If one wants to do live data processing, new implementations (meaning program development effort) are required. This means that developers must develop and maintain two codebases, one for batch and one for stream processing which is very undesirable.

This is where Apache Flink [19] comes into play. The developers of Flink set out the goal of unifying the stream and batch programming models into one common programming model. The key concept of Flink is the data stream within which we distinguish between unbounded and bounded streams (see Figure 8-4). Unbounded streams that only have a start but no end represent live streaming data, whereas bounded streams (with well-defined start and end) are used to describe batch-processed data. Treating traditional batch input data as a bounded stream removes the need for keeping duplicate implementations. Now the same algorithm can work on both types of streams.

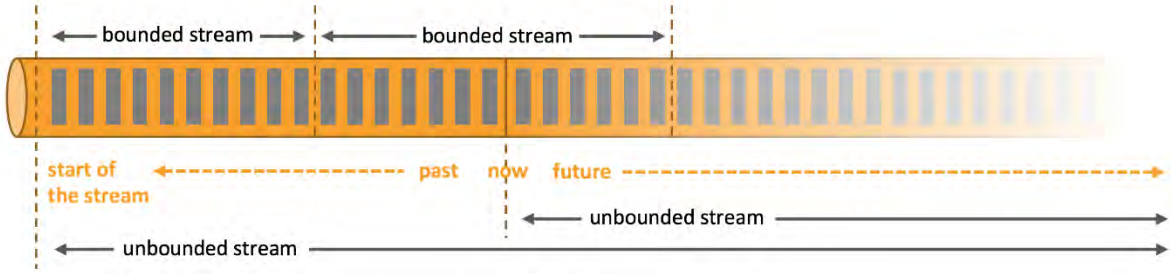

Figure 8-4 The unified treatment of bounded and unbounded streams in Flink

Flink is a feature-rich stream processing engine. As illustrated in Figure 8-5, it can receive data from a variety of sources, can interact with different execution and storage systems and forward results to a number of different data consumers (file system, databases, applications, event queues).

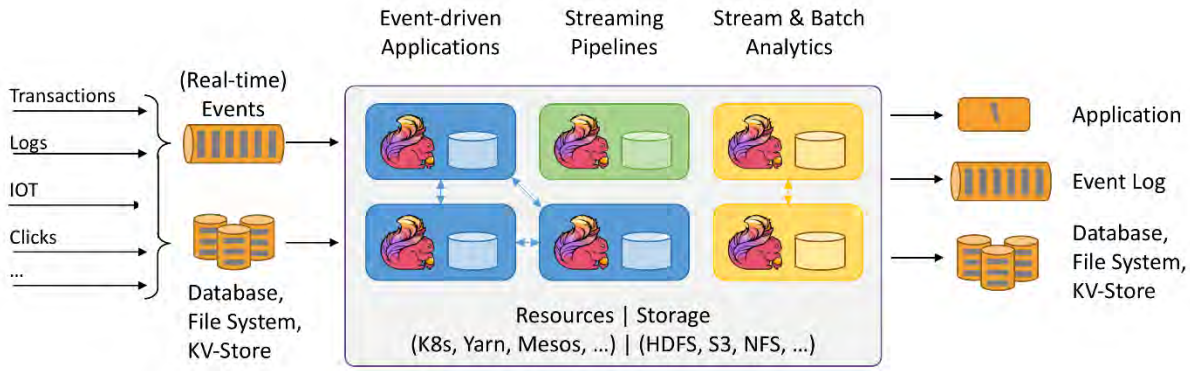

Figure 8-5 Rich integration features provide fast development in Flink[16]

Handling state

One of the key problems of large distributed systems is state. In the presence of failures and errors, it is very difficult to maintain a consistent application state. As a result, many large distributed systems favour or explicitly demand stateless design.

[16] Image source: https://flink.apache.org/flink-architecture.html

Obviously, not every application can be stateless. Most non-trivial applications require state. A distinctive feature of Flink is the support for stateful operations. State is a first-class citizen in Flink and the implementation is very efficient as it is based on in-memory state representation (see Figure 8-6). Flink provides state primitives for different data structures, such as atomic values, lists, or maps. Developers can choose the state primitive that is most efficient based on the access pattern of the function. Application state is managed in and checkpointed by a pluggable state backend. Due to its efficient implementation, Flink can manage application states of size in the order terabytes.

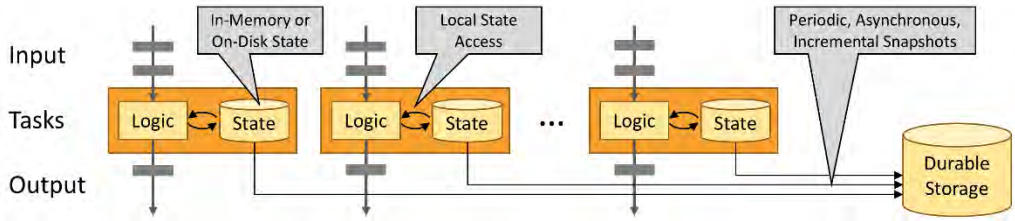

Figure 8-6 Stateful processing supported by in-memory or on-disk state persistence

Time is another important feature of streaming applications. In real-life scenarios, events have causality; one event cannot happen before the other, e.g. we cannot hear the sound of a gun fire before the gun has been triggered. Consequently, stream data most frequently include time stamps that can be used to guarantee event causality. As a result, stream oriented systems can distinguish processing based on *event* or *processing* time. In event-time mode, applications will process events based on their timestamp, guaranteeing consistent results independent of how long the actual processing lasts. On the other hand, in processing-time mode, applications process events based on processing time, e.g. in real-time applications with defined latency requirements. In this case, only events received up to the processing deadline will be considered, meaning that some events may be missed. Flink supports both time modes. Applications can use either event or processing time mode semantics. In event-time mode, further subtleties, e.g. watermark support and late event handling can help in achieving the required processing semantics.

Most event streams have inherent time semantics because each event is produced at a specific point in time. Moreover, many common stream computations are based on time, such as windows aggregations, sessionization, pattern detection, and time-based joins. An important aspect of stream processing is how an application measures time, i.e., the difference of event-time and processing-time. The details of these can be found in the detailed Flink documentation.

8.2.3 Apache Storm / Trident

Apache Storm [20] is an alternative take on the MapReduce/Stream processing problem. Storm approaches this from the unbound stream processing architecture point of view. Imaging an application that runs continuously until it is stopped. The incoming data is processed continuously by a generalised producer-consumer system as a dataflow engine. The goal of Storm is to create an abstraction for this engine and simplify the development of applications fitting this execution strategy.

Creating a Producer-Consumer implementation in Java is relatively simple. However, to create one that has multiple data paths in it, fault-tolerant and scalable (new nodes can be added and remove at will without the need to restart the application) is an extremely complicated task.

The underlying concept of Storm is the *Topology* that represents the above generalised producer-consumer dataflow graph. The topology processes incoming data streams, unbounded data that is continuously arriving at the system. Sources of data streams at the edge of the topology are

represented by special objects called *Spout*s. Data sources can be APIs, queues, etc. that produce data to be operated on. The nodes of the topology are *Bolt*s that represent a processing step. A Bolt can consume streams, apply an operation to them, and output the result as a stream. As shown in Figure 8-7, Bolts are connected to each of the spouts, and then connect to each other to arrange all of the necessary processing. The connecting edges are communication links on which stream data elements (tuples) are transferred. At the end of the topology, a final bolt output may be used as an input for a connected system.

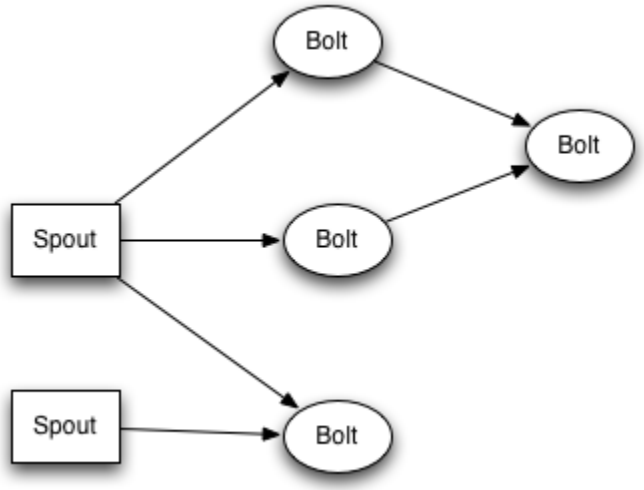

Figure 8-7 A Storm topology consisting of Spouts and Bolts.

A Storm topology is executed on top of a distributed cluster architecture. As show in Figure 8-8, the entry point to the cluster is the Nimbus component. This works similar to the submission point of Hadoop cluster. Zookeeper plays the role of the cluster manager scheduling and monitoring Worker nodes. Workers run with a local Supervisor acting as a local resource manager.

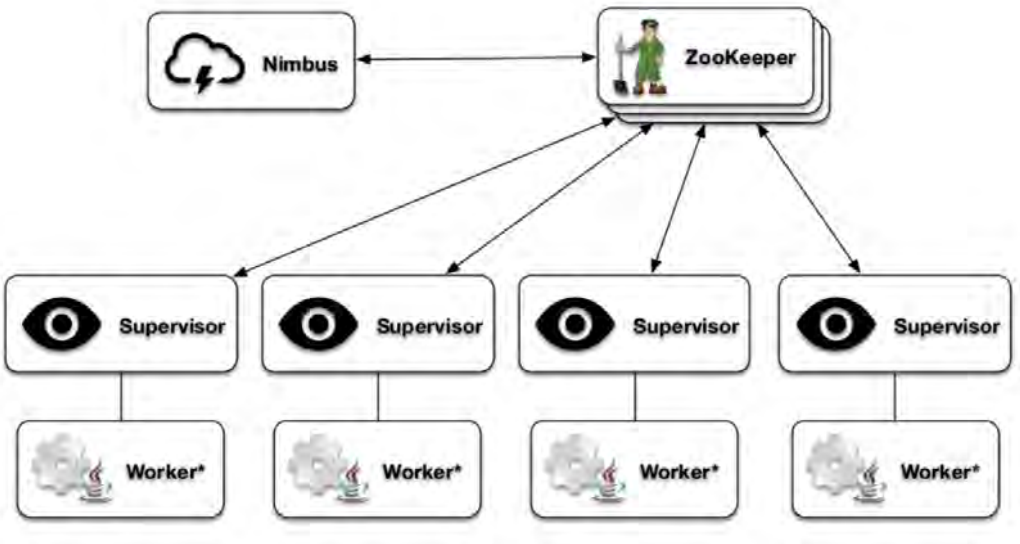

Figure 8-8 The Storm cluster architecture

To illustrate the programming model of Storm, we will look at the Storm implementation of the word count problem. The high-level topology is shown in Figure 8-9. A Spout will produce sentences to a

Bolt that splits the sentence to words, followed by another Bolt counting individual words and producing the word count result.

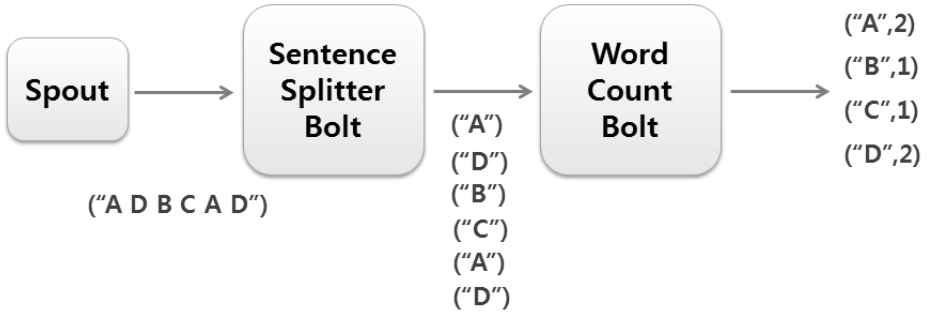

Figure 8-9 Processing topology for the Storm Word Count example.

We start the example with the implementation of the sentence Spout. The key method here is `nextTuple()` that will emit a randomly selected sentence to the topology every 100 milliseconds.

```
public class RandomSentenceSpout extends BaseRichSpout {
  SpoutOutputCollector _collector;
  Random _rand;


  @Override
  public void open(Map conf, TopologyContext context,
                SpoutOutputCollector collector) {
    _collector = collector;
    _rand = new Random();
  }

  @Override
  public void nextTuple() {
    Utils.sleep(100);
    String[] sentences = new String[]{ "the cow jumped over the moon",
         "an apple a day keeps the doctor away",
         "four score and seven years ago", "snow white and the seven dwarfs",
         "i am at two with nature" };
    String sentence = sentences[_rand.nextInt(sentences.length)];
    _collector.emit(new Values(sentence));
  }

  @Override
  public void ack(Object id) {
  }

  @Override
  public void fail(Object id) {
  }

  @Override
  public void declareOutputFields(OutputFieldsDeclarer declarer) {
    declarer.declare(new Fields("word"));
  }

}
```

The next class implements the first Bolt that splits sentences to words. In the Bolt class, the key method to watch is `execute()`. This methods retrieves the content of the incoming Tuple, splits the sentence to words, then emits each word to the topology in a loop iterating over the words of the sentence.

```
public static class SplitSentence extends BaseBasicBolt {
    @Override
    public void declareOutputFields(OutputFieldsDeclarer declarer) {
      declarer.declare(new Fields("word"));
    }

    @Override
    public Map<String, Object> getComponentConfiguration() {
      return null;
    }

    public void execute(Tuple tuple, BasicOutputCollector basicOutputCollector) {
      String sentence = tuple.getStringByField("sentence");
      String words[] = sentence.split(" ");
      for (String w : words) {
        basicOutputCollector.emit(new Values(w));
      }
    }
}
```

The next Bolt is the word counting process. The method execute will start similar to the previous one, retrieving the content of the incoming Tuple. The class holds a map the stored <word,count> key-value pairs. Note that each incoming word will trigger an output result tuple. The internal map is updated continuously as tuples are processed, so the emitted counts for each word are adjusted as the program proceeds. This is the key distinction between batch and stream processing; in stream mode, several intermediate word count values will be produced, reflecting the actual state the processing and partial view of the input data.

```
public static class WordCount extends BaseBasicBolt {
    Map<String, Integer> counts = new HashMap<String, Integer>();

    @Override
    public void execute(Tuple tuple, BasicOutputCollector collector) {
      String word = tuple.getString(0);
      Integer count = counts.get(word);
      if (count == null)
        count = 0;
      count++;
      counts.put(word, count);
      collector.emit(new Values(word, count));
    }

    @Override
    public void declareOutputFields(OutputFieldsDeclarer declarer) {
      declarer.declare(new Fields("word", "count"));
    }
}
```

The final step of the example is creating the topology for the program. A TopologyBuilder class is used for this purpose. In a compositional fashion, the spouts and bolts are set explicitly. Once the topology is created, either an external cluster or a local cluster is used to start the processing sequence /`submitTopology`/.

```
public static void main(String[] args) throws Exception {
    TopologyBuilder builder = new TopologyBuilder();
    builder.setSpout("spout", new RandomSentenceSpout(), 5);
    builder.setBolt("split", new SplitSentence(), 8).shuffleGrouping("spout");
    builder.setBolt("count", new WordCount(), 12).fieldsGrouping("split",
                new Fields("word"));

    Config conf = new Config();
    conf.setDebug(true);

    if (args != null && args.length > 0) {
      conf.setNumWorkers(3);

      StormSubmitter.submitTopologyWithProgressBar(args[0], conf,
                         builder.createTopology());
    } else {
      conf.setMaxTaskParallelism(3);
      LocalCluster cluster = new LocalCluster();
      cluster.submitTopology("word-count", conf, builder.createTopology());
      Thread.sleep(10000);
      cluster.shutdown();
    }
}
```

8.2.4 Apache Kafka

Apache Kafka [20] is a high-performance message broker architecture designed to build real-time streaming data pipelines. It is commonly used as data delivery system in various streaming applications. Although Kafka can be used for stream processing in itself, its main use is typically is streaming data delivery only.

Kafka streams consists of records. Records belong to categories that in Kafka are called *topic*s. Each record consists of a key, a value, and a timestamp.

In contrast to traditional point-to-point message delivery systems, Kafka is a publish-subscribe architecture. It uses multiple message queues internally for topics and subscribers can subscribe for messages arriving to a given topic. The system guarantees that all subscribers will receive the event of interest. Kafka is a reliable, fault-tolerant system. It uses a distributed cluster with automatic partitioning and replication guaranteeing zero message loss even in presence of hardware failures. It is also very fast, and integrates nicely with the Apache Spark and Storm systems.

The high-level view of Kafka and its place in a streaming application is illustrated in Figure 8-10. Multiple producers can feed records into the Kafka system that are delivered to multiple consumers. In addition, Kafka can be connected to databased and stream processors to perform data processing.

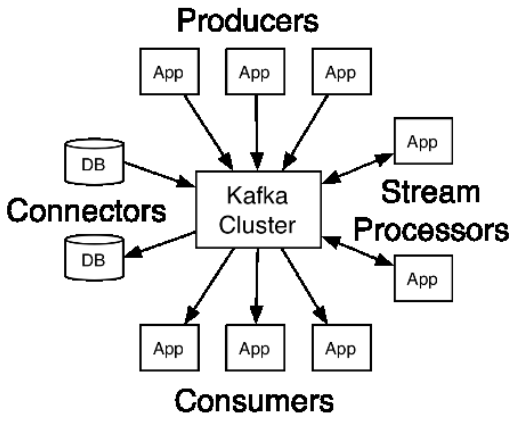

Figure 8-10 The place of Kafka within a stream processing application.

A Kafka cluster consists of several Kafka Brokers. A Broker is basically a Kafka cluster node. Brokers manage producers and consumers, store messages and provide fault tolerance by replication. To support load balancing and reliability, each topic is stored in a broker in form of several message partitions. A partition is a queue in which the system collects messages. A partition is show in Figure 8-11, having one producer writing to the tail of the queue and multiple consumers reading messages. Message positions are marked by offset values.

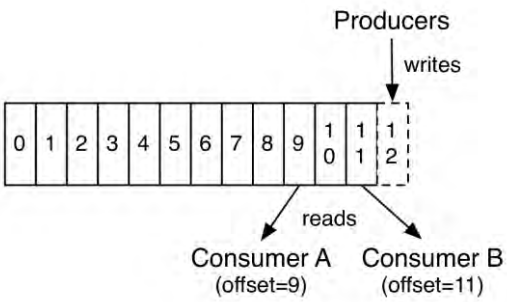

Figure 8-11 Implementation of a topic partition as a queue
with interacting producers and consumers

Figure 8-12 shows the internals of a Kafka cluster consisting of two brokers. The cluster has only one topic, Topic A. The topic is split into two partitions, partition 0 and 1. In a cluster, one node is a leader while the others are the followers. The role of the followers is to replicate the data stored by the leader and in case of a leader failure, to take over the leader role. In this particular case, Broker 1 is the leader for Topic A Partition 0 (A/0 in short), which means that Broker 2 will receive a replica of A/0. Broker 2, however, is the leader for Topic A Partition 1 (A/1) so Broker 1 will receive a replica of A/1. Leader election and partition replication is performed automatically behind the scenes. When producers submit messages to the cluster, the system will automatically route the message to the corresponding leader (see Messages for A/0 and A/1); similarly, consumers will receive messages from the leader of the partition the message originates from. We do not need to understand the more intricate details of message persistence, acknowledgements, replications, etc. This was describe only to make us appreciate the complexity that lies in implementing a fault-tolerant, reliable and scalable messaging system.

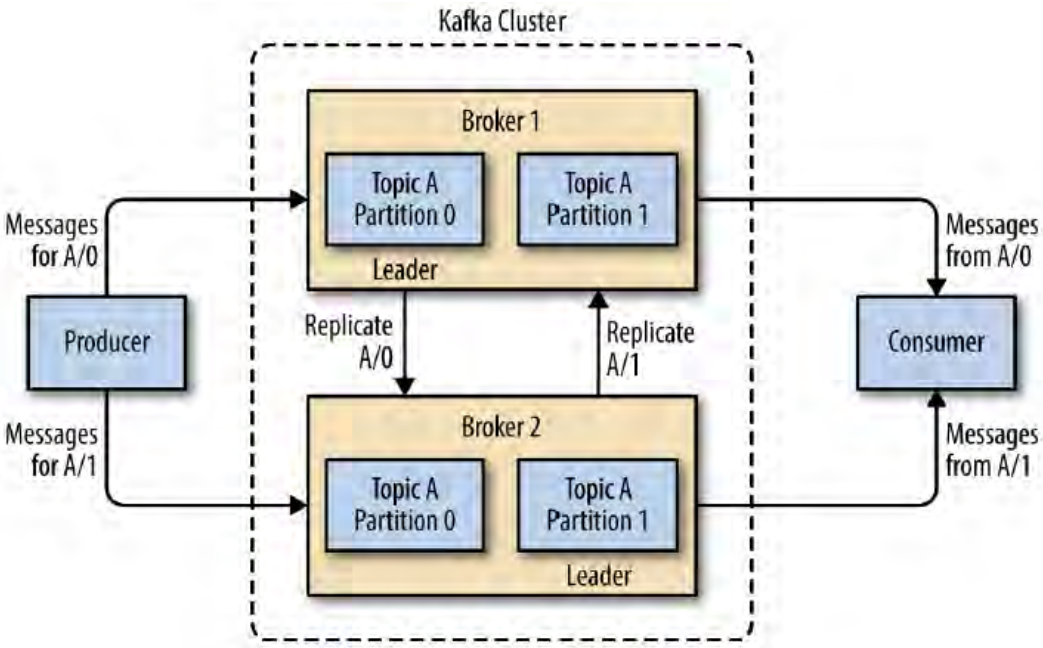

Figure 8-12 Internal details of topic fault-tolerance based on partition replication in a cluster

Kafka Streaming

As stated at the beginning of this subsection, Kafka initially was a message delivery system. Since version 0.10.0, however, it includes a stream processing API as well, which makes it a stream data processing system on its own, so Kafka users might not need additional stream processing frameworks to implement their applications. Since the latest version of Kafka at the time of writing is 2.7, it is evident that stream processing support is not a long-term, mature feature in Kafka.

In the followings, we will briefly overview Kafka's stream processing capabilities. Stream programming in abstracted out in the Kafka Stream API. The main stages of creating a stream applications are (i) application configuration, (ii) defining the processing steps of the application, and (iii) creating the topology of the execution graph and starting the application.

Each application must have an application ID, server parameters and the key-value serialisation class definitions. After setting these as program properties, the computation logic is created either by the `Topology` class in which `Processors` can be connected or by the `StreamBuilder` class that provides a high-level domain-specific language (DSL) for specifying the code for the processing stages. In the following example, The `StreamBuilder` class is used. In this example, the length of the String received from the topic `"my-input-topic"` is sent to the topic `"my-output-topic"`. As the last step, the output of the `StreamBuilder` is passed to the `KafkaStreams` object and the program can be started.

```
Properties props = new Properties();

props.put(StreamsConfig.APPLICATION_ID_CONFIG,
        "my-stream-processing-application");
props.put(StreamsConfig.BOOTSTRAP_SERVERS_CONFIG, "localhost:9092");
props.put(StreamsConfig.DEFAULT_KEY_SERDE_CLASS_CONFIG,
        Serdes.String().getClass());
props.put(StreamsConfig.DEFAULT_VALUE_SERDE_CLASS_CONFIG,
        Serdes.String().getClass());
```

```
StreamsBuilder builder = new StreamsBuilder();
builder.<String, String>stream("my-input-topic")
        .mapValues(value -> String.valueOf(value.length()))
        .to("my-output-topic");

KafkaStreams streams = new KafkaStreams(builder.build(), props);
streams.start();
```

After this toy example we return to our word count example and show its Kafka stream implementation. The property configuration steps are ignored in this case. The input sentence is split into words as before, but now an extra step is inserted; the word "the" will be filtered out from the sequence and the number of occurrences of this word will be counted and output as the result.

```
KStreamBuilder builder = new KStreamBuilder();

KStream<String, String> source = builder.stream("wordcount-input");

final Pattern pattern = Pattern.compile("\\W+");

KStream counts = source.flatMapValues(value->
       Arrays.asList(pattern.split(value.toLowerCase())))
              .map((key, value) -> new KeyValue<Object, Object>(value, value))
              .filter((key, value) -> (!value.equals("the")))
              .groupByKey()
              .count("CountStore")
              .mapValues(value-> Long.toString(value))
              .toStream();
counts.to("wordcount-output");
```

Once the stream is built, we pass it to the KafkaStreams instance and start the program. Since normally the program would run indefinitely, we stop it after 5 seconds.

```
KafkaStreams streams = new KafkaStreams(builder, props);

streams.start();

// usually the stream application would be running forever, in this example we
// just let it run for some time and stop since the input data is finite.
Thread.sleep(5000L);

streams.close();
```

8.2.5 Apache Beam/GCP Dataflow

The last streaming framework we discuss is Apache Beam [21], which provides a unified programming abstraction for batched and streaming data processing. It is an open-source framework, just as the previous Apache systems but its importance lies in the fact that it is the programming system used by Google Dataflow, the GCP stream processing engine service. That is, if you want to run large streaming application on the Google Cloud Platform, Beam is the natural choice as it is tightly integrated with Dataflow as its back-end. Additional benefits of using Dataflow come from the fact that it is nicely integrated with other Google Services, as shown in Figure 8-13, making it a very powerful application development and execution platform.

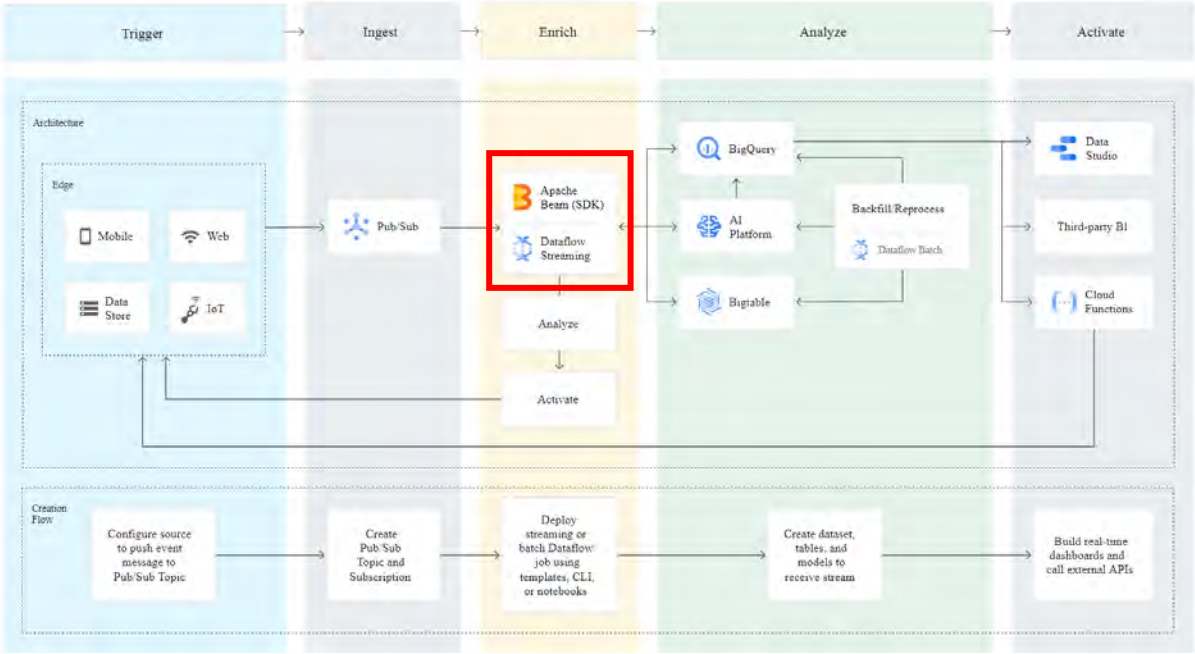

Figure 8-13 Integration of Google Dataflow and the Beam SDK in the GCP service architecture.

We will start with an overview the Beam system and its programming model. Beam uses the concept of the **pipeline** to describe the program. The pipeline represents a program from its start to finish, responsible for data input, processing and data output. Once a pipeline is created, it can be executed on a back-end which is called a *runner*. The Beam pipeline is presented in a Java program as an instance of the class `Pipeline`. The next item is the class `PCollection`, which represents a distributed data set used in the execution of the Beam program. A `PCollection` data can be *bounded* or *unbounded*, depending on the input used. The third item in the model is the class `PTransform` that represents one step of operation executed in the program. Transforms can change, filter, group, analyse, or otherwise process the elements in a `PCollection`. A transform creates a new output `PCollection` *without modifying the input collection*. A transform can have one or more `PCollections` as its input and can produce zero or more `PCollection` objects as its output. A simple sample pipeline is shown in Figure 8-14 consisting of one input data source, three transforms.

Figure 8-14 A simple Beam pipeline using three transforms

This pipeline can be described in the Beam Java programming model by the following code skeleton.

```
[Final Output PCollection] =
        [Initial Input PCollection]
                .apply([First Transform])
                .apply([Second Transform])
                .apply([Third Transform]);
```

The initial PCollection object produced by the database read operation will be used to execute the "first transform". This returns an intermediate PCollection – the output of the first transform – that can be used to invoke the second transform. The next returned object is used for executing the third transform, which results in the last PCollection that can be persisted to database. Reading and writing data to and from the pipeline requires IO transforms. Beam provides a variety of specialised IO transforms that perform input/output operations with the external world. Note the `apply()` method of the `PCollection` class; this method is responsible for executing the transform at the specified position of the pipeline.

Several core transform are already implemented in Beam, such as `ParDo`, `GroupByKey`, `CoGroupByKey`, `Combine`, `Flatten` and `Partition`. ParDo transforms each element of a PCollection in parallel; it can be used for transforming elements, outputting them to a new PCollection or performing calculation on the elements. The description of the other transforms can be found in the Beam documentation.

Once the pipeline is constructed, a driver program is required that executes the pipeline on the selected backend runner. The role of the driver program is to configure the backend runner, create the pipeline and run the program.

Sample WordCount program

We finish this section with a Beam implementation of the Word Count example. The program creates a pipeline that will execute a series of transformations on the data:

Step 1: A root transform, the TextIO.Read will read a set of input text files. TextIO.Read returns a PCollection where each element is one line from the input text (a set of Shakespeare's texts).

Step 2: A FlatMapElements transform is used to split the lines in PCollection<String>, where each element is an individual word in Shakespeare's collected texts.

Step 3: A Count transform is applied to the PCollection of individual words. The Count transform returns a new PCollection of key/value pairs, where each key represents a unique word in the text. The associated value is the occurrence count for that word.

Step 4: A write transform, TextIO.Write, is applied at the end of the pipeline to write the contents of a PCollection to a series of text files.

```
public class MinimalWordCount {

  public static void main(String[] args) {

    // Create a PipelineOptions object.
    PipelineOptions options = PipelineOptionsFactory.create();

    ...configuration skipped...
    // Create the Pipeline object with the options we defined above
    Pipeline p = Pipeline.create(options);

    p.apply(TextIO.read().from("gs://apache-beam-samples/shakespeare/*"))
        .apply(FlatMapElements.into(TypeDescriptors.strings())
                .via((String line) -> Arrays.asList(line.split("[^\\p{L}]+"))))
```

```
        // We use a Filter transform to avoid empty word
        .apply(Filter.by((String word) -> !word.isEmpty()))
        .apply(Count.perElement())
        // Apply a MapElements transform that formats our PCollection of word counts into a
        // printable string, suitable for writing to an output file.
        .apply(
            MapElements.into(TypeDescriptors.strings())
                .via( (KV<String, Long> wordCount) ->
                        wordCount.getKey() + ": " + wordCount.getValue()))

        .apply(TextIO.write().to("wordcounts"));

    p.run().waitUntilFinish();
  }
}
```

The program can be executed on GCP Dataflow using the following command. Note the `--runner` option `DataflowRunner` that specifies Google Dataflow as the backend as well as the Google Storage bucket name templates that must be changed in a real execution setting.

```
$ mvn compile exec:java -Dexec.mainClass=org.apache.beam.examples.WordCount \
     -Dexec.args="--runner=DataflowRunner
                  --gcpTempLocation=gs://YOUR_GCS_BUCKET/tmp \
                   --project=YOUR_PROJECT --region=GCE_REGION \
                   --inputFile=gs://apache-beam-samples/shakespeare/*
                  --output=gs://YOUR_GCS_BUCKET/counts" \
    -Pdataflow-runner
```

8.3 Comparison of stream frameworks

In the previous section, we overviewed quite a number of data stream processing frameworks without the goal of completeness. We have excluded several other frameworks for reasons of brevity. The question then naturally arises, if there are so many streaming frameworks, how can we choose which on is the best or which one we should choose to solve a particular problem. Is there a way to compare them objectively?

In this section, we provide guidelines to how these frameworks should be compared, what features or characteristics should be examined when selecting a system as a basis for implementations.

Since the sole reason of using software frameworks is to reduce the development time by relying on existing implementations, the first set of categories we explore is related to the framework as a tool for developers, such as the *complexity*, *maintainability*, *friendliness* and *maturity* of the framework.

Framework complexity: We need to examine how difficult it is the set up and monitor the system, what level of support it provides for these tasks; how complex is the set of dependencies; are there version limitations, backward compatibility issues, etc.

Framework maintainability: Here we are interested in whether the framework is vendor specific or cloud provider agnostic; it can be used with arbitrary container technologies, if required; and whether it has sufficient and high quality system documentation.

Framework friendliness: It is a key property of a framework how easy it is to understand the model of operation and the programming abstraction. In addition, how many parameters are to be tuned to get the system working in an optimal way. The richness and cleanness of the API is also of high

importance. Finally, developers like frameworks that can be used from/with several programming languages.

Framework maturity: Since choosing a framework is a major decision determining future developments for a long time, it is very important that a system has strong support. Since we have only discussed open-source frameworks, this typically means community support. Hence, the size and vibrancy of the user community are very important indicators. The number of contributors, commit activity, time to fix bugs and frequency of releases, and the number of companies adopting the framework are also good indicators of a successful system.

Moving on to more technical issues, we can compare properties such as the *type of streaming* supported in the system, *message delivery guarantees*, *latency* and *throughput*, *resource consumption*, *programming model*. Several publications can be found comparing streaming frameworks in great depth [22]–[26], here we only highlight the most important characteristics.

Type of streaming can be native or microbatch based. Native streaming means processing every received individual data record. Microbatch, on the other hand collects several records into a small batch that is then processed in one operation. Native streaming naturally provides lower latencies but can have a negative effect on throughput in the presence of message checkpointing. Microbatch operation comes with higher latencies but can give improved throughput due to lower message handling overheads.

One of the most important properties of any streaming system is reliability and fault-tolerance. This is normally expressed in the form of *message delivery guaranties*. There are three message delivery semantics in any distributed system: *At-Most-Once*, *At-Least-Once* and *Exactly Once* semantics. At-Most Once semantics means that any message is attempted to be deliver only once. If message delivery fails for any reason, there will be no attempts for repeat sending of that message. In short, in such a system, messages can be lost. At-Least-Once semantics provides guarantees that any message will be received by the consumer but some messages may be received several times due to repeat send operation on error conditions. In these systems, the application must keep track of the messages and handle the unnecessary duplicate messages. The Exactly Once semantics is the strongest form of guarantee; this ensures that every message is delivered to the consumer and only once. The application does not have to worry about duplicate messages. Normally, message delivery semantics is coupled with message order guarantees. With At-Most-Once and At-Least-Once semantics, the order of messages in delivery may not be guaranteed to follow sending order. Exactly Once semantics provides exact message ordering as well.

A further technical issue is the type of programming abstraction used for expressing stream processing in the framework. One programming style is *compositional*, where programmers must build the processing topology by hand from building blocks (data sources, operators, transforms, data sinks, etc.) and components are defined by interfaces. The other style is *declarative*, where the processing pipeline is created automatically behind the scene, the programmer only has to use/provide operators as functions. This mode results in a functional programming style and can rely on lambda expressions heavily.

Since a full in-depth comparison of all known streaming frameworks is beyond the scope of this book, we finish this section with a small illustrative example that highlights how these systems can be compared. The feature table is shown in Figure 8-15. The Beam system is not part of this comparison. A technical comparison of Beam with respect to other frameworks can be found at the following link: https://beam.apache.org/documentation/runners/capability-matrix

		TRIDENT	Spark Streaming	samza	
Streaming Model	Native	Micro-batching	Micro-batching	Native	Native
API	Compositional		Declarative	Compositional	Declarative
Guarantees	At-least-once	Exactly-once	Exactly-once	At-least-once	Exactly-once
Fault Tolerance	Record ACKs		RDD based Checkpointing	Log-based	Checkpointing
State Management	Not build-in	Dedicated Operators	Dedicated DStream	Stateful Operators	Stateful Operators
Latency	Very Low	Medium	Medium	Low	Low
Throughput	Low	Medium	High	High	High
Maturity	High		High	Medium	Low

Figure 8-15 Comparison of few illustrative streaming frameworks. Systems from left to right: Storm, Storm Trident, Spark Streaming, Samza and Flink.

9 Cloud Functions

Cloud Functions represent the latest development in cloud execution technology. The key feature of Functions is that they allow users to submit code directly to the cloud without creating a runtime infrastructure. Consequently, Functions are called as serverless code execution and now described by cloud vendors are a new type of service, Function-as-a-Service (FaaS).

Functions execute without the worry of server management (provision, manage or upgrade); the cloud provider takes over this responsibility. Functions can automatically scale with the load and provide integrated monitoring, logging and debugging capabilities. The support role-based and per-function level security settings as well as networking capabilities to access other cloud components from the function.

Functions are one of the "coolest" thing in modern cloud technology. They implement in a unifying manner the once utopian visions of past computing systems: seamless execution over geographically distributed resources (i.e. Grid computing) using a set of public/private services (i.e. Service-Oriented Architectures).

In this section, we will focus on Google Cloud Functions [27] only. Similar systems can be found in the offerings of all cloud vendors, e.g. Lambda [28] at AWS. From this point on, the word Function will refer to Google Cloud Functions.

There are two types of Functions we can develop for GCP: HTTP Functions and Background Functions. HTTP Functions can be developed in Node.js, Python, Go, Java or C#, while Background Function can use Node.js, Python, Go or Java. The Cloud Functions environment will automatically select the suitable runtime environment based on the chosen implementation language.

HTTP Function

HTTP Functions are designed to be invoked from standard HTTP requests (GET, PUT, POST, DELETE and OPTIONS). The request will wait until the servicing is complete and the response received. In Java, a custom Function must implement the `com.google.cloud.functions.HttpFunction` interface, which is a functional interface with a single `service()` method. Below is a simple example demonstrating request and response processing in a Java Function implementation. The code and the logic is very similar to the one used for Servlets in the Java Enterprise Platform. More details of programming HTTP Functions can be found at the following link:

https://cloud.google.com/functions/docs/writing/http

Sample HTTP Function in Java

```
public class HelloHttp implements HttpFunction {
  private static final Logger logger = Logger.getLogger(HelloHttp.class.getName());

  private static final Gson gson = new Gson();

  @Override
  public void service(HttpRequest request, HttpResponse response)
      throws IOException {
    // Check URL parameters for "name" field
    // "world" is the default value
    String name = request.getFirstQueryParameter("name").orElse("world");
```

```
    // Parse JSON request and check for "name" field
    try {
      JsonElement requestParsed = gson.fromJson(request.getReader(), JsonElement.class);
      JsonObject requestJson = null;

      if (requestParsed != null && requestParsed.isJsonObject()) {
        requestJson = requestParsed.getAsJsonObject();
      }

      if (requestJson != null && requestJson.has("name")) {
        name = requestJson.get("name").getAsString();
      }
    } catch (JsonParseException e) {
      logger.severe("Error parsing JSON: " + e.getMessage());
    }

    var writer = new PrintWriter(response.getWriter());
    writer.printf("Hello %s!", name);
  }
}
```

Purely for comparison purposes, we show the same Function implemented in Node.js. The compactness of the code is remarkable when compared to the Java implementation.

```
const escapeHtml = require('escape-html');

/**
 * HTTP Cloud Function.
 *
 * @param {Object} req Cloud Function request context.
 *                     More info: https://expressjs.com/en/api.html#req
 * @param {Object} res Cloud Function response context.
 *                     More info: https://expressjs.com/en/api.html#res
 */
exports.helloHttp = (req, res) => {
  res.send(`Hello ${escapeHtml(req.query.name || req.body.name || 'World')}!`);
};
```

Background Function

The other type of Functions is the Background Function, which are designed to handle events originating from the Cloud infrastructure representing e.g. a change in a Cloud Storage bucket or a message arriving to a Cloud Pub/Sub topic. Java Background Functions must implement the `com.google.cloud.functions.BackgroundFunction` interface. Instead of `service()`, this interface has a single `accept()` method which will be invoked upon an event of interest triggered.

In the following example we show a Background Function that is activated whenever some change occurs in a Cloud Storage bucket. Note the generic type parameter in the interface that determines to which types of events this implementation will be invoked. The Function extracts the bucket and file information and the type of event that happened.

```
import com.google.cloud.functions.BackgroundFunction;
import com.google.cloud.functions.Context;
import functions.eventpojos.GcsEvent;
import java.util.logging.Logger;

public class HelloGcs implements BackgroundFunction<GcsEvent> {
  private static final Logger logger = Logger.getLogger(HelloGcs.class.getName());
```

```
  @Override
  public void accept(GcsEvent event, Context context) {
    logger.info("Event: " + context.eventId());
    logger.info("Event Type: " + context.eventType());
    logger.info("Bucket: " + event.getBucket());
    logger.info("File: " + event.getName());
    logger.info("Metageneration: " + event.getMetageneration());
    logger.info("Created: " + event.getTimeCreated());
    logger.info("Updated: " + event.getUpdated());
  }
}
```

More details of writing Background Functions can be found at the following link: https://cloud.google.com/functions/docs/writing/background

The generic steps of creating Functions are the followings:

1. Write functions
2. Deploy functions
3. Execute/Call functions
4. Monitor functions

Function Execution

GCP Functions are always executed in an isolated, secure runtime context. An important constraint for Functions is that they must be *stateless*. If state is required for the correct operation, it must be persisted to Google Storage or Datastore. Each Function instance will service one client/ request, multiple instances are launched if necessary.

GCP provides the following execution guarantees for Functions: HTTP functions are invoked with *At-Most Once*, whicle Background functions are invoked *At-Least-Once* semantics.

Cloud functions are executed by triggers. Type of triggers can be: HTTP Triggers, Cloud Pub/Sub Triggers, Cloud Storage Triggers, Direct Triggers, Cloud Firestore Triggers, Analytics for Firebase Triggers, Firebase Realtime Database Triggers and Firebase Authentication Triggers. We will look at the cloud Pub/Sub mechanism in the next chapter to learn the details of general event/message delivery mechanisms.

Deploying functions

Before a Function can be executed, it must be deployed to the cloud. Deployment involves uploading an archive file containing the function's source code to a Google Cloud Storage bucket. The following options are available for deploying a function.

1. Deploying from your Local Machine
   Execute the following gcloud command after substituting the necessary parameters.

```
gcloud functions deploy NAME --runtime RUNTIME TRIGGER [FLAGS...]
```

2. Deploying from Source Control
   Execute the following gcloud command after substituting the necessary parameters.

```
gcloud functions deploy NAME --source https://source.developers.google.com/
      projects/PROJECT_ID/repos/REPOSITORY_ID/moveable-aliases/master/
      paths/SOURCE
      --runtime RUNTIME TRIGGER...  [FLAGS...]
```

3. Deploying from GCP Console
   We will look at this as an example in more detail while creating an HTTP Function.
   Execute the following steps in order
   1. Open the Functions Overview page in the Cloud Console (as shown in Figure 9-1)
   2. Make sure that the project for which you enabled Cloud Functions is selected.
   3. Click **Create function**.
   4. Name your function.
   5. In the **Trigger** field, select **HTTP**.
   6. In the **Authentication** field, select **Allow unauthenticated invocations**.
   7. Click **Save** to save your changes, and then click **Next**.
   8. In the **Source code** field, select **Inline editor**. In this exercise, you will use the default function provided in the editor.
   9. Use the **Runtime** dropdown to select the desired Java runtime.
   10. At the bottom of the page, click **Deploy**.
   11. After clicking **Deploy**, Cloud Console redirects to the Cloud Functions Overview page.
   12. Once the Deploy is done, you can test the operation of the Function after displaying the menu for your function, and clicking **Test function** (as shown in Figure 9-2).

Figure 9-1 The Function overview window of the Google Cloud Console.

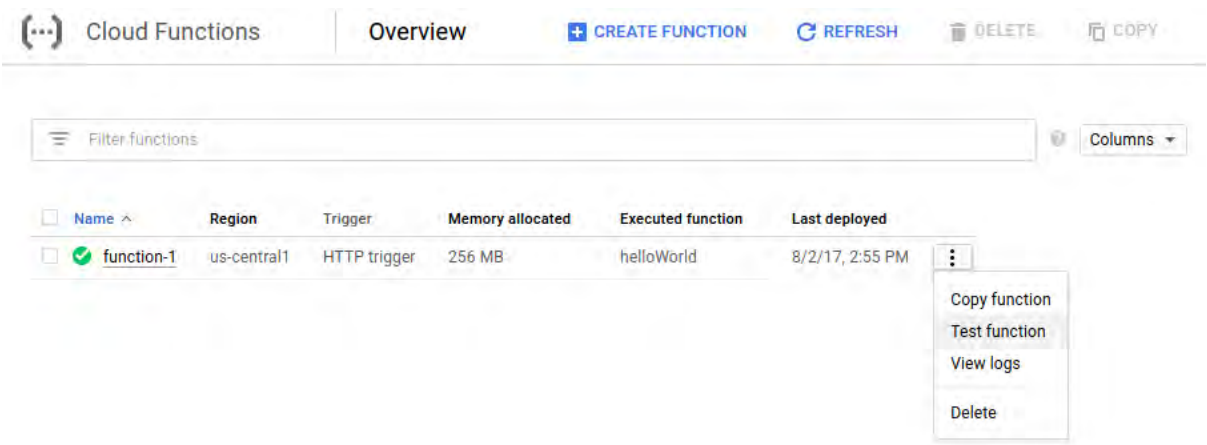

Figure 9-2 Testing the execution of a function from the Google Cloud Console

The execution of the function can be checked by looking at the automatically generated log entries. On the Cloud Functions Overview page, display the menu for your function, and click **View logs** to see a log window similar to the one displayed in Figure 9-3.

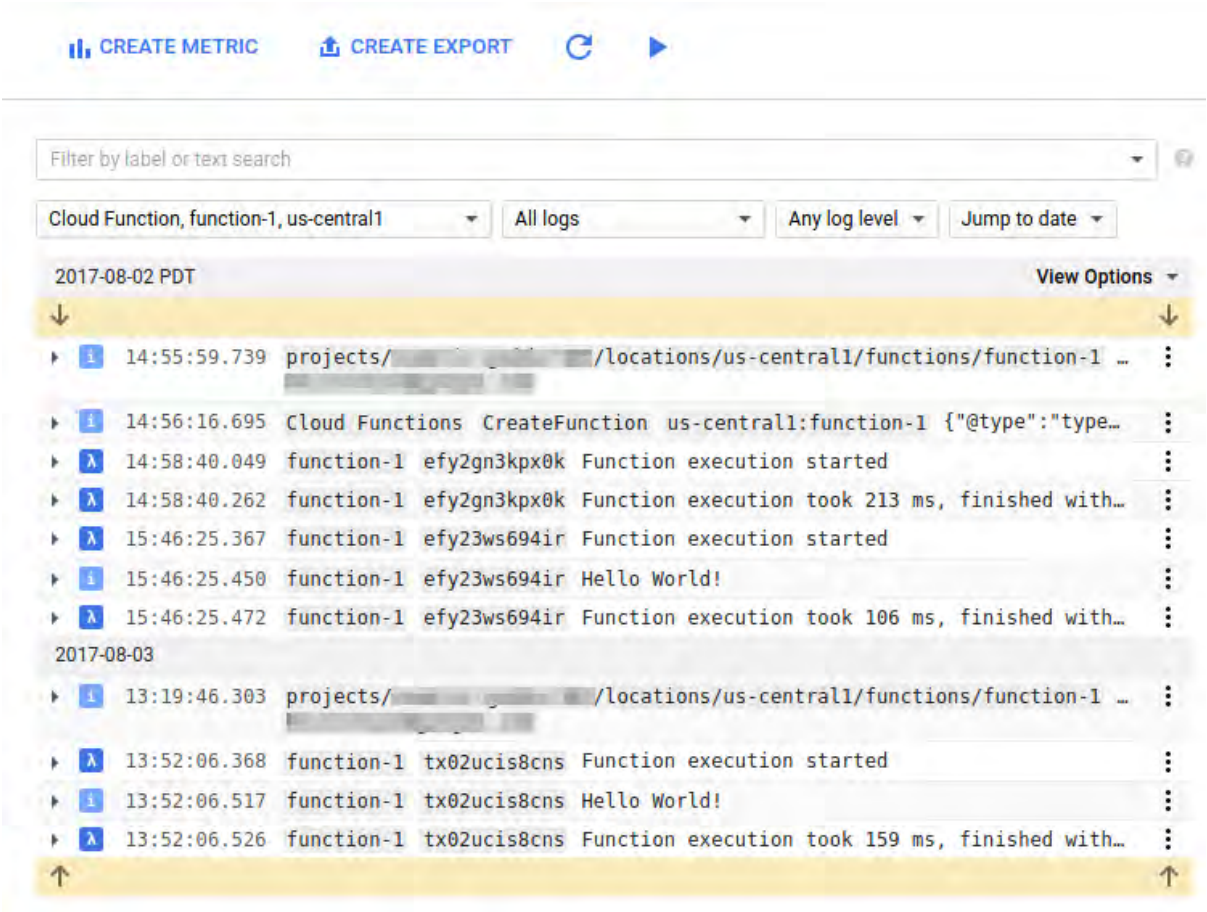

Figure 9-3 Function execution log entries in the Google Cloud Console.

A complete step-by-step example for creating and testing a Storage triggered Background Function can be found at the following link: https://cloud.google.com/functions/docs/tutorials/storage .

10 Service communication: Pub/Sub

Creating a cloud application involves using several off-the-shelf and custom cloud services that must cooperate with one another throughout the lifecycle of the application. Strong coupling via synchronous RPC-style calls is not a suitable option for large distributed applications; loosely coupled communication based on messages is the only viable strategy for large, scalable applications. Consequently, all cloud vendors provide some sort of messaging service in their product list that help plugging components together. In this section, we will focus on the messaging service provided in the Google Cloud Platform, the Google Pub/Sub service.

10.1 Google Pub/Sub overview

The Google Pub/Sub is a high-availability, asynchronous, real-time messaging service based on the publish-subscribe philosophy. Pub/Sub is a low-latency, high-throughput system that operates in all Google Cloud regions providing a universal messaging and event delivery infrastructure for global cloud applications. Pub/Sub can be used as a general messaging middleware or event delivery system triggering Cloud Functions or as a data streaming layer.

Pub/Sub connects *Publisher*s and *Subscriber*s through *Topic*s. A **Topic** is a named resource to which publishers send messages. Subscribers create **Subscription**s that are named resources representing the stream of messages from a single, specific topic, to be delivered to the subscribing application. A **Message** is a combination of data and (optional) attributes that a publisher sends to a topic and is eventually delivered to subscribers. The message attribute is a key-value pair that a publisher can define for a message.

The actual Pub/Sub Message format is as follows. A message consists of fields with the message data and metadata. At least one of the following fields must be present in the message:

- The message data
- An ordering key
- Attributes with additional metadata

The Pub/Sub service adds the following fields to the message:

- A message ID unique to the topic
- A timestamp for when the Pub/Sub service receives the message

Pub/Sub is an integral part of the Google Cloud infrastructure and as such is supported by all Google Services. This makes integration of GCP services an almost trivial task (see Figure 10-1 for a set of key services that can be used with Pub/Sub).

Pub/Sub can connect very different types of components into a unified cloud application. Figure 10-2 illustrates this in more detail. Any application that can make HTTPS requests to pubsub.googleapis.com can be a Publisher, e.g. an App Engine app, a web service hosted on Google Compute Engine or any other third-party network, or an app installed on a desktop or mobile device. Similarly, any application that can make HTTPS requests to pubsub.googleapis.com can be a Pull Subscriber.

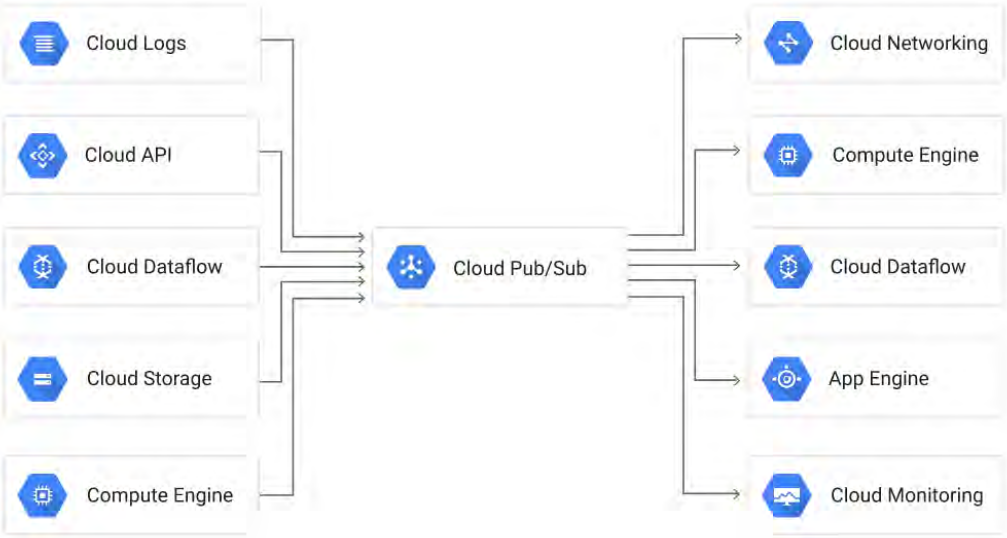

Figure 10-1 Google Pub/Sub support in Google Services.

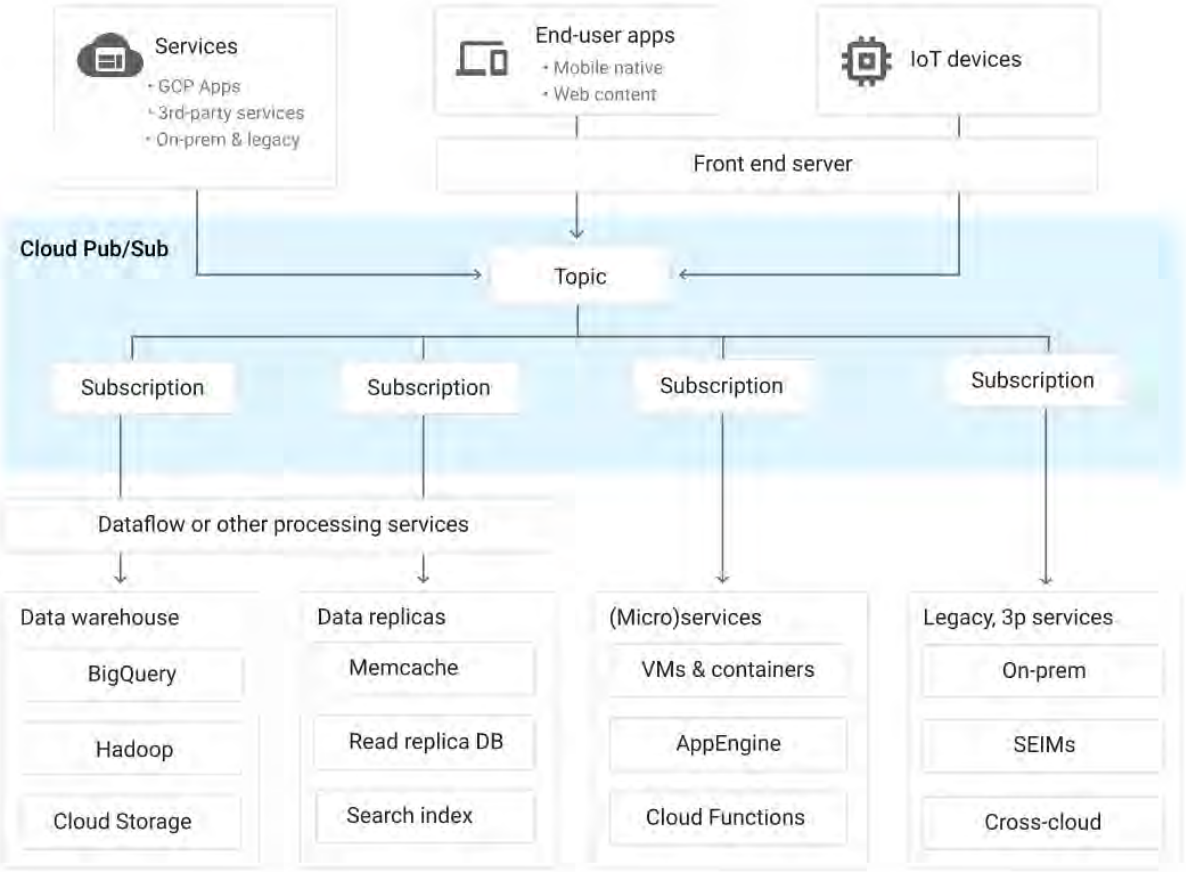

Figure 10-2 Possible service connections in Pub/Sub

Pub/Sub supports an N-to-M mapping between Publishers and Subscribers. Subscribers can subscribe to receive messages from any number of subscribers allowing overlaps. Similarly, Publishers can

deliver messages to distinct or a group of subscribers. The mapping between Publishers and Subscribes are implemented via Topic-Subscription configurations.

Figure 10-3 illustrates these concepts using an example. We have three Publishers A, B and C as well as three Subscribers X, Y and Z. Subscriber X would like to receive messages from both Publishers A and B, so subscribes to Topic A and B. The resulting Subscriptions XA and XB will guarantee that any message sent to Topic A and B will be delivered to Subscriber X.

Subscribers Y and Z, on the other hand, would like to receive messages from the same Publisher C. In this case, Y and Z subscribe for Topic C which results in two subscriptions YC and ZC. When a message arrives from Publisher C, the same message is delivered via subscription YC to Subscriber Y and via subscription ZC to Subscriber Z.

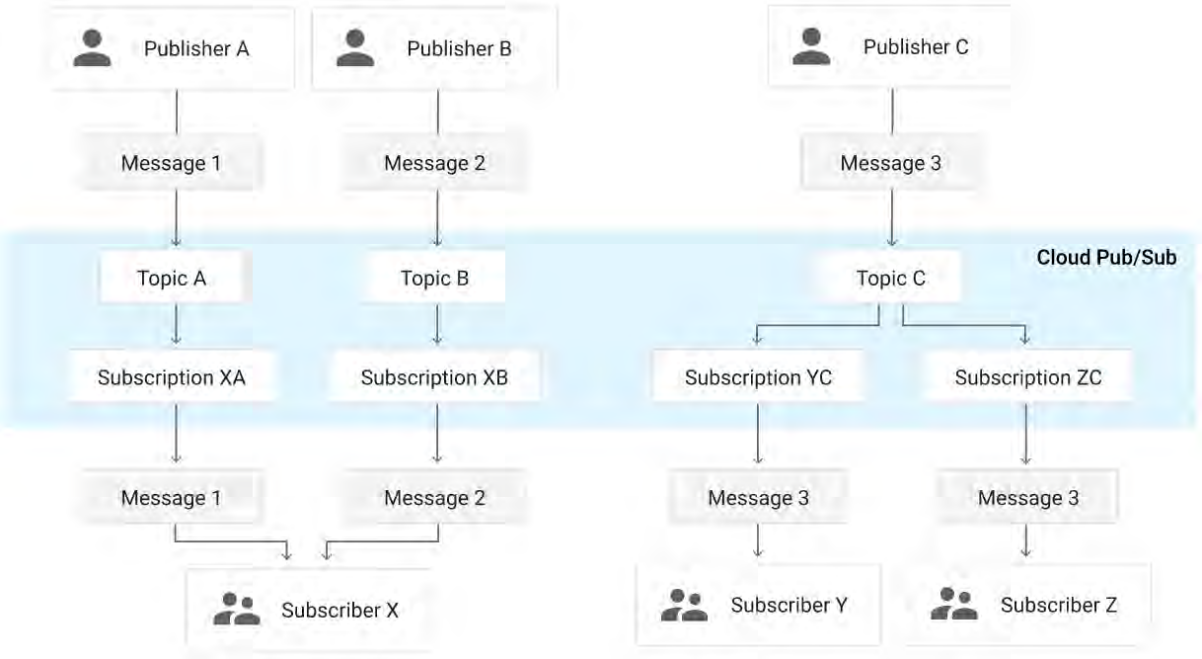

Figure 10-3 Illustrations of many-to-one and one-to-many publisher-subscriber relationships

10.2 Pub/Sub message flow

In this section, we examine the steps of message delivery in details. The delivery of message sent from a Publisher involves the following steps in the Pub/Sub infrastructure.

1. A publisher application creates a topic in the Pub/Sub service and sends messages to the topic. A message contains a payload and optional attributes that describe the payload content.
2. The service ensures that published messages are retained on behalf of subscriptions. A published message is retained for a subscription until it is acknowledged by any subscriber consuming messages from that subscription.
3. Pub/Sub forwards messages from a topic to all of its subscriptions, individually.
4. A subscriber receives messages either by Pub/Sub pushing them to the subscriber's chosen endpoint, or by the subscriber pulling them from the service.
5. The subscriber sends an acknowledgement to the Pub/Sub service for each received message.
6. The service removes acknowledged messages from the subscription's message queue.

Pub/Sub, by default, delivers messages with At-Least-Once semantics. The only exception to message delivery is if a message cannot be delivered within 7 days after its generation, in which case it is deleted after 7 days without delivery. Similarly, if a message does not have a subscription at the time of its generation, that message will not be delivered at all.

When Pub/Sub is used with the Beam SDK streaming programming model, it is possible to achieve Exactly-Once semantics with ordered message delivery.

Further details of the Pub/Sub architecture can be found at the following link: https://cloud.google.com/pubsub/architecture .

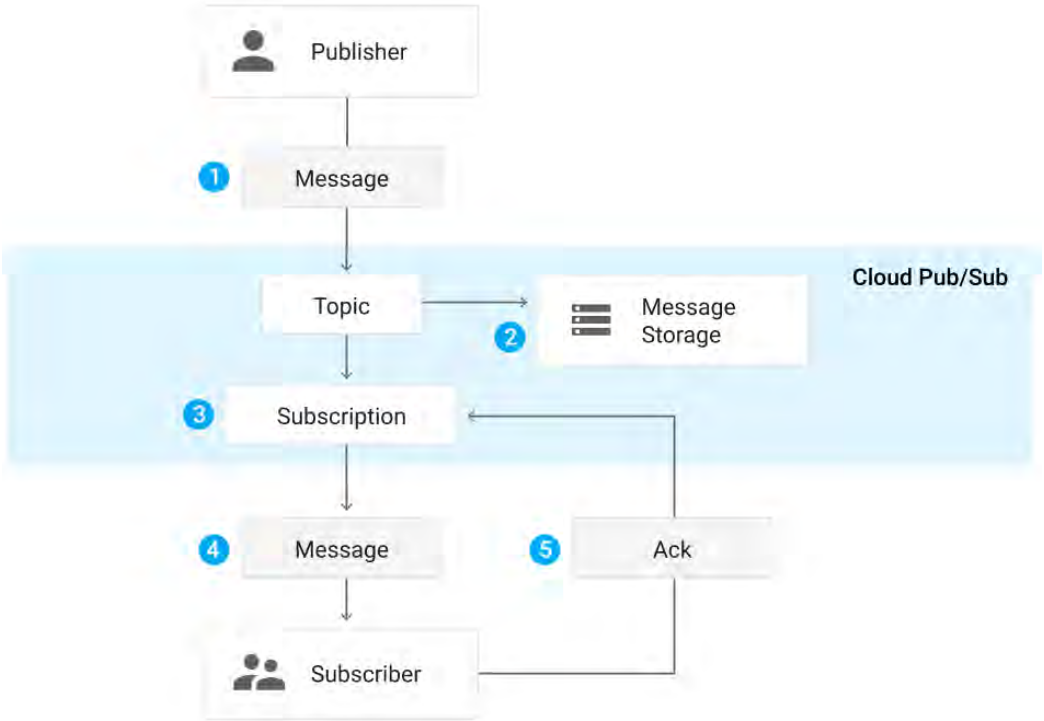

Figure 10-4 The steps of processing a message in Pub/Sub. Note the use of Ack to acknowledge the delivery of the message to the Subscriber. The message will persist in the message store until the acknowledgement is received from all subscribers.

10.3 Message delivery example

Having looked at the overall philosophy and operation of the Pub/Sub service, now we look at a small example illustrating them in practice. The example consists of a Background Function that will be triggered by a Pub/Sub message.

The source code for the Function is below. It will receive a message from Pub/Sub, extracts its content and inserts it into the "Hello …" output string providing a personalised greetings.

```
import com.google.cloud.functions.BackgroundFunction;
import com.google.cloud.functions.Context;
import functions.eventpojos.PubSubMessage;
import java.nio.charset.StandardCharsets;
import java.util.Base64;
import java.util.logging.Level;
```

```
import java.util.logging.Logger;

public class HelloPubSub implements BackgroundFunction<PubSubMessage> {
  private static final Logger logger = Logger.getLogger(HelloPubSub.class.getName());

  @Override
  public void accept(PubSubMessage message, Context context) {
    String name = "world";
    if (message != null && message.getData() != null) {
      name = new String(
          Base64.getDecoder().decode(message.getData().getBytes(StandardCharsets.UTF_8)),
          StandardCharsets.UTF_8);
    }
    logger.info(String.format("Hello %s!", name));
    return;
  }
}
```

Once the code is complete, the Function is deployed in the cloud using the `gcloud` command below. The topic name in this example is *SAMPLE_TOPIC*. The topic name is specified in the deploy command using the `--trigger_topic` option that specifies the subscription to the Pub/Sub topic called *SAMPLE_TOPIC*.

```
gcloud functions deploy java-pubsub-function \
--entry-point functions.HelloPubSub \
--runtime java11 \
--memory 512MB \
--trigger-topic SAMPLE_TOPIC
```

After deploying the Function, we create the topic with the following command.

```
gcloud pubsub topics create SAMPLE_TOPIC
```

Finally, we test the execution of our sample by sending a test message with the content "Einstein" to SAMPLE_TOPIC. For simplicity, the gcloud command is used for generating the message. In the next section we will look at how to send and receive messages from the application code.

```
gcloud pubsub topics publish SAMPLE_TOPIC --message Einstein
```

10.4 Publishing and receiving messages in code

We will now look at how messages can be published and received from code using the Java Pub/Sub API. The first code section will illustrate the publishing of a message.

First, the topic name is retrieved by the topic and project IDs, then a publisher object is created. This is followed by creating a new message with the appropriate data fields. After this, the message is published. The returned future object is passed to an asynchronous callback object that will be accessed when the message is published or when the publish operation has failed. The program finishes with shutting down the publisher.

Publishing a Pub/Sub message

```
import com.google.api.core.ApiFuture;
import com.google.api.core.ApiFutureCallback;
import com.google.api.core.ApiFutures;
import com.google.api.gax.rpc.ApiException;
import com.google.cloud.pubsub.v1.Publisher;
```

```
import com.google.common.util.concurrent.MoreExecutors;
import com.google.protobuf.ByteString;
import com.google.pubsub.v1.PubsubMessage;
import com.google.pubsub.v1.TopicName;
import java.io.IOException;
import java.util.Arrays;
import java.util.List;
import java.util.concurrent.TimeUnit;

public class PublishWithErrorHandlerExample {

  public static void main(String... args) throws Exception {
    // TODO(developer): Replace these variables before running the sample.
    String projectId = "your-project-id";
    String topicId = "your-topic-id";

    publishWithErrorHandlerExample(projectId, topicId);
  }

  public static void publishWithErrorHandlerExample(String projectId, String topicId)
      throws IOException, InterruptedException {
    TopicName topicName = TopicName.of(projectId, topicId);
    Publisher publisher = null;

    try {
      // Create a publisher instance with default settings bound to the topic
      publisher = Publisher.newBuilder(topicName).build();

      List<String> messages = Arrays.asList("first message", "second message");

      for (final String message : messages) {
        ByteString data = ByteString.copyFromUtf8(message);
        PubsubMessage pubsubMessage = PubsubMessage.newBuilder().setData(data).build();

        // Once published, returns a server-assigned message id (unique within the topic)
        ApiFuture<String> future = publisher.publish(pubsubMessage);

        // Add an asynchronous callback to handle success / failure
        ApiFutures.addCallback(
            future,
            new ApiFutureCallback<String>() {

              @Override
              public void onFailure(Throwable throwable) {
                if (throwable instanceof ApiException) {
                  ApiException apiException = ((ApiException) throwable);
                  // details on the API exception
                  System.out.println(apiException.getStatusCode().getCode());
                  System.out.println(apiException.isRetryable());
                }
                System.out.println("Error publishing message : " + message);
              }

              @Override
              public void onSuccess(String messageId) {
                // Once published, returns server-assigned message ids (unique within
                            the topic)
                System.out.println("Published message ID: " + messageId);
              }
            },
            MoreExecutors.directExecutor());
      }
    } finally {
      if (publisher != null) {
        // When finished with the publisher, shutdown to free up resources.
        publisher.shutdown();
```

```
        publisher.awaitTermination(1, TimeUnit.MINUTES);
      }
    }
  }
}
```

The second sample will illustrate how to receive a message in an application. There are two types of message delivery in Pub/Sub; the Pull and Push delivery. In the Pull style delivery, the subscriber must explicitly request the message from the Pub/Sub system. In the Push mode, the Pub/Sub system will send the message to the subscriber directly without waiting for request. The sequence diagrams of the Pull and Push message delivery are shown in Figure 10-5 and Figure 10-6, respectively.

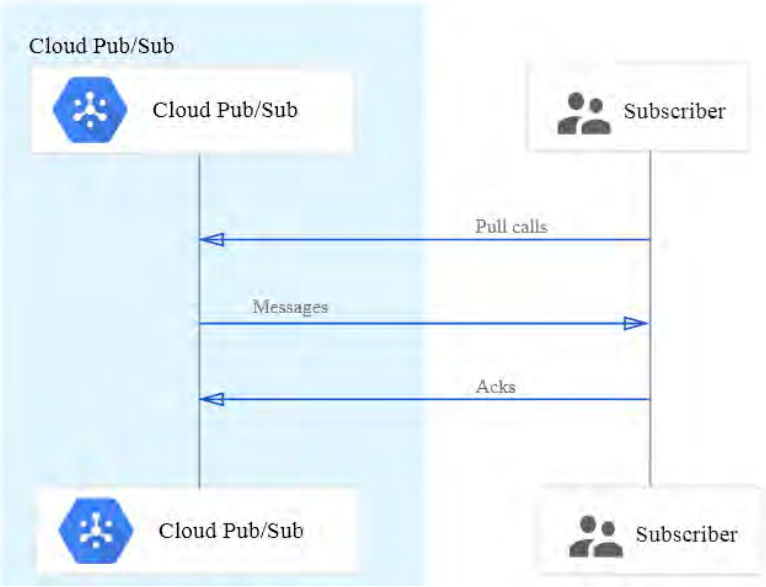

Figure 10-5 Sequence diagram of the Pull message delivery

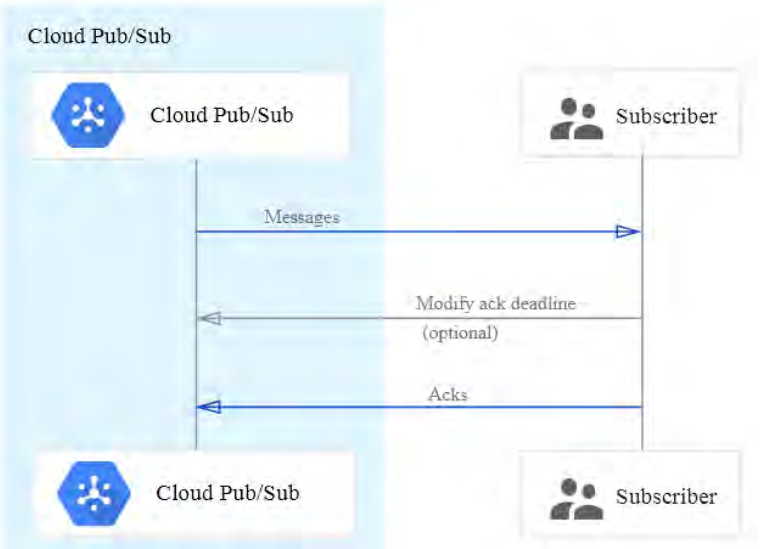

Figure 10-6 Sequence diagram of the Push message delivery

Our example will show only the Pull delivery implementation. The program first retrieves the name of the subscription using the supplied project and subscription IDs. Then a subscriber instance is created using the subscription name and a receiver object. The subscriber then is started to initiate the process of receiving messages. The subscriber will connect to the Pub/Sub system behind the scenes and request for messages. Whenever a message is received, the lambda expression of the receiver object passed to the subscriber as a parameter is activated, in which the content of the data is retrieved and the message delivery is acknowledged.

Receiving a Pub/Sub message

```
import com.google.cloud.pubsub.v1.AckReplyConsumer;
import com.google.cloud.pubsub.v1.MessageReceiver;
import com.google.cloud.pubsub.v1.Subscriber;
import com.google.pubsub.v1.ProjectSubscriptionName;
import com.google.pubsub.v1.PubsubMessage;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.TimeoutException;

public class SubscribeAsyncExample {
  public static void main(String... args) throws Exception {
    // TODO(developer): Replace these variables before running the sample.
    String projectId = "your-project-id";
    String subscriptionId = "your-subscription-id";

    subscribeAsyncExample(projectId, subscriptionId);
  }

  public static void subscribeAsyncExample(String projectId, String subscriptionId) {
    ProjectSubscriptionName subscriptionName =
        ProjectSubscriptionName.of(projectId, subscriptionId);

    // Instantiate an asynchronous message receiver.
    MessageReceiver receiver =
        (PubsubMessage message, AckReplyConsumer consumer) -> {
          // Handle incoming message, then ack the received message.
          System.out.println("Id: " + message.getMessageId());
          System.out.println("Data: " + message.getData().toStringUtf8());
          consumer.ack();
        };

    Subscriber subscriber = null;
    try {
      subscriber = Subscriber.newBuilder(subscriptionName, receiver).build();
      // Start the subscriber.
      subscriber.startAsync().awaitRunning();
      System.out.printf("Listening for messages on %s:\n", subscriptionName.toString());
      // Allow the subscriber to run for 30s unless an unrecoverable error occurs.
      subscriber.awaitTerminated(30, TimeUnit.SECONDS);
    } catch (TimeoutException timeoutException) {
      // Shut down the subscriber after 30s. Stop receiving messages.
      subscriber.stopAsync();
    }
  }
}
```

11 GCP service examples

In the preceding chapters we looked at different IaaS level services in the Google Cloud Platform. We also discussed how to execute large computational and data processing jobs in batch and streaming fashion. We learned about Cloud Functions and the Pub/Sub publish-subscribe message/event delivery system. The Google Cloud Platform offers, however, several high-level, ready-to-use service components that can be used in cloud applications significantly increasing the capabilities of these applications and – at the same time – drastically reducing development effort.

The most exciting examples of these services are in the computationally complex and difficult areas such as artificial intelligence and man-machine interfacing. In this chapter, we will provide a brief overview and introduction to some of these services. Due to the ever-increasing number and functionality of these services, the treatment is in no way complete; we are selecting some of the most interesting ones and describe their capabilities and their programming APIs. The goal is just to give you a taste of the possibilities.

The GCP AI and Machine Learning products include the following services at the time of writing:

AutoML	Service to provide Machine Learning functionality building on top of Google's machine learning capabilities. Custom ML models can be easily created for several application areas, such as text processing (AutoML Natural Language), image classification and object detection (AutoML Vision), video intelligence and object tracking (AutoML Video Intelligence), data analysis (AutoML Tables).
Vision AI	Service to provide image detection and recognition features without training (using custom or pre-trained models), including text extraction and recognition (OCR), image labelling, face and landmark detection, content tagging.
Video AI	Service enabling powerful content discovery, detection, object tracking and shot analysis in video sequences.
Cloud Natural Language	Service providing natural language understanding technologies to developers, including sentiment analysis, entity analysis, entity sentiment analysis, content classification, and syntax analysis to derive insights from unstructured text.
Cloud Translation	Service to dynamically translate between arbitrary language pairs extending application and web site usability for the global market.
Text-to-Speech	Service that converts text or Speech Synthesis Markup Language (SSML) input into natural-sounding speech using ML.
Speech-to-Text	This service enables easy integration of Google speech recognition technologies into developer applications. You send audio to the service and receive a text transcription from the service.
Dialogflow	Service to help building engaging voice and text-based conversational interfaces based on conversational agents.

From this large set of services, we will look at the Vision AI, Cloud Translation and the Text-to-Speech services in more detail.

11.1 Vision AI

The Vision AI Java API contains hundreds of classes as part of the package `com.google.cloud.vision.v1`. Thankfully, only a handful are really necessary to start performing exciting operations on images. We will focus on image detection tasks. The key class in this case is the class `ImageAnnotatorClient`. This class has several `...Annotate...()` methods that perform the detection task on local or remote files.

The first step of the process is to create an `ImageAnnotatorClient` instance as shown below. Using this instance as the Vision API client, we can perfom the detection by invoking the `batchAnnotateImages()` method. Once the detection is performed, a response object is returned containing a list of detected objects. These can be retrieved individually and used as required in subsequent parts of the program.

```
// Initialize client that will be used to send requests.
// This client only needs to be created once, and can be reused for multiple
// requests. After completing all of your requests, call
// the "close" method on the client to safely clean up any remaining background
resources.
try (ImageAnnotatorClient client = ImageAnnotatorClient.create()) {
   BatchAnnotateImagesResponse response = client.batchAnnotateImages(requests);
   List<AnnotateImageResponse> responses = response.getResponsesList();
   ...
   // process responses
   ...
```

The `batchAnnotateImages()` method requires a request list parameter of type `List<AnnotateImageRequest>`. This list must contain annotation request objects holding detectable features we are looking for in the images. Each individual feature is created using a specific builder object. Feature types are define in the class Feature. In the example below, the feature we select is TEXT_DETECTION. Variable `img` is the target image (local or remote) on which the detection should be performed.

```
List<AnnotateImageRequest> requests = new ArrayList<>();
Feature feat = Feature.newBuilder().setType(Feature.Type.TEXT_DETECTION).build();
AnnotateImageRequest request =
        AnnotateImageRequest.newBuilder().addFeatures(feat).setImage(img).build();
requests.add(request);
```

Other types of features supported in the current API include: CROP_HINTS, DOCUMENT_TEXT_DETECTION, FACE_DETECTION, IMAGE_PROPERTIES (compute a set of image properties, such as the image's dominant colours), LABEL_DETECTION, LANDMARK_DETECTION, LOGO_DETECTION, OBJECT_LOCALIZATION, PRODUCT_SEARCH, SAFE_SEARCH_DETECTION (detect potentially unsafe or undesirable content), TEXT_DETECTION.

For instance, if we wanted to detect not only text but handwritten text and Logos as well, our feature builder code would look like as follows.

```
List<AnnotateImageRequest> requests = new ArrayList<>();
Feature feat1 = Feature.newBuilder().setType(Feature.Type.TEXT_DETECTION).build();
AnnotateImageRequest request1 =
        AnnotateImageRequest.newBuilder().addFeatures(feat1)
             .setImage(img).build();
Feature feat2 = Feature.newBuilder().setType(Feature.Type.DOCUMENT_TEXT_DETECTION)
```

```
            .build();
AnnotateImageRequest request2 = AnnotateImageRequest.newBuilder()
       .addFeatures(feat2).setImage(img).build();
Feature feat3 = Feature.newBuilder().setType(Feature.Type.LOGO_DETECTION).build();
AnnotateImageRequest request3 = AnnotateImageRequest.newBuilder()
       .addFeatures(feat3).setImage(img).build();

requests.add(request1);
requests.add(request2);
requests.add(request3);
```

11.2 Cloud Translation

The cloud Translation Java API classes are part of the package `com.google.cloud.translate`. The entry point to using the API is interface `Translate`. This represents a service proxy that can be used to perform the required translation. In the example below, a default translate service is retrieved using class `TranslateOptions`. Once the reference to the service is obtained. The `translate()` method is invoked to execute the operation. The resulting Translation object contains the translated text.

```
import com.google.cloud.translate.*;
Translate translate = TranslateOptions.getDefaultInstance().getService();

Translation translation = translate.translate("¡Hola Mundo!");
System.out.printf("Translated Text:\n\t%s\n", translation.getTranslatedText());
```

It is possible to use more specific parameters for the translation. In the previous example, the language detection is automatic. If we want, we can set the source and target languages explicitly as follows (in this case, translating from Spanish to German).

```
Translation translation =
    translate.translate(
        "Hola Mundo!",
        Translate.TranslateOption.sourceLanguage("es"),
        Translate.TranslateOption.targetLanguage("de"),
        // Use "base" for standard edition, "nmt" for the premium model.
        Translate.TranslateOption.model("nmt"));
```

The Translation API has an Advanced version that includes additional features, e.g. the possibility to create translation glossaries, performing batched translations for large amounts of texts in a manner similar to the code skeleton shown in the image detection sample using class `TranslationServiceClient` and the method call `client.translateText(request)`. The details of the advanced mode can be found in the documentation of the Translation service (https://cloud.google.com/translate/docs/basic/quickstart).

11.3 Text-to-Speech

The Text-to-Speech service is used to convert text to human sounding speech using Google's speech generation engines. The service supports nearly all major languages of the world based with male and female voices using selectable speech generation technologies. The full list of languages can be found at the following link: https://cloud.google.com/text-to-speech/docs/voices.

The Java are part of package `com.google.cloud.texttospeech.v1`. The main class for using the service is class `TextToSpeechClient`. The following sample creates an instance of this class, then builds a `SynthesisInput` object with the text "Hello World" as the source text. In the following step, the translation parameters (target language, voice) are set using the methods of the class `VoiceSelectionParams`. After specifying the format of the output audio file, the method `synthesizeSpeech` is called on the `textToSpeechClient` object. The program finally saves the result into an output file.

```
// Imports the Google Cloud client library
import com.google.cloud.texttospeech.v1.AudioConfig;
import com.google.cloud.texttospeech.v1.AudioEncoding;
import com.google.cloud.texttospeech.v1.SsmlVoiceGender;
import com.google.cloud.texttospeech.v1.SynthesisInput;
import com.google.cloud.texttospeech.v1.SynthesizeSpeechResponse;
import com.google.cloud.texttospeech.v1.TextToSpeechClient;
import com.google.cloud.texttospeech.v1.VoiceSelectionParams;
import com.google.protobuf.ByteString;
import java.io.FileOutputStream;
import java.io.OutputStream;

/**
 * Google Cloud TextToSpeech API sample application. Example usage: mvn package
exec:java
 * -Dexec.mainClass='com.example.texttospeech.QuickstartSample'
 */
public class QuickstartSample {

  /** Demonstrates using the Text-to-Speech API. */
  public static void main(String... args) throws Exception {
    // Instantiates a client
    try (TextToSpeechClient textToSpeechClient = TextToSpeechClient.create()) {
      // Set the text input to be synthesized
      SynthesisInput input = SynthesisInput.newBuilder().setText(
                                "Hello, World!").build();

      // Build the voice request, select the language code ("en-US")
      // and the ssml voice gender
      // ("neutral")
      VoiceSelectionParams voice =
          VoiceSelectionParams.newBuilder()
              .setLanguageCode("en-US")
              .setSsmlGender(SsmlVoiceGender.NEUTRAL)
              .build();

      // Select the type of audio file you want returned
      AudioConfig audioConfig =
          AudioConfig.newBuilder().setAudioEncoding(AudioEncoding.MP3).build();

      // Perform the text-to-speech request on the text input with
      // the selected voice parameters and
      // audio file type
      SynthesizeSpeechResponse response =
          textToSpeechClient.synthesizeSpeech(input, voice, audioConfig);

      // Get the audio contents from the response
      ByteString audioContents = response.getAudioContent();
```

```
      // Write the response to the output file.
      try (OutputStream out = new FileOutputStream("output.mp3")) {
        out.write(audioContents.toByteArray());
        System.out.println("Audio content written to file \"output.mp3\"");
      }
    }
  }
}
```

12 Cloud Service Orchestration

The purpose of this section is to synthesise what we discussed in the preceding chapters and see how cloud technology enables us to create useful, feature-rich and reliable applications very efficiently serving a large number of users. We can create services for providing specific functionality or use services from cloud providers or third party developers to be used as components in future applications. Integration technologies are also provided by vendors to connect these components together in a simple and effective manner, allowing the operation of distributed cloud applications at a global scale.

In the Google Cloud Platform, the key technologies we can use are the followings: infrastructure services (Storage and computation based on Dataproc or Dataflow), Cloud Functions and Pub/Sub for integration and application specific cloud services for providing task-specific functionalities.

The integration of services or service components in this manner is often called orchestration that refers to the coordinated use and execution of components/services for the execution of a complex task. These complex tasks are typically described as workflow graphs executing a series of processing steps in a loosely coupled manner.

In this chapter, we introduce a sample program that demonstrates elegance of cloud service integration and the use of the various technologies studied so far.

12.1 A complex Optical Character Recognition application example

The program we describe here is part of the Google Cloud Platform documentation[17]. The goal of the application is to examine images uploaded to the cloud, extract text content from them, translate the text to a given language and save the result into a location in the cloud. The workflow and the architecture of the application is shown in Figure 12-1.

The order of execution is the following:

Step 1: An image is uploaded to Google Storage

Step 2: A cloud function is triggered that calls the Vision API to extract the text content of the image

Step 3: The extracted text is written into a Pub/Sub topic

Step 4: A cloud function is triggered by the topic message that invokes the Translation service

Step 5: The translated text messages are written in the Pub/Sub topic

Step 6: A cloud function is triggered to store results in Google Storage

In the rest of this chapter, the implementation details of the above steps and the execution of the program is described.

[17] Further details can be found at https://cloud.google.com/functions/docs/tutorials/ocr

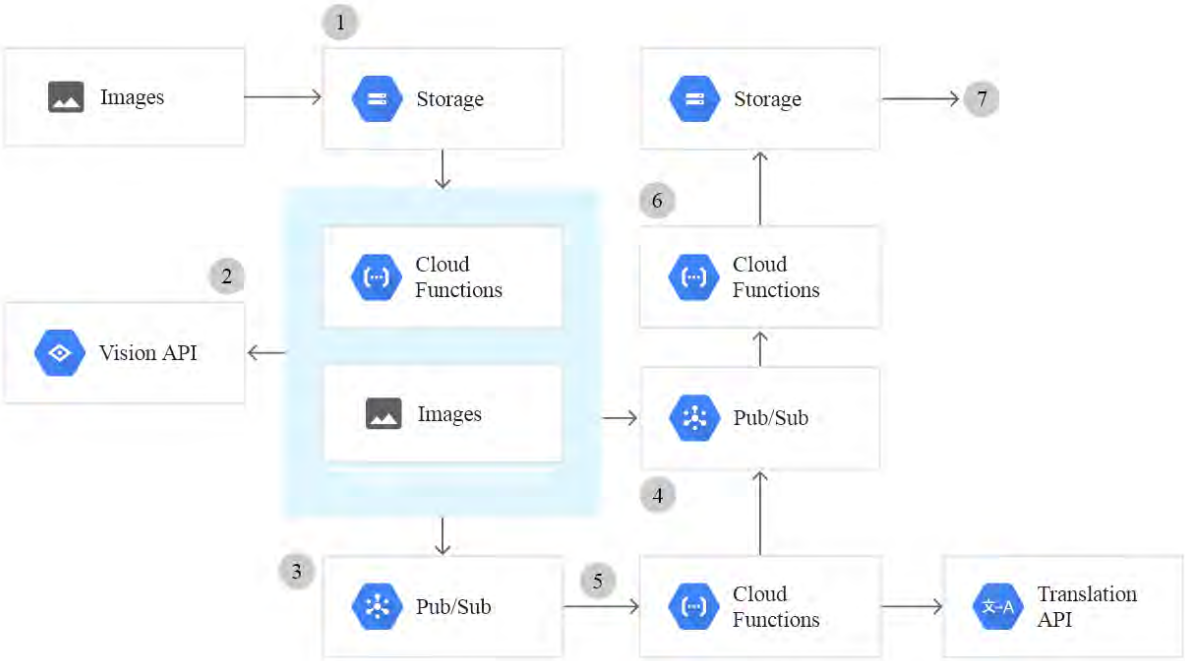

Figure 12-1 The workflow and the architecture of the optical Character Recognition and Translation application

Function to extract text

The following code section implements a background function triggered by a Storage event (executing the method `accept()`) that executes a text detection task and publishes a Pub/Sub message to a topic. The `accept()` method retrieves the file from the Storage bucket and calls `detectText()`. This method is where the main processing steps happen. First, an `ImageAnnotatorClient` instance is created, then text detection is executed by invoking the method `batchAnnotateImages` with the input feature request set to `Feature.Type.TEXT_DETECTION`. Following this, the Vision APi is used again to detect the possible languages the text in the image is written to. For this, an instance of the `TranslationServiceClient` class is created on which the method `detectLanguage` is called. As the last step, for all detected languages, a translation message is published to the Pub/Sub publisher created in the constructor using the `publish` method.

Function to perform text and language detection on images

```
import com.google.cloud.functions.BackgroundFunction;
import com.google.cloud.functions.Context;
import com.google.cloud.pubsub.v1.Publisher;
import com.google.cloud.translate.v3.DetectLanguageRequest;
import com.google.cloud.translate.v3.DetectLanguageResponse;
import com.google.cloud.translate.v3.LocationName;
import com.google.cloud.translate.v3.TranslationServiceClient;
import com.google.cloud.vision.v1.AnnotateImageRequest;
import com.google.cloud.vision.v1.AnnotateImageResponse;
import com.google.cloud.vision.v1.Feature;
import com.google.cloud.vision.v1.Image;
import com.google.cloud.vision.v1.ImageAnnotatorClient;
import com.google.cloud.vision.v1.ImageSource;
import com.google.protobuf.ByteString;
import com.google.pubsub.v1.ProjectTopicName;
import com.google.pubsub.v1.PubsubMessage;
import functions.eventpojos.GcsEvent;
```

```
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.ExecutionException;
import java.util.logging.Level;
import java.util.logging.Logger;

public class OcrProcessImage implements BackgroundFunction<GcsEvent> {
  // TODO<developer> set these environment variables
  private static final String PROJECT_ID = System.getenv("GCP_PROJECT");
  private static final String TRANSLATE_TOPIC_NAME = System.getenv("TRANSLATE_TOPIC");
  private static final String[] TO_LANGS = System.getenv("TO_LANG").split(",");

  private static final Logger logger = Logger.getLogger(OcrProcessImage.class.getName());
  private static final String LOCATION_NAME = LocationName.of(PROJECT_ID,
"global").toString();
  private Publisher publisher;

  public OcrProcessImage() throws IOException {
    publisher = Publisher.newBuilder(
        ProjectTopicName.of(PROJECT_ID, TRANSLATE_TOPIC_NAME)).build();
  }


  @Override
  public void accept(GcsEvent gcsEvent, Context context) {

    // Validate parameters
    String bucket = gcsEvent.getBucket();
    if (bucket == null) {
      throw new IllegalArgumentException("Missing bucket parameter");
    }
    String filename = gcsEvent.getName();
    if (filename == null) {
      throw new IllegalArgumentException("Missing name parameter");
    }

    detectText(bucket, filename);
  }
}

private void detectText(String bucket, String filename) {
  logger.info("Looking for text in image " + filename);

  List<AnnotateImageRequest> visionRequests = new ArrayList<>();
  String gcsPath = String.format("gs://%s/%s", bucket, filename);

  ImageSource imgSource = ImageSource.newBuilder().setGcsImageUri(gcsPath).build();
  Image img = Image.newBuilder().setSource(imgSource).build();

  Feature textFeature = Feature.newBuilder().setType(Feature.Type.TEXT_DETECTION).build();
  AnnotateImageRequest visionRequest =
      AnnotateImageRequest.newBuilder().addFeatures(textFeature).setImage(img).build();
  visionRequests.add(visionRequest);

  // Detect text in an image using the Cloud Vision API
  AnnotateImageResponse visionResponse;
  try (ImageAnnotatorClient client = ImageAnnotatorClient.create()) {
    visionResponse = client.batchAnnotateImages(visionRequests).getResponses(0);
    if (visionResponse == null || !visionResponse.hasFullTextAnnotation()) {
      logger.info(String.format("Image %s contains no text", filename));
      return;
    }

    if (visionResponse.hasError()) {
      // Log error
```

```
      logger.log(Level.SEVERE, "Error in vision API call: " +
                  visionResponse.getError().getMessage());
      return;
    }
  } catch (IOException e) {
    // Log error (since IOException cannot be thrown by a Cloud Function)
    logger.log(Level.SEVERE, "Error detecting text: " + e.getMessage(), e);
    return;
  }

  String text = visionResponse.getFullTextAnnotation().getText();
  logger.info("Extracted text from image: " + text);

  // Detect language using the Cloud Translation API
  DetectLanguageRequest languageRequest =
      DetectLanguageRequest.newBuilder()
          .setParent(LOCATION_NAME)
          .setMimeType("text/plain")
          .setContent(text)
          .build();
  DetectLanguageResponse languageResponse;
  try (TranslationServiceClient client = TranslationServiceClient.create()) {
    languageResponse = client.detectLanguage(languageRequest);
  } catch (IOException e) {
    // Log error (since IOException cannot be thrown by a function)
    logger.log(Level.SEVERE, "Error detecting language: " + e.getMessage(), e);
    return;
  }

  if (languageResponse.getLanguagesCount() == 0) {
    logger.info("No languages were detected for text: " + text);
    return;
  }

  String languageCode = languageResponse.getLanguages(0).getLanguageCode();
  logger.info(String.format("Detected language %s for file %s", languageCode, filename));

  // Send a Pub/Sub translation request for every language we're going to translate to
  for (String targetLanguage : TO_LANGS) {
    logger.info("Sending translation request for language " + targetLanguage);
    OcrTranslateApiMessage message = new OcrTranslateApiMessage(
                  text, filename, targetLanguage);
    ByteString byteStr = ByteString.copyFrom(message.toPubsubData());
    PubsubMessage pubsubApiMessage = PubsubMessage.newBuilder().setData(byteStr).build();
    try {
      publisher.publish(pubsubApiMessage).get();
    } catch (InterruptedException | ExecutionException e) {
      // Log error
      logger.log(Level.SEVERE, "Error publishing translation request: " + e.getMessage(),
e);
      return;
    }
  }
}
```

The next step of the application is to actually perform the translations. The class `OcrTranslateText` below implements a background function that responds to translation request messages. The accept method is invoked for each received message. As the first step of processing, the method extracts the source text and the target language code from the incoming message, then creates an instance of the `TranslationServiceClient` class. An appropriate translation request object is created then the text is translated by invoking the `translateText` method. The returned translated text and the name of

the output file is packaged into a new Pub/Sub message and sent to the cloud using the `publish` method on the publisher created in the constructor or the class.

Function to translate text

```
import com.google.cloud.functions.BackgroundFunction;
import com.google.cloud.functions.Context;
import com.google.cloud.pubsub.v1.Publisher;
import com.google.cloud.translate.v3.LocationName;
import com.google.cloud.translate.v3.TranslateTextRequest;
import com.google.cloud.translate.v3.TranslateTextResponse;
import com.google.cloud.translate.v3.TranslationServiceClient;
import com.google.protobuf.ByteString;
import com.google.pubsub.v1.ProjectTopicName;
import com.google.pubsub.v1.PubsubMessage;
import functions.eventpojos.PubSubMessage;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.util.concurrent.ExecutionException;
import java.util.logging.Level;
import java.util.logging.Logger;

public class OcrTranslateText implements BackgroundFunction<PubSubMessage> {
  private static final Logger logger = Logger.getLogger(OcrTranslateText.class.getName());

  // TODO<developer> set these environment variables
  private static final String PROJECT_ID = getenv("GCP_PROJECT");
  private static final String RESULTS_TOPIC_NAME = getenv("RESULT_TOPIC");
  private static final String LOCATION_NAME = LocationName.of(
                                              PROJECT_ID, "global").toString();

  private Publisher publisher;

  public OcrTranslateText() throws IOException {
    publisher = Publisher.newBuilder(
        ProjectTopicName.of(PROJECT_ID, RESULTS_TOPIC_NAME)).build();
  }

  @Override
  public void accept(PubSubMessage pubSubMessage, Context context) {
    OcrTranslateApiMessage ocrMessage = OcrTranslateApiMessage.fromPubsubData(
        pubSubMessage.getData().getBytes(StandardCharsets.UTF_8));

    String targetLang = ocrMessage.getLang();
    logger.info("Translating text into " + targetLang);

    // Translate text to target language
    String text = ocrMessage.getText();
    TranslateTextRequest request =
        TranslateTextRequest.newBuilder()
            .setParent(LOCATION_NAME)
            .setMimeType("text/plain")
            .setTargetLanguageCode(targetLang)
            .addContents(text)
            .build();

    TranslateTextResponse response;
    try (TranslationServiceClient client = TranslationServiceClient.create()) {
      response = client.translateText(request);
    } catch (IOException e) {
      // Log error (since IOException cannot be thrown by a function)
      logger.log(Level.SEVERE, "Error translating text: " + e.getMessage(), e);
      return;
    }
    if (response.getTranslationsCount() == 0) {
      return;
```

```
    }

    String translatedText = response.getTranslations(0).getTranslatedText();
    logger.info("Translated text: " + translatedText);

    // Send translated text to (subsequent) Pub/Sub topic
    String filename = ocrMessage.getFilename();
    OcrTranslateApiMessage translateMessage = new OcrTranslateApiMessage(
        translatedText, filename, targetLang);
    try {
      ByteString byteStr = ByteString.copyFrom(translateMessage.toPubsubData());
      PubsubMessage pubsubApiMessage = PubsubMessage.newBuilder().setData(byteStr).build();

      publisher.publish(pubsubApiMessage).get();
      logger.info("Text translated to " + targetLang);
    } catch (InterruptedException | ExecutionException e) {
      // Log error (since these exception types cannot be thrown by a function)
      logger.log(Level.SEVERE, "Error publishing translation save request: " +
e.getMessage(), e);
    }
  }

  // Avoid ungraceful deployment failures due to unset environment variables.
  // If you get this warning you should redeploy with the variable set.
  private static String getenv(String name) {
    String value = System.getenv(name);
    if (value == null) {
      logger.warning("Environment variable " + name + " was not set");
      value = "MISSING";
    }
    return value;
  }
}
```

The last Function in the application is responsible for storing the translation results in a Storage bucket. A reference to the Storage service client is obtained in the static block of the class. Once the `accept` method is invoked, the name of the output file, and language code is extracted from the incoming message, from which a new file name is created by concatenation. The translated text then is saved into this new file be invoking the `Storage.create` method.

Function to save results in Storage

```
import com.google.cloud.functions.BackgroundFunction;
import com.google.cloud.functions.Context;
import com.google.cloud.storage.BlobId;
import com.google.cloud.storage.BlobInfo;
import com.google.cloud.storage.Storage;
import com.google.cloud.storage.StorageOptions;
import functions.eventpojos.PubSubMessage;
import java.nio.charset.StandardCharsets;
import java.util.logging.Logger;

public class OcrSaveResult implements BackgroundFunction<PubSubMessage> {
  // TODO<developer> set this environment variable
  private static final String RESULT_BUCKET = System.getenv("RESULT_BUCKET");

  private static final Storage STORAGE = StorageOptions.getDefaultInstance().getService();
  private static final Logger logger = Logger.getLogger(OcrSaveResult.class.getName());

  @Override
  public void accept(PubSubMessage pubSubMessage, Context context) {
    OcrTranslateApiMessage ocrMessage = OcrTranslateApiMessage.fromPubsubData(
```

```
        pubSubMessage.getData().getBytes(StandardCharsets.UTF_8));

    logger.info("Received request to save file " +  ocrMessage.getFilename());

    String newFileName = String.format(
        "%s_to_%s.txt", ocrMessage.getFilename(), ocrMessage.getLang());

    // Save file to RESULT_BUCKET with name newFileNaem
    logger.info(String.format("Saving result to %s in bucket %s", newFileName,
RESULT_BUCKET));
    BlobInfo blobInfo = BlobInfo.newBuilder(BlobId.of(RESULT_BUCKET, newFileName)).build();
    STORAGE.create(blobInfo, ocrMessage.getText().getBytes(StandardCharsets.UTF_8));
    logger.info("File saved");
  }
}
```

To complete the application, the above functions must be deployed in the cloud, followed by the creation of the input and output Storage bucket as well as Pub/Sub topics. Once these steps are complete, an upload of an image file to the input bucket will trigger the execution of the application.

This sample illustrated the key concepts of cloud service orchestration as well as the simplicity and elegance of the use of loosely coupled message/event communication among services. The developer does not have to deal with infrastructure management details, can concentrate on the application logic only. Thanks to the automatic scaling property of Cloud Functions, the application will be able to service arbitrary number of clients without any further intervention.

The interested reader should consult the Google Cloud documentation for further samples on service orchestration.

13 Conclusions

The goal of this book was to provide background material for the Cloud Programming MSc course at the University of Pannonia. As such, it was not meant to be a definitive work covering all theoretical and practical aspects of cloud computing. The focus of this course is on programming cloud systems in contrast to creating and managing cloud infrastructures, hence system management topics were largely ignored. It was also not meant to provide full coverage on programming at the level of detail that would be required for professional cloud developers. Nonetheless, I hope it serves as a useful introduction for the key concepts and provides enough practical programming details to get interest students started in the very exciting cloud computing world.

We know that the future can never be predicted accurately but we can say with a high level of confidence that the role and capability of cloud systems will continually expand in the coming years. This will have important consequences for developers and the entire software industry. As web technology had a major impact on desktop and mobile applications, we can expect that cloud will have a similarly significant effect on future software. It is expected from applications already to integrate with the cloud at least for input/output operations so we can process our data stored in the cloud or save our results in the cloud. Future applications will rely on cloud services more and more, leading to true cloud-based applications built from cloud service components.

Cloud vendors will do their best to improve further the availability, reliability and security of their systems, and the increasing competition and the use of clean energy are expected to reduce prices in the long term, making cloud systems and cloud computing an affordable and trusted platform for mankind. There are several trends that make future cloud computing even more interesting that it is today. High-performance computing and artificial intelligence are finding their way into cloud systems. It is expected that cloud vendors will soon provide on-demand supercomputing resources and access to infrastructure designed specifically for AI workloads. This will provide a platform for innovation and possibly will lead to the development of new services, even industries. Internet-of-Things based data analytics is expected to revolutionise manufacturing and logistics. When used with body sensors coupled with cloud-based health-care and diagnostics, cloud computing can revolutionise our health-care system as well.

These were just a few examples or glimpses of what the future could bring in cloud computing, but we can say that this future is certainly bright and exciting. So, do not get left behind then, stay tuned and become part of the future!

Bibliography

[1] L. Dagum and R. Menon, "OpenMP: an industry standard API for shared-memory programming," *IEEE Comput. Sci. Eng.*, vol. 5, no. 1, pp. 46–55, 1998.

[2] W. Gropp, *MPI : the complete reference. Vol. 2, The MPI-2 extensions*. Cambridge, Mass. ; M.I.T., 1998.

[3] W. Gropp, E. L. Lusk, and A. Skjellum, *Using MPI : portable parallel programming with the Message-Passing Interface*. Cambridge, Mass. ; MIT Press, 1994.

[4] Nvidia, "CUDA C Programming Guide." [Online]. Available: https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html. [Accessed: 02-Feb-2019].

[5] R. Thurlow, "RPC: Remote Procedure Call Protocol Specification Version 2."

[6] B. J. N. Andrew D. Birrell, "Implementing remote procedure calls."

[7] J. Waldo, "Remote procedure calls and Java Remote Method Invocation," *IEEE Concurr.*, vol. 6, no. 3, pp. 5–7, Aug. 2002.

[8] V. S. Sunderam, "PVM: A Framework for Parallel Distributed Computing."

[9] A. S. Grimshaw and W. A. Wulf, "The Legion Vision of a Worldwide Virtual Computer," *Commun. ACM*, vol. 40, no. 1, pp. 39–45, Jan. 1997.

[10] I. Foster and C. Kesselman, "Globus: A metacomputing infrastructure toolkit," *Int. J. High Perform. Comput. Appl.*, vol. 11, no. 2, pp. 115–128, 1997.

[11] K. Czajkowski, I. Foster, N. Karonis, C. Kesselman, S. Tuecke, and M. Rey, "A Resource Management Architecture for Metacomputing Systems," pp. 1–19.

[12] E. J. Korpela, D. P. Anderson, and J. Cobb, "SETI@home-massively distributed computing for SETI," *Comput. Sci. Eng.*, vol. 3, no. 1, pp. 78–83, 2001.

[13] "Amazon Web Services (AWS) - Cloud Computing Services." [Online]. Available: https://aws.amazon.com/?nc2=h_lg. [Accessed: 15-Nov-2020].

[14] "Apache Hadoop." [Online]. Available: http://hadoop.apache.org/. [Accessed: 09-Jan-2021].

[15] "Apache Spark™ - Unified Analytics Engine for Big Data." [Online]. Available: http://spark.apache.org/. [Accessed: 09-Jan-2021].

[16] T. F. Chan, G. H. Golub, and R. J. Leveque, "Statistical computing: Algorithms for computing the sample variance: Analysis and recommendations," *Am. Stat.*, vol. 37, no. 3, pp. 242–247, 1983.

[17] B. P. Welford, "Note on a Method for Calculating Corrected Sums of Squares and Products," *Technometrics*, vol. 4, no. 3, pp. 419–420, 1962.

[18] "Spark Streaming | Apache Spark." [Online]. Available: https://spark.apache.org/streaming/. [Accessed: 09-Jan-2021].

[19] "Apache Flink: Stateful Computations over Data Streams." [Online]. Available: https://flink.apache.org/. [Accessed: 09-Jan-2021].

[20] "Apache Storm." [Online]. Available: http://storm.apache.org/index.html. [Accessed: 09-Jan-2021].

[21] "Apache Beam." [Online]. Available: https://beam.apache.org/. [Accessed: 09-Jan-2021].

[22] D. García-Gil, S. Ramírez-Gallego, S. García, and F. Herrera, “A comparison on scalability for batch big data processing on Apache Spark and Apache Flink,” *Big Data Anal.*, vol. 2, no. 1, p. 1, Dec. 2017.

[23] S. Chatterjee and C. Morin, “Experimental study on the performance and resource utilization of data streaming frameworks,” in *Proceedings - 18th IEEE/ACM International Symposium on Cluster, Cloud and Grid Computing, CCGRID 2018*, 2018, pp. 143–152.

[24] B. Blamey, A. Hellander, and S. Toor, “Apache spark streaming, kafka and harmonicio: a performance benchmark and architecture comparison for enterprise and scientific computing,” in *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, 2020, vol. 12093 LNCS, pp. 335–347.

[25] Z. Karakaya, A. Yazici, and M. Alayyoub, “A Comparison of Stream Processing Frameworks,” in *2017 International Conference on Computer and Applications, ICCA 2017*, 2017, pp. 1–12.

[26] “Spark Streaming vs Flink vs Storm vs Kafka Streams vs Samza : Choose Your Stream Processing Framework | by chandan prakash | Medium.” [Online]. Available: https://medium.com/@chandanbaranwal/spark-streaming-vs-flink-vs-storm-vs-kafka-streams-vs-samza-choose-your-stream-processing-91ea3f04675b. [Accessed: 02-Jan-2021].

[27] “Cloud Functions | Google Cloud.” [Online]. Available: https://cloud.google.com/functions#all-features. [Accessed: 02-Jan-2021].

[28] “AWS Lambda – Serverless Compute - Amazon Web Services.” [Online]. Available: https://aws.amazon.com/lambda/. [Accessed: 02-Jan-2021].